The Verse Programming Language

Language Specification — Version 0.1 (Draft)

File extension: `.vs` *· Command-line tool:* `verse`

Contents

1. Philosophy and design principles
2. Quick tour
3. Lexical structure and syntax
4. Types
5. Functions
6. Control flow
7. Pattern matching
8. Errors and missing values
9. Collections
10. Traits and generics
11. Concurrency
12. Modules
13. Resource management
14. Testing and docs
15. Standard library overview
16. Tooling
17. EBNF grammar sketch
18. Design notes
19. Verse vs other languages
20. Complete example programs
21. Review log and resolved decisions

---

1. Philosophy and design principles

Verse is a high-level, statically typed, general-purpose language whose single overriding goal is **readability**. A Verse program should read like a short, clear piece of prose: you should be able to show it to a colleague who has never seen the language and have them follow along.

Everything else in the language follows from a handful of principles.

1.1 Code is read far more often than it is written

Verse optimizes for the reader. Keywords are short English words (`mut`, `if`, `for`, `match`, `with`, `and`, `or`, `not`). Punctuation is used only where it carries real meaning. Blocks are marked by

indentation, so the visual shape of the code *is* its structure.

1.2 Minimal punctuation, maximal meaning

There are no semicolons, no braces around blocks, and no parentheses around conditions. The punctuation that remains stands for **one idea each**. Some marks are used in several places, but always for the same idea — the table lists every use honestly:

Symbol	Meaning
`( )`	"group": calls, parameters, tuples, grouping, variant fields
`[ ]`	"position": list literals, indexing, slicing, list patterns
`{ }`	"contents": map literals (`{"a": 1}`, empty `{:}`), set literals (`set{…}`), and — inside text — interpolation (`"{name}"`)
`< >`	"type parameter": generics
`->`	"produces": lambdas and match arms (nothing else)
`|`	"one of": variants, union and error types (`Int | Error`), pattern alternatives
`?`	"maybe missing or failed": `T?` (a type that may be missing), postfix `expr?` (stop early on error or missing), and the pattern `x?` (match a present value)
`:`	"has type" or "is named": type annotations, return types, record fields, named arguments, map entries, supertraits and generic constraints (`T: Order`), typed patterns (`n: Int`), field patterns (`age: 18`), loop labels (`outer:`), and interpolation format specifiers (`{price:.2}`)

1.3 Safe by default

- There is **no null**. A value that might be absent has an optional type `T?`, and the compiler makes you handle the absence.
- Errors are **values in the type**: a function that can fail says so in its signature (`Int | Error`), and failure must be handled or explicitly propagated with `?`. Silently discarding a fallible result is a compile-time error; `_ = …` is the visible way to ignore one on purpose.
- Types are **static but visually optional**: the compiler infers them — including function parameters and return types — and when it cannot decide, it stops and asks for one annotation at that exact spot. It never guesses.
- Bindings are **immutable unless marked `mut`**: `x = 1` binds a name that never changes, `mut count = 0` one that can. A name is bound once per function — there is no shadowing — so `x = …` on a name that already exists can only be an assignment to a `mut` binding.
- Every value has **exactly one owner** and is freed deterministically when its owner's scope ends. There is **no garbage collector**, no manual allocation, no pointer or reference you can write down, no dangling reference and no use-after-free. Functions *borrow* their arguments unless they say `mut` (exclusive, mutable) or `take` (consume).
- Integer overflow, out-of-bounds list indexing and division by zero **stop the program with a clear message** rather than silently producing garbage.
- Concurrency is **race-free by construction**: a task owns what it is given; sharing between tasks happens only through opt-in handles (channels, `Shared`, `Guarded`).

1.4 One obvious way

Verse prefers one clear construct per idea. There is one loop over collections (`for ... in`), one general branching construct for data (`match`), one way to mark public items (`export`), and one official formatter (`verse fmt`) whose output is the canonical style.

1.5 Scripts grow into programs

A Verse file is a script: statements run from top to bottom, no ceremony and no `main` function required. The same file can grow functions, types, tests and modules without ever being rewritten into a "real program" shape.

1.6 Tests and documentation are part of the language

Doc comments (`##`) and test blocks (`test "name"`) are first-class syntax, not conventions. The tooling understands them directly.

1.7 Notation in this document

Code blocks are Verse unless marked otherwise. Inside examples, `...` stands for code that has been left out for brevity; it is not Verse syntax. Comments of the form `# 3` show what a line prints or evaluates to.

---

2. Quick tour

The following program shows most of Verse on one page. It reads a list of people, greets them, and summarizes some facts.

```
## A small tour of Verse.

import read_text from std.fs

type Person
    name: Text
    age: Int
    email: Text? = none

type Shape = Circle(radius: Float) | Square(side: Float)

# Types are optional. This function is fully inferred…
fn greet(person, greeting = "Hello")
    "{greeting}, {person.name}!"

# …and this one spells its types out. Both are statically checked.
fn area(shape: Shape): Float
    match shape
        Circle(r) -> 3.14 * r * r
        Square(s) -> s * s

fn parse_age(text: Text): Int | Error
    age = text.trim().to_int()?
    if age < 0
        return error("age cannot be negative: {age}")
    age

# `mut` borrows exclusively; `take` consumes the argument.
fn add_guest(mut list, take guest)
    list.push(guest)

mut people = [
    Person(name: "Ana", age: 30, email: "ana@example.com"),
    Person(name: "Bruno", age: 17),
]
add_guest(mut people, Person(name: "Carla", age: 41))

for person in people                      # each person is borrowed
    print(greet(person, greeting: "Welcome"))
    print("  contact: {person.email otherwise "no email"}")

adults = people.filter(p -> p.age >= 18).map(.name.copy())
label = if adults.len() == 1 then "adult" else "adults"
print("{adults.len()} {label}: {adults.join(", ")}")

bruno_email = people.find(p -> p.name == "Bruno").email otherwise "none"
print("Bruno's email: {bruno_email}")

total = [Circle(radius: 1.0), Square(side: 2.0)].map(area).sum()
print("Total area: {total:.2}")

mut count = 0                             # `x = …` binds once; `mut` can change
for i in 1..=10
    if i % 2 == 0
        count += 1
print("{count} even numbers between 1 and 10")

mut first = "Ana"
mut second = "Bruno"
(first, second) = (second, first)         # both are `mut`: a swap
print("{first}, then {second}")

config = spawn read_text("config.txt")
print(config.wait() otherwise "no config found")

test "greeting uses the given name"
    ana = Person(name: "Ana", age: 30)
    assert greet(ana) == "Hello, Ana!"
```

```
test "negative ages are rejected"
    assert parse_age("-3") is _: Error
    assert parse_age(" 42 ")? == 42
```

Run it with `verse run tour.vs`; run its tests with `verse test tour.vs`. Two things are worth noticing. `greet` and `add_guest` have no type annotations at all, yet everything is checked at compile time (§4.2). And the `.copy()` in the `adults` line is the one place ownership shows: `filter` passes views of the people along without copying anything, and the chain ends in `map`, whose result is a new list that must own its names — so each name is copied, visibly (§4.9, §9.5).

---

3. Lexical structure and syntax

3.1 Source files

A Verse source file is UTF-8 text with the extension `.vs`. Each file is a module (see §12). Line endings may be `\n` or `\r\n`.

3.2 Comments

```
# A line comment runs to the end of the line.

## A doc comment documents the item that follows it.
## It may span several lines and uses Markdown.
fn square(n: Int): Int
    n * n
```

- `#` starts an ordinary comment.
- `##` starts a doc comment. A block of `##` lines **directly above** a declaration (no blank line in between) documents that declaration (`fn`, `type`, `alias`, `trait`, `impl`, a module constant, a field, or a `test`).
- A `##` block at the very top of a file that is **followed by a blank line** is the *module* doc. `verse fmt` enforces the blank line, so the two cases always look different. See §14.3.

There are no block comments; the formatter and editors handle commenting out regions.

3.3 Indentation and blocks

Blocks are introduced by a line that ends a header (such as `if cond`, `for x in xs`, `fn f()`, `type T`, `match v`) and consist of the following lines that are indented further.

```
if temperature > 30
    print("Hot")
    print("Stay hydrated")
else
    print("Pleasant")
```

Rules:

- Indentation uses **spaces only**; a tab character in leading whitespace is a compile-time error. The canonical width is **4 spaces**.
- A block's lines must all share the same indentation.
- An empty block is not allowed; write `pass` when a block intentionally does nothing.

3.4 Line continuation

A statement normally ends at the end of a line. It continues onto the next line when:

1. the line ends inside an open `(`, `[`, or `{`; or
2. the line ends with a binary operator (`+`, `and`, `or`, `otherwise`, `==`, `**`, …); or
3. the next line begins with `.` after a complete expression (a method chain); or
4. the next line begins with `|` (the next alternative of a variant list, §4.4).

A line that ends with `->` opens an indented block instead: the body of a match arm, or — inside an open `(` — the body of a lambda. In the lambda case the dedent that ends the block also closes the `(` opened on that line (the *closing rule*, §5.4).

A leading dot that *starts* an expression (right after `(`, `,` or an operator) is not a continuation but a key-path lambda such as `.name` (§5.4).

A `type X =` header always continues on the same logical line; it never opens a field block. Only a header of the form `type X` (or `type X<T>`) with nothing after the name opens an indented block of fields.

```
report = orders
    .filter(o -> o.paid)
    .map(o -> o.total)
    .sum()

ok = user.active and
    user.verified

point = Point(
    x: 1.0,
    y: 2.0,
)
```

Trailing commas are allowed everywhere a comma-separated list appears.

3.5 Identifiers and naming

Identifiers start with a letter or `_` and continue with letters, digits or `_`. Unicode letters are allowed. **The first character decides what kind of name it is**, and the compiler relies on this:

- An identifier whose first letter is an **uppercase** letter (Unicode category Lu or Lt) is a *type-level name*: a type, variant, trait, or generic parameter.
- Every other identifier — starting with a lowercase letter, `_`, or a letter that has no case at all (for example `数量` or `مدينة`) — is a *value-level name*: a variable, constant, function, field, parameter, or module.

Kind	Style	Example
Variables, functions, fields, modules	`snake_case`	`user_count`, `read_text`
Constants (module-level bindings)	`snake_case` (same as variables)	`max_retries`
Types, variants, traits, generic parameters	`PascalCase`	`HttpClient`, `NotFound`, `Speaker`, `T`

There is no `SCREAMING_CASE`: `MAX_RETRIES = 3` is an error, because names are `snake_case`; capitalised names are types and patterns. Within those two kinds, `snake_case` and `PascalCase` are conventions enforced by `verse fmt --check`.

A single `_` is a wildcard: it discards a value in patterns and bindings.

3.6 Keywords

```
alias      and        as         assert     break      continue   else
export     false      fn         for        from       if         impl
import     in         is         let        match      mut        none
not        or         otherwise  pass       return     self       Self
set        spawn      take       test       then       trait      true
type       while      with
```

`let` is reserved but not used: Verse has no `let` (bindings are written `x = …` and `mut x = …`, §3.9), and the compiler answers a `let` with the error "`let` is not used in Verse; write `x = …`".

3.7 Literals

```
# Integers
42
1_000_000
0xFF
0o755
0b1010_0101

# Floats
3.14
6.02e23
1e-9                          # an exponent alone makes a Float
1_000.5
```

Right after a `.`, a digit sequence is always a single `INT`, never the start of a `FLOAT`: `t.0.1` is the field `1` of the field `0` of `t` (nested tuple access), not `t` followed by `.` and `0.1`. A number followed by `.` and an identifier lexes as an `INT` and a method call (`10.seconds()`); a float needs a digit after the dot (`10.0`), and `10.` alone is an error.

```
# Booleans
true
false

# Characters
'a'
'\n'
'é'

# Text
"Hello, world"
"Hello, {name}!"              # interpolation
"""
    Multi-line text.
    Leading indentation is stripped.
    """
r"C:\raw\path\{not interpolated}"   # raw text

# Bytes
b"GET / HTTP/1.1\r\n"

# Missing value
none

# Collections
[1, 2, 3]                     # List<Int>
{"ana": 30, "bruno": 17}      # Map<Text, Int>
{:}                           # empty map
set{"a", "b"}                 # Set<Text>
set{}                         # empty set
[0, ..rest]                   # spread a list into a list literal
{..defaults, "port": 80}      # spread a map; later keys win
(1, "one")                    # (Int, Text) tuple

# Ranges
1..10                         # 1 up to 9
1..=10                        # 1 up to 10
nums[2..]                     # open-ended: only inside a slice
nums[..3]
```

A bare `{}` is a compile-time error: it does not say whether a map or a set is meant, and the message points to `{:}` (empty map) and `set{}` (empty set).

Spread. Inside a list, set or map literal, a *prefix* `..` spreads another collection's elements into the new one: `[0, ..rest, 99]`, `set{..admins, "root"}`, `{..defaults, "port": 80}`. In a map literal, when the same key appears more than once, the **later entry wins**, so overrides go after the spread. A prefix `..` (nothing on its left, right after `[`, `{` or `,`) never clashes with the infix range operator `a..b`. Spreading **moves** the elements out of the source, so the source must be at its last use (a temporary, or a binding not used afterwards). If the source is used later, the spread is a compile error that suggests spreading a copy, `..xs.copy()`; nothing is copied silently (§9.5).

Open-ended ranges — `a..` (from `a` to the end) and `..b` / `..=b` (from the start) — are allowed **only inside slice brackets**, such as `nums[1..]`, `nums[..3]`, `name[..=2]`. Everywhere else a range needs both ends, so `for i in 1..` is an error.

3.7.1 Text escapes

Escape	Meaning
`\n`, `\t`, `\r`	newline, tab, carriage return
`\\`	backslash
`\"`	double quote
`\{`, `\}`	literal braces (not interpolation)
`\0`	NUL character
`\u{1F600}`	Unicode scalar value by hex code

3.7.2 Interpolation

Inside an ordinary (non-raw) text literal, `{expr}` evaluates `expr` and inserts its textual form (via the `Show` trait, §10). Any expression is allowed. A text literal inside an interpolation is allowed one level deep (`"{email otherwise "none"}"`); interpolation inside such a nested literal is an error. An optional format specifier follows a colon. **The format colon is the first `:` that is not inside a bracket (`( )`, `[ ]`, `{ }`) or a nested text literal**, so `"{f(x: 1)}"` and `"{ {"a": 1}["a"] }"` contain no format specifier, while `"{price:.2}"` does. An expression whose own syntax has a top-level colon — such as a typed pattern in `"{v is n: Int}"` — must be wrapped in parentheses, `"{(v is n: Int)}"`; otherwise the colon is read as the format colon.

```
price = 1234.5
print("Total: {price:.2}")          # Total: 1234.50
print("Padded: {42:>6}")           # Padded:     42
print("Hex: {255:x}")              # Hex: ff
print("Contact: {email otherwise "none"}")
```

Style: keep nested literals short. A long expression or one with its own text literals reads better bound to a name first (`label = if count == 1 then "item" else "items"`, then `"{count} {label}"`), and `verse fmt --check` suggests doing so.

3.7.3 Multi-line and raw text

A triple-quoted literal `"""` begins a multi-line text. The opening `"""` must end its line. The indentation of the closing `"""` is removed from every line, so multi-line text can be indented naturally with the surrounding code. Interpolation and escapes work as usual.

```
fn letter(name: Text): Text
    """
    Dear {name},

    Thank you for trying Verse.
    """
```

A raw literal `r"..."` or `r"""..."""` performs no escapes and no interpolation, so `{` and `\` are ordinary characters — the form to use for JSON, regular expressions and Windows paths.

`r"""` follows the same layout rules as `"""` (the opening quotes end their line; the closing indentation is removed). A single-line `r"…"` cannot contain `"`.

3.8 Operators and precedence

From **highest** to **lowest** binding:

Level	Operators	Associativity	Notes
1	`f(x)` `a[i]` `a.b` `a?`	left	postfix, applied left to right (notes below)
2	`**`	right	power; binds tighter than unary minus (§3.8.4)
3	`-x`	right	numeric negation
4	`*` `/` `%`	left	
5	`+` `-`	left	`+` also concatenates `Text` and lists
6	`..` `..=`	none	ranges
7	`==` `!=` `<` `<=` `>` `>=` `in` `not in` `is`	chained	see §3.8.3
8	`not`	right	boolean negation
9	`and`	left	boolean, short-circuit
10	`or`	left	boolean, short-circuit
11	`otherwise`	left	fallback, short-circuit; restricted right operand (§3.8.2)

Notes on level 1:

- A second postfix `?` needs brackets: `(x?)?`.
- `a.b` on an optional `a` follows the lifted-chain rule (§8.1).
- Postfix `?` is an ordinary postfix operator. A `?` immediately followed by `.` (`a?.b`) is a compile-time error: "`?.` is not Verse; write `(a?).b` to propagate then read, or `a.b otherwise …` / `a.b?` to look through the optional". `(find_user(id)?).email` is valid: propagate a missing user out of the function, then read the field.

Three constructs are deliberately **not** operators in this table:

- `spawn` is a prefix that applies to exactly one call expression (`spawn fetch(url)`); its operand is never an arbitrary expression, so it has no precedence to compare (§11).
- `if a then b else c` is a complete expression form; each branch is a full expression, and the whole form extends as far to the right as possible. Wrap it in parentheses to use it inside a larger expression (§6.1). Right after `return`, `break` or `continue`, an `if` is **always** a postfix guard (§6.7): `return if done` means "return when `done`". To return a conditional value, parenthesize it: `return (if c then a else b)`.
- A lambda `x -> expr` likewise extends as far to the right as possible. A key-path lambda (`.name`) is a primary expression (§5.4).

Prefix `..` (spread, §3.7) appears only directly inside a collection literal and is not an operator either.

Assignment (`=`, `+=`, `-=`, `*=`, `/=`, `%=`) is a statement, not an expression, so `if x = 3` is always a syntax error.

3.8.1 Boolean operators: `and`, `or`, `not`

Boolean logic uses words, and **only** words: there is no `&&`, `||` or `!`. `and`, `or` and `not` accept only `Bool` operands — there is no "truthiness", so `if items` or `name or "x"` is a type error.

```
can_enter = is_member or has_ticket
valid = not name.is_empty() and age >= 18
```

`and` and `or` short-circuit.

3.8.2 The fallback operator: `otherwise`

`left otherwise right` means **"use `left` if it is usable, otherwise `right`."** The left operand must be optional or fallible:

Left operand type	Fallback happens when…	Result type
`T?`	it is `none`	`T` if right is `T`; `T?` if right is `T?`
`T | E` (`E` an error)	it is an error	`T` if right is `T`; `T | E` if right is `T | E`

```
name = nickname otherwise "friend"                          # Text? → Text
timeout = settings["timeout"] otherwise 30                  # Int? → Int
text = read_text(path) otherwise "default"                  # Text | Error → Text
email = find_user(id).email otherwise "no email"            # User? → Text
source = read_text(primary) otherwise read_text(backup)       # still Text | Error
```

The right side is evaluated only when needed. Using `otherwise` on a value that can never be missing or failed (a plain `Int`, a `Bool`) is a compile-time error, because the fallback could never happen. The one exception is a binding with a refutable pattern (below): there the fallback runs when the pattern does not match, so the left side may be of any type (`Circle(r) = shape otherwise return 0.0` with a plain `Shape`).

Ownership of the result. Either side of `otherwise` may be a view (a borrow, an element from `find` or `m[k]`, §9.5) or an owned value. Text and number literals and other static data count as views when needed, so they fit either way. The result is:

Left	Right	Result
a view	a view or a literal	a **view**: `args.first() otherwise "world"`
owned	owned, or a literal	**owned**: `read_text(path) otherwise "default"`
a view	an owned temporary (not static)	**owned**, so the left side must be copied: `m["k"].copy() otherwise make_default()`
owned	a non-static view	**owned**, so the right side must be copied: `read_text(path) otherwise fallback_text.copy()`

Leaving out the `.copy()` in either of the last two rows is a compile-time error that names the side to copy. A view result follows the usual rules: it can be bound to a name as a local view

or passed along, but storing or returning it needs `.copy()`.

A literal in an owned position (the second row) is **materialised**: the compiler creates an owned value from it. This is the one implicit creation Verse allows, because it duplicates no user value. A lifted chain (§8.1) on an owned temporary yields an owned field, so in `find_user(id).email otherwise "no email"` (where `find_user` returns an owned `User?`) the left side is owned and the result is an owned `Text`.

The same rule applies to every expression that chooses between branches: `if c then a else b`, a block `if`/`else` used as a value, and the arms of a `match`. The result is a view only if **every** branch is a view (or a literal); otherwise it is owned, and each branch that is a view needs `.copy()` — a compile-time error otherwise.

`otherwise` binds more loosely than every other binary operator. To keep that from ever surprising a reader, **the right operand of `otherwise` must be** a literal (including a negative number such as `-1`), a call, a postfix expression (a name, field access, index, or a chain of them), a parenthesized expression, or another `otherwise`. Anything else is a compile-time error:

```
n = counts["a"] otherwise 0 + 1          # error: write `(counts["a"] otherwise 0) + 1`
                                         #        or `counts["a"] otherwise (0 + 1)`
count = counts.get("a", default: 0) + 1  # clearer still, for maps (§9.2)
```

`otherwise return`, `otherwise break`, `otherwise continue`

As a deliberate exception to that rule, the right operand of `otherwise` may also be one of the exits `return` (with or without a value), `break` or `continue` — but only as the **whole** right operand, never nested inside a larger expression:

```
fn label_for(id)
    user = find_user(id) otherwise return "unknown"         # User? → User
    "{user.name} ({user.age})"

for line in lines
    n = line.to_int() otherwise continue                    # skip non-numbers
    total += n

x = value otherwise (return 1) + 2                          # error: an exit must be
the
                                                            #   whole right operand
```

When the left side is optional, the binding gets the unwrapped `T`. This is the one form that also allows a **refutable pattern in a binding**, because the failure case has somewhere to go:

```
fn radius_of(shape)
    Circle(r) = shape otherwise return 0.0
    r
```

With a **fallible** left side, `otherwise return …` (or `break`/`continue`) *throws the error away* — the reason for the failure is lost. That is sometimes exactly right (skip a malformed line); when

the reason matters, propagate it instead with `.context("…")?` (§8.4). There is no `guard` keyword: `x = … otherwise return` is the guard.

3.8.3 Comparison chaining

Comparisons can be chained and read like mathematics:

```
if 0 <= index < items.len()
    print(items[index])
```

There is no ambiguity with explicit type arguments: those are written only after a capitalised type name and a dot (`Channel.new<Int>(capacity: 10)`, §11.4), so `a.b < c > (d)` with a lowercase receiver is always a chained comparison.

`a < b < c` means `a < b and b < c`, with `b` evaluated once.

3.8.4 Arithmetic

- `Int` arithmetic is checked: overflow stops the program with a clear error. Wrapping and saturating arithmetic are available as methods (`a.wrapping_add(b)`, `a.saturating_mul(b)`).
- `/` on two `Int`s divides and truncates toward zero; `%` is the matching remainder. Dividing an `Int` by zero stops the program. `a.checked_div(b)` returns `Int?`.
- `/` on `Float`s follows IEEE 754.
- There are no implicit numeric conversions: write `n.to_float()` or `x.round().to_int()`.
- `a ** b` raises `a` to the power `b`. It is right-associative (`2 ** 3 ** 2` is `2 ** 9`) and binds tighter than unary minus, so `-2 ** 2` is `-4`, as in mathematics. On `Int` it is overflow-checked, and the exponent must not be negative: `2 ** -1` is a compile-time error (use `2.0 ** -1.0`), and a negative `Int` exponent computed at run time panics.

3.9 Statements and expressions

Most constructs are expressions: `if … then … else …`, block `if`/`else`, `match`, blocks, and calls all produce values. Declarations (bindings, `fn`, `type`, `trait`, `impl`, `import`, `test`) and assignments are statements. The value of a block is the value of its last expression.

Bindings and assignment

There is no `let`. A name is introduced by writing it, and `mut` marks a name that can change:

```
total = 0                       # a new immutable binding
mut count = 0                   # a new mutable binding
count = count + 1               # `count` exists and is `mut`: reassignment
count += 1                      # the same
n: Int = 5                      # with a type annotation
mut ratio: Float = 0.0
(low, high) = bounds            # destructuring (§7.7)
Circle(r) = shape otherwise return 0.0   # refutable: only with `otherwise` (§3.8.2)
```

The rule for `name = expr` depends only on what is already visible:

`name` is…	`name = expr` means
not yet bound in this function	a **new immutable binding**
a visible `mut` binding	**reassignment**
a visible immutable binding or immutable parameter	a compile-time error: "`total` is immutable; write `mut total = …` where it is declared, or pick a new name"

`mut name = expr` always declares a new mutable binding. In a pattern, a name starting with an uppercase letter is a constructor or variant and a lowercase name is a new binding (§3.5), so `Circle(r) = shape` destructures and never calls anything. A refutable pattern (one that might not match) is allowed only with `otherwise return`/`break`/`continue`; without it, it is a compile-time error.

Destructuring always creates new bindings. Every lowercase name in a pattern on the left of `=` is a new binding, so a name that already exists is an error ("`x` already exists; a pattern always binds new names — to change `x`, declare it `mut` and assign it on its own"). The one exception is a **parallel assignment**: a tuple made *only* of visible `mut` names assigns them all at once, with the right side fully evaluated first, so `(a, b) = (b, a)` swaps. Mixing new and existing names in one tuple is an error.

```
mut a = 1
mut b = 2
(a, b) = (b, a)                # parallel assignment: a == 2, b == 1
(a, c) = (3, 4)                # error: `a` exists but `c` is new; do not mix
```

No shadowing. The rule is: **you cannot bind a name that is currently visible**, where *visible* means **locals only**: parameters, earlier bindings, and loop, pattern and lambda names in scope. Parameters (of functions and of lambdas), `for` loop variables, pattern bindings in `match` arms and `if … is`, and `with … as` names all count: each lives in its own block, but it may not reuse a name that is already visible there. A name bound before a block cannot be bound again inside it; a name bound inside a block may be bound again after that block ends, because the inner one is out of scope. Sibling blocks — the branches of one `if`/`else` or `match`, or two separate loops — may reuse a name, because neither can see the other:

```
fn report(items)
    count = items.len()
    for item in items
        count = item.size          # error: `count` is visible here (immutable)
        size = item.size           # fine: new inside the loop
    size = 0                       # fine: the loop's `size` is out of scope
    items.map(count -> count + 1)  # error: lambda parameter reuses `count`
```

Without `let`, a second `x = …` would otherwise be impossible to tell apart from an assignment, so Verse does not allow it at all; a transformed value gets a new name (`raw = read_text(path)?` then `text = raw.trim()`). Entry-file top-level bindings follow the script-local rules of §12.4.

Names from outside the function are not locals. A local may **hide a prelude name** silently (`input = lines[0]`, `error = e.message()`) and may hide a module constant or an imported name (`time = elapsed.seconds()` after `import std.time`). Hiding an imported name **warns**

only when the same function also uses that import — an actual collision ("`fs` hides the import `std.fs`, which this function also uses"). The same holds one level up: a file-level import hides a prelude name of the same spelling, so after `import info, error from std.log` the name `error` means `std.log`'s function in that file, and the prelude `error` is not reachable there (there is no qualifier for prelude names; import `std.log` as a module, `log.error(…)`, to keep both). Because this changes a name for the whole file, it **warns on the import line**: "`error` hides the prelude `error`". Names bound by `with … as` follow the same rule as other bindings: on a lock guard, the bound value can be changed only when written `as mut value` (§11.4).

A `mut` parameter is the caller's place under an exclusive borrow, so assigning to it replaces the caller's value in place (§5.3):

```
fn reset(mut list)
    list = []                      # the caller's list is now empty
```

There is still one form, `x = …`, for both meanings, so tools carry the difference: editors and `verse fmt --check` tooling must render a reassignment distinctly from a new binding (for example an inlay hint "reassign" before the name). This is a tooling requirement, not syntax.

Two warnings catch the usual slips:

- an **unused binding** warns (prefix the name with `_` to keep it on purpose); it is a warning only, and illustrative snippets in this document are exempt — the spec's examples are not compiled with warnings-as-errors;
- a **new binding whose name is one edit away from a visible `mut` binding** warns, because it is probably a typo for an assignment:

```
warning: new binding `totl` looks like the mutable `total`
  --> sum.vs:6:5
   |
 2 |     mut total = 0
   |         ----- `total` is declared here
 6 |     totl = total + n
   |     ^^^^ this creates a new name
   |
help: did you mean to reassign `total`?   total = total + n
```

---

4. Types

4.1 Built-in scalar types

Type	Description	Examples
`Int`	64-bit signed integer, overflow-checked	`0`, `-12`, `1_000`
`Float`	64-bit IEEE 754 floating point	`3.14`, `1e-9`
`Bool`	truth value	`true`, `false`
`Char`	one Unicode scalar value	`'a'`, `'\u{263A}'`
`Text`	immutable UTF-8 string	`"hello"`
`Bytes`	immutable sequence of bytes	`b"\x00\xFF"`
`Nothing`	the type of "no useful value"	(result of `print`)

Sized numeric types (`Int8` … `Int64`, `UInt8` … `UInt64`, `Float32`) and arbitrary-precision `BigInt` and `Decimal` are provided by `std.num` for code that needs them.

`Text` is indexed by *characters*, not bytes: `"héllo".len()` is `5`. Use `text.bytes()` for the raw bytes.

4.2 Type annotations and inference

Verse is statically typed, but types are **visually optional**: annotations may be left out almost everywhere — on local bindings, on function parameters, and on return types. The compiler infers them, checks everything at compile time, and never guesses: when it cannot decide, it stops and asks for one annotation at that exact spot.

```
count = 0                           # Int
ratio = 0.5                         # Float
ratio2: Float = 1                   # error: 1 is an Int; write 1.0
mut tags = []                       # element type decided by later use…
tags.push("new")                    # …so tags is List<Text>

fn greet(name)                      # inferred: any `name` that is Show; returns Text
    "Hello, {name}!"

fn greet_typed(name: Text): Text    # the same function, spelled out
    "Hello, {name}!"

fn double(n)                        # numeric; Int unless a caller says otherwise
    n * 2
```

Both styles are equally checked. Annotate where it helps a reader — at important boundaries, in teaching code, or to narrow a function on purpose — and leave the rest to the compiler.

Inference rules

1. **Unannotated parameters are implicit generics.** Each one gets exactly the constraints its body needs, and nothing more:

2. trait constraints from operations — interpolation or `print` gives `Show`, `+` gives `Add`, `==` gives `Eq`, `<` gives `Order`, `for` gives `Iterable`, indexing `x[i]` gives `Index`, `.copy()` gives `Copyable`, calling a trait method gives that trait;
3. *callable* from calls — `f(x)` on a parameter `f` makes it a function type with matching parameters;
4. *structural* constraints from field access and inherent method calls — `person.name` gives "has a field `name`", `text.trim()` gives "has a method `trim()`";
5. a variant pattern (`Circle(r) -> …`) or passing the parameter to an annotated parameter fixes or narrows the type.

Every call site is checked against those constraints at compile time, and the compiler generates specialized code for each concrete type used — but the **ownership check runs once, at the definition** (rule 9), so whether a body compiles never depends on which types its callers use. So `greet(42)`, `greet("Ana")` and `greet(ana)` all work if the argument is `Show`, while `greet(some_function)` is a compile error at the call site. Structural constraints are inferred only; they cannot be written by hand in 0.1. To narrow a function on purpose, annotate a concrete type (`person: Person`). 2. **Numeric literals default to `Int` and `Float`.** `fn double(n)` with `n * 2` accepts any numeric type; where nothing else decides (for example `x = double(3)`), the literal makes it `Int`, and `double(2.5)` is a `Float`. 3. **Recursion needs an annotation.** A function that calls itself, directly or through other functions, must annotate its parameters and return type; otherwise the compiler reports an ambiguity at the recursive call. 4. **When inference cannot decide, it says so — exactly there.** The error names the spot, says which annotation to add, and lists the constraints it did infer:

```
error: cannot infer the element type of this list --> stats.vs:2:5 | 1 | fn empty_scores() | - add a return type here, for example `: List<Int>` 2 | [] | ^^ a List of something, but nothing says of what
```

1. **Fallible return types are inferred from `?`.** A function whose body uses `?` on fallible values (or returns an `error(…)`) gets a fallible return type made of its success type and the errors that can escape, normalized as in §4.6. A `fn load(path)` whose body is `text = read_text(path)?` followed by `text.trim()` returns `Text | Error`. A `?` on an optional gives an optional return type.
2. **Inferred types are visible on demand.** Editors (through `verse lsp`) show inferred parameter and return types as inline hints. `verse fmt` never inserts or removes annotations.
3. **Record fields keep their annotations** unless a default value implies the type (`active = true` is a `Bool` field; `tags: List<Text> = []` still needs its annotation because `[]` says nothing about the elements).
4. **`mut` and `take` are always written.** How a function treats its argument (borrow, exclusive `mut` borrow, or `take`) is never inferred (§5.3).
5. **Ownership is checked once, at the definition.** Inside a generic or unannotated function, every parameter type that is unannotated or generic is treated as an opaque, **non-`Copy`** type, unless a `Copy` bound says otherwise (`<T: Copy>`). One bound is inferred instead of rejected, so that simple getters need no annotation: an unannotated function that returns a field, element or lookup read through a borrowed parameter gets an **inferred `Copy` bound** on the returned type, recorded in `verse.api` like any other inferred constraint:

``verse fn age_of(person) person.age # fine: inferred bound "person.age` is Copy"

fn first_name(person) person.name # fine here too: inferred bound "`person.name` is Copy"

first_name(ana) # error at the call site: `first_name` returns # `person.name` by value, so the field must be # Copy, and `Person.name` is `Text` # help: return `person.name.copy()`, or take # the parameter with `take` ```

Errors about a function's body appear at the definition. The only call-site errors are unmet bounds, written or inferred, and they name the bound. Because the check never looks at callers, `verse.api` is enough to check other modules (§12.3).

Inference and module boundaries

Inference never crosses a module boundary. Exported functions may have inferred signatures **only** because every package commits a generated interface file, `verse.api`, that records each exported signature as the compiler inferred it. Other modules are type-checked against `verse.api`, never against another module's body, and the build fails if an inferred exported signature drifts from the recorded one until someone runs `verse api update` on purpose. A body edit therefore cannot silently change the public API. See §12.3 and §16.4.

4.3 Record types

A record type groups named fields. Fields are listed on indented lines.

```
type User
    name: Text
    age: Int
    email: Text? = none
    roles: List<Text> = []
    active = true                # type implied by the default: Bool
```

Fields keep their type annotations unless a default value implies the type (§4.2). Fields may have default values. Construct a record by calling the type name with named arguments; positional arguments are allowed in field order. When a variable has the same name as the field, the value may be left out — `User(name:, age:)` means `User(name: name, age: age)`. This *field punning* works the same way in construction and in patterns (§7.3).

```
ana = User(name: "Ana", age: 30)
bruno = User("Bruno", 17)
name = "Carla"
age = 41
carla = User(name:, age:)
print(ana.name)
```

Fields are visible inside the module that defines the type. To make a field visible to other modules, mark it `export` (§12.3).

Copies

Every value has one owner (§4.9). `copy()` with no arguments makes an independent duplicate; the same method with named arguments makes a duplicate with some fields

changed:

```
twin = ana.copy()                # an independent User
older = ana.copy(age: 31)        # ana is unchanged
```

`.copy()` is the one way to duplicate a value, with or without changes. It is provided by the built-in trait `Copyable` (§10.4), which almost every type has automatically; the exceptions are `Task<T>`, types that implement `Cleanup` (files, locks, temporary directories), and anything that contains them.

4.4 Variant types

A variant type (also called a sum type or tagged union) is one of several named alternatives, each with its own fields.

```
type Shape = Circle(radius: Float) | Square(side: Float)

type Direction = North | East | South | West

type Json = Null
    | Flag(value: Bool)
    | Number(value: Float)
    | Str(value: Text)
    | Array(items: List<Json>)
    | Object(fields: Map<Text, Json>)
```

Long variant lists may continue across lines when each line starts with `|` (§3.4). A `type X =` header never opens a field block.

A variant name may not collide with a type that is already in scope (`Int`, `Float`, `Text`, `Error`, an imported type, …). This one rule keeps `type` and `alias` apart:

```
type Direction = North | East | South | West   # fine: four new unit variants
type Meters = Float            # error: `Float` is an existing type
                               #   help: to give it another name, write `alias Meters =
Float`
type Value = Int | Text        # error: help: write `alias Value = Int | Text`
type Reply = Text(body: Text) | Empty   # error: variant name `Text` is already a type
```

Variants are constructed like records — `Circle(radius: 2.0)` or `Circle(2.0)` — and may be qualified with the type name when a name is ambiguous: `Shape.Circle(2.0)`. Variant types may be recursive; the compiler handles the indirection.

4.5 Tuples

A tuple groups a fixed number of values of possibly different types.

```
pair = (3, "three")                # (Int, Text)
print(pair.0)                      # 3
(number, word) = pair              # destructuring
```

Tuple elements may optionally be **labeled**, in which case they can be read by name as well as by position:

```
fn divide(a: Int, b: Int): (quotient: Int, remainder: Int)
    (quotient: a / b, remainder: a % b)

result = divide(17, 5)
print("{result.quotient} remainder {result.remainder}")   # 3 remainder 2
(q, r) = result
```

4.6 Optional, union and fallible types

- `T?` is an **optional**: a `T` or `none`.
- `A | B` is a **union**: a value that is either an `A` or a `B`. The member types must be distinct.
- A union with a member that implements the `Error` trait is **fallible**: `Int | Error` is "an `Int`, or an error". Members implementing `Error` form the *error side*; the others form the *success side*.

`?` binds tighter than `|` in types: `Text? | Error` is "an optional text, or an error". Optionals nest: `Int??` is a different type from `Int?` (§8.1).

Union normalization. Unions are flattened and de-duplicated (`(A | B) | A` is `A | B`), and a specific error type in a union that also contains `Error` is **absorbed** by `Error`: `Text | FetchError | Error` normalizes to `Text | Error`. Once a value has been widened to `Error`, `?` cannot narrow it back to `FetchError`; use a typed pattern in `match` (`err: FetchError -> …`) to inspect the concrete error.

Both are covered in depth in §8.

4.7 Function types

The type of a function is written `fn(ParamTypes): Ret`:

```
double: fn(Int): Int = n -> n * 2
fn apply_twice(f: fn(Int): Int, x: Int): Int
    f(f(x))
```

4.8 Type aliases

An alias gives an existing type a second, shorter name. It does not create a new type; the two names are interchangeable.

```
alias Headers = Map<Text, Text>
alias Handler = fn(Request): Response | Error
alias Loaded = Config | Error
```

`alias` is required to name a union of *existing* types. In a `type` declaration, `|` always introduces *new* variants (§4.4), so `type X = A | B` and `alias X = A | B` can never be confused.

To create a genuinely *distinct* type (so that a `UserId` cannot be mixed up with an ordinary `Int`), declare a one-field record:

```
type UserId
    value: Int
```

4.9 Ownership, moves and copies

Every value has **exactly one owner**: a binding, a field of a record, an element of a collection, or a task. When the owner's scope ends, the value is freed — immediately and deterministically.

Copy types. Scalars (`Int`, `Float`, `Bool`, `Char`, `Nothing`) and records, variants and tuples made *only* of copy types (a `Point(x: Float, y: Float)`, a `Shape`, an `(Int, Int)`) are copied implicitly whenever they are assigned or passed. There is no size limit: whether a type is `Copy` depends only on the kinds of its fields, never on its size. A copy is never inserted silently except for `Copy` types and literals (§3.8.2), and moves happen at a value's last use; adding a non-`Copy` field (say a `Text`) to an all-scalar record makes it non-`Copy`, and the compiler then reports every place that relied on the implicit copy. Everything else — `Text`, `Bytes`, collections, and records that contain them — is **moved** when it gets a new owner. A non-`Copy` value is never duplicated implicitly; anywhere an implicit duplicate would be needed, the compiler asks for `.copy()`.

Moves. A value moves when it is:

- bound with `mut b = a`, or with `b = a` when `a` is not used afterwards;
- stored in a field or a collection (record construction, `push`, `map[k] = v`);
- returned from a function;
- passed to a `take` parameter (§5.3) or into a task with `spawn` (§11.1);
- captured by an escaping lambda (§5.4).

When `b = a` appears and `a` *is* used afterwards, nothing moves: `b` is a read-only local view of `a` (a borrow, §5.3), and `a` cannot be changed while `b` is in use. A binding to a collection chain is a local view in the same way (§9.5).

Moving a field out. Moving a field out of an owned value at the value's last use is allowed: the field gets its new owner and the rest of the value is freed. If the value is used afterwards (or is only borrowed), moving the field is the usual move error, with a `.copy()` suggestion:

```
fn body_of(url)
    response = get(url)?
    response.body            # fine: `response` is owned and not used again
```

Moves are tracked **per control-flow path**. A value moved in one branch of an `if` or `match` counts as moved after it on that path, and a value moved inside a loop counts as moved on the next iteration.

```
mut a = [1, 2, 3]
mut b = a.copy()              # an independent list
b.push(4)
print(a)                      # [1, 2, 3]
print(b)                      # [1, 2, 3, 4]

names = ["Ana", "Bruno"]
mut roster = names            # moves: `names` is not used afterwards
roster.push("Carla")

p = Point(x: 1.0, y: 2.0)     # an all-Float record: copied implicitly
q = p
print(p.x + q.x)              # fine: q is a copy
```

Using a moved value is a compile-time error that says where the move happened and suggests `.copy()`:

```
mut inbox: List<Text> = []
note = "remember the milk"
inbox.push(note)
print(note)
```

```
error: `note` is used after it was moved
  --> notes.vs:4:7
   |
 3 | inbox.push(note)
   |            ---- `note` moved into `inbox` here
 4 | print(note)
   |       ^^^^ used here after the move
   |
help: to keep using `note`, store a copy:   inbox.push(note.copy())
```

Errors name the later use. Whether `b = a`, a `for` loop variable (§6.3), a pattern binding (§7.7) or a chain result (§9.5) owns its value or only views it depends on whether the source is used later. So adding a line below can turn working code into an error. Every error caused by such a flip must name the later use that kept the source alive:

```
error: `person` is only a view, so it cannot be stored in `adults`
  --> people.vs:8:16
   |
 7 | for person in people
   |     ------ `person` is a view because `people` is used again on line 12
 8 |     adults.push(person)
   |                 ^^^^^^ stored here
...
12 | print(people.len())
   |       ------ the later use that keeps `people` alive
   |
help: write `person.copy()`, or move the use on line 12 above the loop
```

Inside a loop, the error points at both the move and the later use — which is the same line, one iteration later:

```
greeting = "hi"
mut all: List<Text> = []
for name in names
    all.push(greeting)        # error: `greeting` moved here in one iteration
                              #   and used again here in the next
                              #   help: all.push(greeting.copy())
```

Mutability is a property of the **binding**, and it is deep. `mut` means "can change" everywhere it appears — on a binding, a parameter (§5.3) or `self` (§5.6):

- A plain binding (`user = …`) can never be reassigned, and the value it owns can never be changed — not its fields, not its elements.
- A `mut` binding (`mut guest = …`) can be reassigned with `guest = …`, and its fields and elements can be changed in place. Reassigning frees the old value first.

Because there is no `let` and no shadowing (§3.9), `guest = …` on an existing name is always a reassignment, and on an immutable name always an error.

```
user = User(name: "Ana", age: 30)
user.age = 31                     # error: `user` is immutable; write
                                  #   `mut user = …` where it is declared

mut guest = User(name: "Guest", age: 0)
guest.age = 18                    # fine
guest = User(name: "Other", age: 5)   # fine; the old guest is freed
```

Methods that change their receiver declare `mut self` and can only be called on mutable bindings (§5.6).

Order inside an assignment. In `place = value` (and `place += value`), the right side is evaluated completely before the place, so a value can be read on the right and moved into the place on the left in the same statement. The receiver and index expressions on the left are evaluated after the right side, left to right among themselves; everything else is evaluated left to right:

```
counts[word] = counts.get(word, default: 0) + 1   # borrows `word`, then moves it
```

4.10 Memory model

Verse has **no garbage collector** and no manual memory management:

1. **Single ownership.** Every value has one owner and is freed at the end of its owner's scope (or when a `mut` owner is reassigned). There is no collector running in the background and no pause.
2. **No references in the surface language.** There are no pointer types, no reference types, and no lifetime syntax. Functions *borrow* their arguments, but borrows are **second-class**: they cannot be stored in a field, returned, or sent to another task (§5.3). This is what lets the compiler prove, without annotations, that nothing is ever used after it is freed.
3. **Recursive types are boxed for you.** Variant types such as `Json` or `Tree<T>` are stored with the indirection they need; the boxes are owned like any other value and freed with their owner.
4. **Cleanup is deterministic.** A type that implements `Cleanup` has its `cleanup(mut self)` run automatically when its owner's scope ends — also during a panic (§13).
5. **Sharing is opt-in.** The only ways for two owners to share one value are the handle types `Shared<T>` (shared read-only), `Guarded<T>` (shared, mutable behind a lock), `Weak<Shared<T>>` / `Weak<Guarded<T>>` (non-owning), and the channel sender `Sender<T>`

(§11.4). Copying a handle is cheap and shares the value behind it; the copy must be written explicitly as `.copy()`, like every other copy (a `Task<T>` and a `Receiver<T>` are never shared: they cannot be copied, and they move; `wait` takes a task).

Consequences:

- No use-after-free, no double free, no dangling reference, and no data race, in any program that compiles.
- **Cycles are possible only through `Shared`/`Guarded` handles** (for example two `Guarded` nodes pointing at each other). Such a cycle is never freed; use a `Weak<Shared<T>>` or `Weak<Guarded<T>>` for back-pointers to avoid it.
- Arbitrary graphs (trees with parent links, graphs with cycles) are built either with `Shared`/`Weak` handles or — usually simpler and faster — with an **index-based arena**: store the nodes in a `List<Node>` and refer to them by `Int` index.

```
type Node
    name: Text
    children: List<Int> = []      # indices into the arena
    parent: Int? = none

type Tree
    nodes: List<Node> = []

impl Tree
    fn add(mut self, take name, parent: Int?): Int
        self.nodes.push(Node(name:, parent:))
        if parent is p?
            # The argument is evaluated first, then the `mut` borrow of the
            # receiver begins (§5.3), so reading `self.nodes` here is fine.
            self.nodes[p].children.push(self.nodes.len() - 1)
        self.nodes.len() - 1
```

The trade-offs of this model are discussed honestly in §18.10.

---

5. Functions

5.1 Declaring functions

A function is declared with `fn`, its parameters, an optional return type after a colon, and an indented body. **The last expression of the body is the return value.** Parameter and return types may be written or left to inference (§4.2):

```
fn greet(name)
    "Hello, {name}!"

fn hypotenuse(a: Float, b: Float): Float
    squares = a * a + b * b
    squares.sqrt()
```

A function whose body ends in a statement (or that is annotated without a return type and never returns a value) returns `Nothing`:

```
fn log(message)
    print("[log] {message}")
```

5.2 Early return

`return` exits a function early. It is needed only when leaving before the last line:

```
fn classify(n)
    if n < 0
        return "negative"
    if n == 0
        return "zero"
    "positive"
```

Two compact forms exist for the most common early exits. A postfix guard puts the condition after the exit (§6.7), and `otherwise return` unwraps an optional or leaves (§3.8.2):

```
fn describe_user(id)
    return "invalid id" if id <= 0
    user = find_user(id) otherwise return "no such user"
    "{user.name}, {user.age}"
```

5.3 Parameters and arguments

Borrow, `mut`, and `take`

How a function receives each argument is part of its signature and is always written explicitly (it is never inferred):

Parameter	Meaning	Call site
`p`	**read-only borrow** (the default): the function may look, not change or keep	`f(x)`
`mut p`	**exclusive mutable borrow**: the function may change the caller's value in place	`f(mut x)` — `x` must be a `mut` binding
`take p`	**consume**: the function becomes the owner; the caller's value moves (copy types are copied)	`f(x)` — no marker; the move is visible because `x` must not be used afterwards

`mut` means "can change" in every position: a `mut` binding can be reassigned and changed in place (§3.9, §4.9), a `mut` parameter may change the caller's value, and a `mut self` method may change its receiver (§5.6). Parameter names count as bindings of the function, so a body cannot bind a parameter's name again (§3.9). A `mut` parameter is the caller's place under an exclusive borrow, so assigning to it replaces the caller's value in place:

```
fn reset(mut list)
    list = []                          # the caller's list is now empty

mut names = ["Ana", "Bruno"]
reset(mut names)                       # names == []
```

```
type Cart
    items: List<Item> = []

fn total(cart)                          # borrows `cart`
    cart.items.map(.price).sum()

fn add_item(mut cart, take item)        # changes `cart`, keeps `item`
    cart.items.push(item)

mut cart = Cart()
apple = Item(name: "apple", price: 0.5)
add_item(mut cart, apple)               # `mut` at the call site; `apple` moves
print(total(cart))
print(apple.name)                       # error: `apple` was moved into `add_item`
                                        #   help: add_item(mut cart, apple.copy())
```

The `mut` marker is written at the call site of a *plain function*, so a reader can see which arguments may change. Method calls (`cart.add(item)`) do not carry it (§5.6). A value cannot be passed as `mut` and also as another argument of the same call (`merge(mut a, a)` is an error), so a function with a `mut` parameter always has exclusive access.

Arguments first, then the exclusive borrow. All arguments of a call are evaluated first, left to right; only then does the exclusive borrow for `mut` parameters (and for a `mut self` receiver, §5.6) begin. So an argument may read the value that is about to be borrowed mutably — the same idea as the assignment rule in §4.9:

```
add_item(mut cart, Item(name: "extra", price: total(cart)))   # fine
history.push(history.len())                                   # fine
```

Borrows are second-class

A borrowed parameter is a view of the caller's value for the duration of the call. It **cannot be stored** in a field or collection, **cannot be returned**, and **cannot be sent to another task**. In particular, a function cannot return a reference into its argument — it returns a copy, an owned value, or an optional:

```
fn first_name(person: Person)
    person.name                 # error: cannot return a borrow of `person`
                                #   (`name` is `Text`, not Copy)
                                #   help: return a copy: person.name.copy()

fn first_name(person: Person)
    person.name.copy()          # fine: the caller owns the result

fn age_of(person)
    person.age                  # fine: a copy, with an inferred `Copy` bound
```

An unannotated getter such as `age_of` compiles with an inferred `Copy` bound on the returned field (§4.2, rule 9). Errors about a function's body appear at the definition. The only call-site errors are unmet bounds, written or inferred, and they name the bound.

This is the price of having no lifetimes: there is no lifetime syntax and no elision rules to learn, because a borrow can never outlive the call that created it. The cost, stated plainly: accessor-style functions copy, and code that would return a view into a large structure in

Rust or C++ is written differently in Verse — it changes the structure in place through `mut`, takes ownership with `take`, returns an index, or shares the data through `Shared<T>` (§11.4).

Default values and named arguments

Parameters may have default values. At the call site, arguments can be passed by position or by name. Positional arguments come first; once a named argument appears, the rest must be named.

```
fn connect(host, port = 80, secure = false)
    ...

connect("a.com")
connect("a.com", port: 443, secure: true)
connect("a.com", secure: true)              # port uses its default
connect(host: "a.com", port: 8080)
```

A named argument is written `name: value` — the same shape as a map entry (`{"name": value}`) or a record field. When the value is a variable with the same name as the parameter, it can be left out: `connect(host:, port:)` means `connect(host: host, port: port)` (*punning*, the same shorthand as in record construction and patterns). Default values are evaluated at each call, never shared between calls, and a default fixes the parameter's type when nothing else does (`port = 80` makes `port` an `Int`). Named arguments are especially encouraged for `Bool` parameters, and `verse fmt --check` warns on positional `true`/`false` arguments.

5.4 Lambdas

A lambda is written `param -> expression`. Multiple parameters are wrapped in parentheses; types are inferred from context but may be written.

```
double = n -> n * 2
add = (a, b) -> a + b
shout = (text: Text) -> text.upper() + "!"
greet_all = () -> print("Hello, everyone")
```

Multi-line lambdas: the closing rule

When a line inside an open `(` ends with `->`, the lambda's body is the indented block that follows, and **the dedent that ends the block also closes the parenthesis opened on that line**. The lambda must be the last argument of that call:

```
labels = orders.map(order ->
    status = if order.paid then "paid" else "due"
    due = order.due_date.format("%d %b")
    "{order.id}: {status} (due {due})"
print(labels.join("\n"))
```

There is no separate "trailing lambda" syntax: a lambda is always an argument inside the call's parentheses, and the closing rule only saves writing the lonely `)`. In detail:

- One dedent closes **every** parenthesis opened by block lambdas on the lines it ends (for example a block lambda nested inside another).

- Nothing can follow the closed call on the dedent line, so a call cannot be chained after it; bind the result first (`labels = orders.map(order ->` … then `labels.sum()`).
- Writing the `)` explicitly on its own line is accepted; `verse fmt` removes it (§16.1).

Lambdas in match arms

`->` is used only by lambdas and match arms, so a lambda that forms the body of a match arm must be wrapped in parentheses to keep the two arrows apart:

```
fn operation(op: Op): fn(Int): Int
    match op
        Double -> (n -> n * 2)
        Negate -> (n -> -n)
```

Lambdas that appear as call arguments are already inside parentheses and need nothing extra: `Evens -> items.filter(n -> n % 2 == 0)`.

Key-path shorthand

A leading dot that *starts* an expression — right after `(`, `,` or an operator — is a one-parameter lambda that reads from its argument:

```
ages = users.map(.age)                      # same as u -> u.age
names = users.map(.name.copy())             # u -> u.name.copy()  (owned results, §9.5)
loud = users.map(.name.upper())             # u -> u.name.upper()
firsts = pairs.map(.0)                      # p -> p.0
by_age = users.sort_by(.age)
```

Only plain access chains (fields, tuple positions, method calls) are allowed after the dot. Anything with an operator is a compile error that suggests a lambda: `users.filter(.age >= 18)` → write `users.filter(u -> u.age >= 18)`. After a complete expression, a leading dot still continues a method chain (§3.4). There are no leading-dot variant names (`.north` is not `North`).

Captures: borrowed or moved

A lambda uses variables from its surroundings in one of two ways, decided by what happens to the lambda:

- A lambda that is **only called while the function it was passed to runs** (`map`, `filter`, `sort_by`, `any`, `each`, …) *borrows* its captures read-only. So `items.map(x -> x + offset)` does not consume `offset`, and `people.filter(p -> p.city == city)` leaves `city` usable. A function-typed parameter is non-escaping in this sense unless it is declared `take`.
- An **escaping** lambda — returned, stored in a field or collection, passed to a `take` parameter, or used in `spawn` — *moves* its captures (copy types are copied). Using a captured value after the lambda escaped is the usual use-after-move error, with a `.copy()` suggestion.

A lambda cannot reassign a captured `mut` variable; use a fold, a loop, or a returned value instead.

```
fn make_greeter(greeting)                    # returns an escaping lambda
    words = greeting.copy()                  # the lambda needs its own copy
    name -> "{words}, {name}!"

hello = make_greeter("Hello")
print(hello("Ana"))
```

Named functions are values too: `shapes.map(area)` passes the function `area`.

Tuple auto-spread

A lambda (or function) with *N* parameters may be called with a single *N*-tuple; the tuple is spread across the parameters. This is what makes two-parameter lambdas work with `zip`, `enumerate` and `entries`, which produce pairs:

```
totals = prices.zip(quantities).map((price, qty) -> price * qty)
lines = names.enumerate().map((i, name) -> "{i + 1}. {name}")
report = ages.entries().map((name, age) -> "{name} is {age}")
```

Spreading happens only when the arity matches exactly; a one-parameter lambda receives the whole tuple (`pair -> pair.0`).

`?` inside a lambda

`?` inside a lambda returns from the *lambda*, not from the enclosing function, so the lambda's own return type becomes fallible (see §8.3):

```
doubled = inputs.map(text -> text.trim().to_int()? * 2)      # List<Int | ParseError>
```

5.5 Generic functions

Every unannotated parameter is already an implicit generic (§4.2). Explicit generic parameters name the type so it can be used in several places:

```
fn first<T: Copyable>(items: List<T>): T?
    items.get(0).copy()          # `.copy()` on the optional: an owned `T?`
```

The `Copyable` bound is what allows `.copy()` on a `T` (§10.4): the ownership check runs once, at the definition, and treats `T` as an opaque, non-`Copy` type unless a bound says otherwise (§4.2, rule 9). Without the bound, `items.get(0).copy()` is a compile error that suggests adding `T: Copyable`.

See §10.

5.6 Methods

Methods are functions declared inside an `impl` block for a type. The first parameter is `self`.

```
type Dog
    name: Text
    tricks: List<Text> = []

impl Dog
    fn speak(self)
        "{self.name} says woof"

    fn learn(mut self, take trick)
        self.tricks.push(trick)

    ## A constructor-style function with no `self`.
    fn puppy(take name)
        Dog(name:)

mut rex = Dog.puppy("Rex")
rex.learn("sit")
print(rex.speak())
```

- `self` borrows the receiver read-only.
- `mut self` borrows it exclusively and may change it — including assigning its fields directly, as in `self.name = new_name` or `self.age += 1`. As with `mut` parameters, the arguments are evaluated first and only then does the exclusive borrow of the receiver begin (§5.3), so `self.nodes[p].children.push(self.nodes.len() - 1)` is fine.
- `take self` consumes the receiver (for example `task.wait()`); the receiver moves, so it must not be used afterwards.
- Method calls look the same whether or not they mutate — `cart.add(item)` carries no `mut` marker, deliberately: the receiver is the dotted name in front, so it is already visible. Instead, the **receiver of a `mut self` method must be a `mut` binding** (or a mutable field/element of one); calling it on an immutable binding is a compile-time error that names the binding.
- A function in an `impl` block without `self` is an **associated function**, called as `Type.name(...)`.
- `Self` inside an `impl` names the type being implemented.

A type can have several `impl` blocks, in the same module as the type.

5.7 Uniform call syntax

Any function in scope can also be called with dot syntax, `x.f(y)`, when its first parameter accepts `x`. Then `x.f(y)` means `f(x, y)`:

```
fn shout(text: Text)
    text.upper() + "!"

print("hello".shout())                # same as shout("hello")

report = orders.filter(o -> o.paid).total_by_customer().format_table()
```

- **A method always wins.** If the type of `x` has a method `f` (inherent or from a trait in scope), `x.f(y)` calls the method.
- **Ambiguity is an error.** If two functions in scope could both be called as `x.f(…)`, the call is a compile-time error that lists both and asks you to call one as `module.f(x, …)`.

- Parameter modes carry over: if the first parameter is `mut`, the receiver must be a `mut` binding (and, as with methods, no marker is written); if it is `take`, the receiver moves.

Uniform call syntax is how Verse spells left-to-right pipelines; there is no separate `|>` operator.

---

6. Control flow

6.1 `if` / `else`

```
if balance < 0
    print("Overdrawn")
else if balance == 0
    print("Empty")
else
    print("In credit")
```

A block always starts on the line after its header — there is no colon at the end of `if balance < 0`. Conditions must have type `Bool`; there is no truthiness.

One-line form: `if … then … else …`

When choosing between two values, use the one-line expression form. Both branches are required, and each is a single expression:

```
label = if count == 1 then "item" else "items"
sign = if n < 0 then -1 else if n == 0 then 0 else 1
print("{count} {if count == 1 then "item" else "items"}")
```

The block form is also an expression when it has an `else` and each branch ends in a value:

```
greeting = if hour < 12
    part = "morning"
    "Good {part}"
else
    "Hello"
```

`then` appears only in the one-line form; `else` appears only with `if`.

When an `if` (either form) or a `match` produces a value, its result is a view only if every branch is a view or a literal; otherwise it is owned and each view branch needs `.copy()` — the same rule as `otherwise` (§3.8.2).

6.2 `if … is` — testing a pattern

To run a block only when a value has a certain shape, test it with `is` (§7.6). The names bound by the pattern are available inside the block. The pattern `user?` matches a *present* optional and binds the value inside it:

```
if find_user(id) is user?
    print("Found {user.name}")
else
    print("No such user")

if shape is Circle(r)
    print("A circle of radius {r}")
```

There is no separate pattern-binding form of `if`; `is` is the one way to branch on a pattern.

6.3 `for` loops

`for` iterates over anything that implements the `Iterable` trait: lists, sets, maps, ranges, text (characters), channel receivers, and more.

```
for name in ["Ana", "Bruno", "Carla"]
    print("Hi, {name}")

for i in 0..3
    print(i)                        # 0, 1, 2

for (word, count) in word_counts   # maps yield (key: K, value: V) pairs
    print("{word}: {count}")

for (index, item) in items.enumerate()
    print("{index + 1}. {item}")
```

The loop variable is a fresh immutable binding on every iteration. Patterns may be used to destructure (§7).

Ownership in loops. `for x in items` *borrows* each element while `items` is still used after the loop. When the loop is the collection's last use, the loop takes the collection and each `x` is owned, so it can be moved elsewhere without a copy:

```
mut adults: List<Person> = []
for person in people                # last use of `people`: each person is owned
    if person.age >= 18
        adults.push(person)         # moves the person; no copy needed
```

6.4 Ranges

Range	Contains
`1..10`	1, 2, ..., 9 (end excluded)
`1..=10`	1, 2, ..., 10 (end included)
`(1..10).step(2)`	1, 3, 5, 7, 9
`(1..=10).reverse()`	10, 9, ..., 1

Ranges are values of type `Range<Int>` (also `Range<Char>`), support `in` (`if age in 13..=19`), and can slice lists and text: `items[1..3]`, `name[0..=2]`.

Inside slice brackets only, either end may be left open:

Slice	Meaning
`items[2..]`	from index 2 to the end
`items[..3]`	the first three elements (indices 0, 1, 2)
`items[..=3]`	indices 0 through 3

Open-ended ranges are not values: `r = 2..` and `for i in 1..` are compile-time errors. Slicing past the end of a list panics, like indexing.

6.5 `while`, `break`, and `continue`

```
mut attempts = 0
while attempts < 3
    attempts += 1
    if try_connect()
        break
    print("Retrying…")

for line in lines
    continue if line.starts_with("#")     # postfix guard, §6.7
    process(line)
```

`break` leaves the innermost loop; `continue` skips to its next iteration. A loop may be given a label — written **before** the loop as `label:`, in the "`:` means *is named*" sense — when you need to leave or continue an outer loop:

```
outer: for row in grid
    for cell in row
        if cell == target
            break outer
```

`continue outer` works the same way. Labels may be placed on `for` and `while` loops only.

A `while true` loop with `break` is the idiom for an unbounded loop.

6.6 `pass`

`pass` is an explicit "do nothing" for blocks that must exist syntactically:

```
match event
    Click(x, y) -> handle_click(x, y)
    _ -> pass
```

6.7 Postfix guards

The three exits — `return`, `break` and `continue` — may be followed by `if condition` on the **same line**. The exit happens only when the condition is true:

```
fn safe_ratio(a, b)
    return 0.0 if b == 0.0
    a / b

for line in lines
    continue if line.is_empty()
    break if line == "END"
    process(line)
```

The rules are deliberately narrow: only `return`, `break` and `continue` take a postfix `if`; it fits on one line; there is no `else`; and there is no `unless`. Everything else uses an ordinary `if` block. The guard reads as a sentence ("continue if the line is empty") and keeps the main path of a loop or function unindented. For unwrapping an optional while exiting, use `x = … otherwise return` (§3.8.2). `return`, `break` or `continue` followed by `if` is always a guard; to return a conditional expression, parenthesize it: `return (if c then a else b)` (§3.8).

---

7. Pattern matching

7.1 `match`

`match` compares a value against patterns, top to bottom, and evaluates the arm of the first pattern that fits. Each arm is `pattern -> expression`, or `pattern ->` followed by an indented block.

```
type Shape = Circle(radius: Float) | Square(side: Float)

fn area(s: Shape): Float
    match s
        Circle(r) -> 3.14 * r * r
        Square(side) -> side * side

fn describe(s: Shape): Text
    match s
        Circle(r) ->
            size = if r > 10.0 then "large" else "small"
            "a {size} circle"
        Square(_) -> "a square"
```

`match` is an expression; all arms must produce compatible types. An arm whose body is a lambda must parenthesize it — `Double -> (n -> n * 2)` — because the first `->` after a pattern always ends the pattern (see §5.4).

7.2 Exhaustiveness

The compiler checks that every possible value is handled. Forgetting a variant is a compile-time error that names the missing case:

```
error: match is not exhaustive
  --> shapes.vs:4:5
   |
 4 |     match s
   |     ^^^^^^^ missing case: Triangle(_, _, _)
```

Use `_` to explicitly handle "everything else".

Unreachable arms. An arm whose pattern is a single lowercase name (or `_`) with no guard matches everything. If such an arm is **not the last arm**, every arm after it is reported as an *unreachable arm* error. This catches a common slip — writing `north ->` instead of `North ->`, which would otherwise bind a new variable and swallow every case:

```
error: unreachable arm
  --> compass.vs:5:9
   |
 4 |         north -> "up"
   |         ----- this arm matches every value (`north` is a new variable)
 5 |         South -> "down"
   |         ^^^^^ so this arm can never run
   |
help: did you mean the variant `North`?
```

Even in the **last** position, where a catch-all is legal, a binding whose name is the lowercase spelling of a variant of the matched type (`north` when `North` exists) gets a warning that suggests the variant or a different name.

7.3 Kinds of patterns

Pattern	Matches	Example
Literal	an equal value	`0`, `"quit"`, `'a'`, `true`
Binding	anything, naming it	`x`
Wildcard	anything, discarding it	`_`
Variant	a variant, destructuring fields by position	`Circle(r)`
Named fields	a variant or record by field name	`User(name:, age: 18)`
Tuple	a tuple of matching parts	`(0, y)`
List	a list by shape	`[]`, `[x]`, `[first, ..rest]`
Range	a number or char within range	`0..=9`, `'a'..='z'`
Alternatives	any of several patterns	`"y" | "yes"`
Typed	a union member by type (top level only)	`n: Int`, `err: Error`, `_: Error`
Present	a present optional, binding its value	`user?`, `_?`
`none`	an absent optional	`none`

Field patterns. Inside the parentheses of a variant or record pattern, `name: pattern` **always** means "the field `name` must match `pattern`", and `name:` alone is punning for `name: name` (bind the field to a variable of the same name). `User(name:, age: 18)` binds `name` and requires `age` to equal 18. Fields not mentioned are ignored.

Typed patterns (`n: Int`) are allowed only at the *top level* of an arm, to pick a member of a union. Inside parentheses, a typed binding is written as a field pattern whose pattern is itself typed: `Reply(body: b: Text)`. This keeps `User(age: Int)` from having two readings — it can only mean "field `age` matches the pattern `Int`", which is rejected because `Int` is a type, not a variant.

Present patterns. `p?` matches an optional that is present and binds its value to `p`; `_?` matches any present value. Together with `none` they cover an optional exactly, and they nest: `Person(email: e?)` matches a person whose email is present and binds it to `e`.

```
fn summary(items: List<Int>): Text
    match items
        [] -> "empty"
        [x] -> "just {x}"
        [first, ..rest] -> "{first} and {rest.len()} more"

fn grade(score: Int): Char
    match score
        90..=100 -> 'A'
        80..90 -> 'B'
        70..80 -> 'C'
        _ -> 'F'

fn answer(input: Text): Bool
    match input.lower()
        "y" | "yes" -> true
        _ -> false
```

7.4 Guards

An arm may add a condition with `if`:

```
match point
    (0, 0) -> "origin"
    (x, y) if x == y -> "on the diagonal"
    (x, _) if x < 0 -> "left half"
    _ -> "somewhere else"
```

7.5 Matching optionals and fallibles

```
match find_user(id)
    none -> print("not found")
    user? -> print("hello, {user.name}")     # user: User

match parse_age(input)
    age: Int -> print("age is {age}")
    err: Error -> print("invalid: {err.message()}")

match person
    Person(email: e?) -> send_welcome(e)
    Person(name:, email: none) -> print("{name} has no email")
```

For a nested optional such as `Int??` (§8.1), `x??` matches a present inner value, `none?` matches "present but empty", and `none` matches the outer absence.

7.6 `is` — a pattern as a condition

`value is Pattern` is a `Bool` expression. When used as an `if` condition, the bindings in the pattern are available inside the block. This is the one way to branch on a pattern without a full `match` (there is no separate pattern-binding `if`):

```
if shape is Circle(r)
    print("radius {r}")

if settings["theme"] is theme?
    apply(theme)

circles = shapes.filter(s -> s is Circle(_))
```

Bindings from `is` are visible only in the `if` block, never in `else` or after the `if`. `is` used outside an `if` condition binds nothing.

7.7 Destructuring in bindings and `for`

Patterns that always match (irrefutable patterns) may be used on the left of a binding and in `for`. A pattern always binds new names; the one exception, a parallel assignment to a tuple of `mut` names such as `(a, b) = (b, a)`, is described in §3.9:

```
(x, y) = position
User(name:, age:) = current_user
[head, ..tail] = items               # error: refutable; the list might be empty
for (key, value) in settings
    print("{key} = {value}")
```

A refutable pattern in a binding is a compile-time error; use `match` or `if … is` instead — or, when the failure case should leave the function or loop, the `pattern = … otherwise return/break/continue` form (§3.8.2):

```
[head, ..tail] = items otherwise return none
```

Ownership of bindings. Names bound by a pattern borrow from the matched value. If the matched value is owned and this is its last use, the bindings take ownership of the parts instead, so they can be moved on without copies.

7.8 `match` without a subject

When the choice depends on conditions rather than on the shape of one value, leave out the subject. Each arm is then a `Bool` condition, tried top to bottom:

```
fn sign_label(n)
    match
        n < 0 -> "negative"
        n == 0 -> "zero"
        _ -> "positive"

fee = match
    order.total > 100.0 -> 0.0
    order.express -> 15.0
    _ -> 5.0
```

A subject-less `match` is an expression, uses the ordinary arm syntax (including blocks after `->`), and **must end with a `_ ->` arm**; without it the compiler reports "match is not exhaustive", because no set of conditions can be proved to cover every case. It replaces long `if … else if … else` chains that choose a value.

8. Errors and missing values

Verse has two related, but distinct, ideas:

- **Missing values** — "there may be nothing here" — use optional types `T?`.
- **Errors** — "something went wrong, and here is why" — use fallible types `T | E`.

Neither involves exceptions or null. Two forms of `?`, one pattern and one word, `otherwise`, cover almost everything:

Form	Where	Meaning
`T?`	type	a value that may be missing
`expr?`	postfix	stop early: if `expr` is an error or missing, return it from the current function
`x?`	pattern	match a present value (§7.3)
`a otherwise b`	expression	fallback: `a` if usable, else `b`

A plain dot looks through an optional when the chain is consumed by one of these (§8.1); there is no separate safe-access operator.

8.1 Optional types `T?`

There is no `null`. A value of type `T` is always a real `T`. A value that might be absent has type `T?`, and is either a `T` or `none`.

```
fn find_user(id: Int): User?
    users.find(u -> u.id == id).copy()      # `users`: a module constant; `find`
                                            #   returns a view (§9.5), so copy it

maybe: Text? = none
present: Text? = "here"         # a T is automatically a T?
```

You cannot use a `T?` where a `T` is expected; the compiler insists you decide what to do about `none`. The tools for this are:

Tool	Meaning	Example
`otherwise`	fallback value	`nickname otherwise "friend"`
`if … is x?`	branch on presence	`if find_user(id) is u?`
`match`	full control	`none -> …` / `u? -> …`
`?`	propagate `none` out of a function returning an optional	`user?`
`.expect(msg)`	assert presence; stops the program with `msg` otherwise	`config.expect("config required")`
`.or_error(msg)`	turn `none` into an error, usually followed by `?` (§8.3)	`find_user(id).or_error("no such user")?`

Plain dots through optionals

There is no `?.` operator. A plain dot (a field, a method call, a chain of them) may look *through* an optional, but only where the chain is **immediately consumed** by something that deals with `none`:

The chain is consumed by…	Example
(a) `otherwise`	`city = order.customer.address.city otherwise "unknown"`
(b) postfix `?`	`email = find_user(id).email?` (returns `none` from the function)
(c) an `is` pattern or a `match` subject	`if order.customer.address is a?`
(d) a position whose expected type is optional	`city: Text? = order.customer.address.city`; `return user.email` in a function returning `Text?`; an argument to a `Text?` parameter

Anywhere else, a dot on a `T?` is a compile-time error that suggests the four ways out — a fallback with `otherwise`, propagation with `?`, a branch with `is`, and, for an unannotated binding, annotating the optional type:

```
error: `order.customer` may be missing, so `.address` cannot be read here
  --> orders.vs:3:23
   |
 3 | city = order.customer.address.city
   |        ^^^^^^^^^^^^^^^ this is a `Customer?`
   |
help: say what happens when it is missing:
        city = order.customer.address.city otherwise "unknown"
        city = order.customer.address.city?           (return none)
        if order.customer is c?  …                     (branch)
      or keep it optional:   city: Text? = order.customer.address.city
```

The rules of a lifted chain:

- **It short-circuits completely.** At the first `none`, the rest of the chain is not evaluated — later hops, method calls, and their arguments.
- **It yields one level of optional.** A hop that itself returns `X?` does not create `X??`: "something along the chain was missing" is the only question a chain answers, so `order.customer.nickname` (with `nickname: Text?`) is a plain `Text?`. Lookups keep their own types: `m.get(k)` on a `Map<Text, Int?>` is still `Int??` (below).
- **Ownership follows the view rule** (§9.5): views flow along the chain and the end of the chain decides, exactly as for chains without optionals.
- **The optional's own methods win.** `.expect(msg)`, `.or_error(msg)` and `.copy()` on a `V?` act on the optional itself, not on the value inside: `find_user(id).copy()` is an owned `User?`. If a name is both a method of the optional and a field or method of the inner type, the call is a compile-time error that asks you to choose with `is` or `otherwise`.
- **`.or_error(msg)` applies to the final result of the chain.** In `order.customer.address.city.or_error("no city")?`, a `none` anywhere along the chain becomes the error.

Postfix `?` is an ordinary postfix operator, applied left to right like a call or a field access (§3.8). A `?` **immediately followed by** `.` is a compile-time error, because other languages read `?.` as safe access: "`?.` is not Verse; write `(a?).b` to propagate then read, or `a.b otherwise …` / `a.b?` to look through the optional". The parenthesised `(find_user(id)?).email` is valid: it propagates a missing user out of the function and then reads `.email` from the `User`. `find_user(id).email?` reads through the optional and propagates if anything along the chain is missing. `verse fmt` never removes these parentheses.

Nested optionals are real

Style: avoid nested optionals such as `T??` in public APIs; prefer a variant that names each case (`type Reply = NoSuchKey | Pending | Value(v: Int)`). The `none?` and `score??` patterns below exist for generic code.

Outside lifted chains, optionals are **never** flattened: `T??` is a different type from `T?`. This matters for lookups. `map[key]` and `list.get(i)` return `V?`, where `V` is the element type — so on a `Map<Text, Int?>` a lookup has type `Int??`, and "the key is missing" (`none`) stays distinguishable from "the key is present and its value is `none`" (a present `none`):

```
scores: Map<Text, Int?> = {"ana": 7, "bruno": none}

match scores["bruno"]          # Int??
    none -> print("no such player")
    none? -> print("bruno has not played yet")
    score?? -> print("score {score}")
```

On ordinary maps and lists (whose elements are not optional), lookups are plain `V?` and none of this comes up.

8.2 Fallible types `T | E`

A function that can fail declares it in its return type:

```
fn read_text(path: Text): Text | Error      # signatures only
fn parse_port(text: Text): Int | ParseError
```

`ParseError` is defined in `std.num`; it is the error type of `Text.to_int()`, which returns `Int | ParseError` (and of `Text.to_float()`, which returns `Float | ParseError`).

The body returns either a `T` (success) or an error value. A value of the error type returned as the last expression, or with `return`, is the failure:

```
fn divide(a: Float, b: Float): Float | Error
    if b == 0.0
        return error("division by zero")
    a / b
```

The `Error` trait

Error types implement the built-in `Error` trait:

```
trait Error: Show
    fn message(self): Text
    fn cause(self): Error?
        none
```

`T | Error` (with the trait name itself) accepts *any* error type — the most common choice for application code. Libraries often define specific error types:

```
type FetchError = NoResponse(seconds: Int) | BadStatus(code: Int) | Offline

impl Error for FetchError
    fn message(self): Text
        match self
            NoResponse(s) -> "no response after {s}s"
            BadStatus(c) -> "server answered {c}"
            Offline -> "no network connection"
```

The built-in function `error(message: Text, cause: Error? = none): Error` creates a simple error value.

8.3 Propagation with `?`

Postfix `?` means **"stop early: if this is an error or missing, return it from the current function; otherwise give me the value."**

```
import std.json
import read_text from std.fs

fn load_settings(path: Text): Settings | Error
    text = read_text(path)?
    data = json.parse(text)?
    Settings.from_json(data)
```

Rules for `expr?`:

`expr` has type	enclosing function returns	effect
`T | E`	`U | E`	on error, return that error; otherwise yields `T`
`T | E`	`U | Error`	on error, return it as `Error`; otherwise yields `T`
`T | E`	`U?`	compile-time error — an error cannot become `none`; handle it, or change the return type to `U? | Error`
`T | E1`	`U | E2` (unrelated `E2`)	compile-time error — convert with `.context("…")` (gives `Error`) or `match`
`T?`	`U?`	on `none`, return `none`; otherwise yields `T`
`T?`	`U | E`	compile-time error — say why with `.or_error("…")`
`Error` (widened)	`U | FetchError`	compile-time error — `?` never narrows `Error` back to a specific type (§4.6)

Layers are peeled from the outside, in written order. For a type with several layers, `?` and `otherwise` act on the *outermost* layer only. In `Text? | Error`, the outermost layer is the error side:

```
fn nickname(id: Int): Text? | Error
    ...

a = nickname(7)?                              # Text?  (errors propagated)
b = nickname(7) otherwise none                # Text?  (errors replaced by none)
c = nickname(7) otherwise "anonymous"         # error: right side must be `Text?`
d = nickname(7)? otherwise "anonymous"        # Text   (error propagated, then none
handled)
e = nickname(7) otherwise none otherwise "anonymous"       # Text
```

The same rule makes an `Int??` need two steps. In expressions, two postfix `?` in a row are not allowed — write `(x?)?` — or use `x otherwise none otherwise 0`. (`x??` exists only as a *pattern*, meaning "present and present", §7.5.)

Views and `otherwise`. `a otherwise b` is a view when both sides are views (text and number literals and other static data count as views), and owned when the right side is an owned temporary; in that case a view on the left needs `.copy()`, and leaving it out is a compile-time error that suggests it (§3.8.2):

```
greeting = args.first() otherwise "world"                  # a view
label = labels["x"].copy() otherwise build_label()         # owned
wrong = labels["x"] otherwise build_label()                # error: the left side
                                                           #   is a view; write
                                                           #   labels["x"].copy()
```

The full table, including an owned left side with a non-static view on the right (copy the right side), literals materialised in owned positions, and lifted chains on owned temporaries, is in §3.8.2.

Where `?` returns to. `?` always returns from the *innermost* function-like body that contains it:

`?` appears in…	it returns from…
a named function	that function
a lambda	the lambda (its return type becomes fallible or optional; §5.4)
a function run with `spawn`	the spawned function; the error is delivered by `.wait()` (§11.1)
a `test` block	the test, which then fails with the error's message (§14.1)
entry-file top-level code	the script, which exits (see below)

To turn a missing value into an error explicitly:

```
user = find_user(id).or_error("no user with id {id}")?
```

At the **top level of a script** (outside any function), `?` on a failed value prints the error message to standard error and exits with status 1. On `none` it prints `error: missing value at file.vs:LINE` and exits with status 1.

8.4 Handling errors

```
import std.fs
import read_text from std.fs

# Fallback value
notes = read_text("notes.txt") otherwise ""

# Branch on success or failure
match read_text("notes.txt")
    text: Text -> print(text)
    err: Error -> print("could not read notes: {err.message()}")

# Add context while propagating
config_text = read_text(path).context("loading {path}")?

# Ignore a result on purpose (the only way; see below)
_ = fs.remove("cache.tmp")
```

`.context(msg)` wraps the error in a new error whose `cause()` is the original.

Fallible results cannot be dropped

Discarding a fallible value is a compile-time error. A call whose result is `T | E` cannot stand alone as a statement, and neither can a value that *contains* fallible results — a `List<T | Error>` from `wait_all`, a `Map<K, V | E>`, a tuple with a fallible part. Either handle it (`?`, `otherwise`, `match`, a loop over the results) or discard it visibly with `_ = …`:

```
fs.remove("cache.tmp")                # error: result `Nothing | Error` is ignored
                                      #   help: add `?`, handle it, or write `_ = …`
fs.remove("cache.tmp")?               # propagate
_ = fs.remove("cache.tmp")            # deliberately ignored
```

8.5 Panics

Some failures are bugs, not conditions to handle: indexing a list out of range, integer overflow, dividing an integer by zero, a failed `assert`, or `.expect(...)` on `none`. These **panic**: the current task stops (running its `with` cleanups), the message and a stack trace are **always printed to standard error at the moment of the panic**, and — if the task is the main program — the program exits with status 101. A panic in a spawned task is delivered to its waiter as a `TaskPanicked` error (§11.2). Panics are not caught by `?` and are not meant for normal control flow. Code that wants to avoid them uses the checked alternatives (`items.get(i)`, `a.checked_div(b)`).

---

9. Collections

9.1 Lists

`List<T>` is an ordered, growable sequence.

```
nums = [1, 2, 3]
print(nums[0])                    # 1      — Int; out of range panics
print(nums.get(10))               # none   — Int?
print(nums.get(10, default: 0))   # 0      — Int
print(nums.len())                 # 3
print(nums[1..])                  # [2, 3]
print(nums[..2])                  # [1, 2]
print(nums + [4, 5])              # [1, 2, 3, 4, 5]

mut queue: List<Text> = []
queue.push("first")
queue.push("second")
next = queue.pop_front()       # Text?
queue[0] = "changed"
```

Negative indices are not allowed; use `nums.last()` (which returns `T?`).

9.2 Maps

`Map<K, V>` associates keys with values. Insertion order is preserved when iterating.

```
ages = {"ana": 30, "bruno": 17}
print(ages["ana"])                    # 30 — Int? (present)
print(ages["zoe"])                    # none
print(ages.get("zoe", default: 0))    # 0  — Int

mut stock: Map<Text, Int> = {:}
stock["apples"] = 10                                # insert or replace
stock["apples"] = stock.get("apples", default: 0) + 5
_ = stock.remove("pears")                           # remove returns the old value, Int?
print("apples" in stock)                            # true
```

Writing `map[key] = value` inserts or replaces. Keys must implement `Hash` and `Eq`. Iterating a map — or calling `map.entries()` — yields labeled pairs `(key: K, value: V)`, which can be destructured (`for (name, age) in ages`), passed to a two-parameter lambda (§5.4), or read by label (`entry.key`, `entry.value`). An empty map is written `{:}`; a bare `{}` is a compile-time error that points to `{:}` and `set{}`.

9.3 Lookups side by side

Lists and maps answer "what is at this position/key?" differently, on purpose:

Expression	On a `List<T>`	On a `Map<K, V>`
`c[i]` / `m[k]`	`T` — **panics** if out of range	`V?` — `none` if the key is missing
`c.get(i)` / `m.get(k)`	`T?`	`V?` (same as `m[k]`)
`c.get(i, default: x)` / `m.get(k, default: x)`	`T` — `x` if out of range	`V` — `x` if missing
`c[i] = v` / `m[k] = v`	replace (panics if out of range)	insert or replace

A bad list index is almost always a bug (and loops over `0..items.len()` should not need unwrapping), while a missing map key is ordinary data — see §18.11.

The one-argument `get` returns `V?` **without flattening**: on a `Map<Text, Int?>` it returns `Int??` (§8.1). The two-argument `get(key, default: x)` always returns a plain `V`, never an optional, and is the idiomatic way to count or accumulate:

```
counts[word] = counts.get(word, default: 0) + 1
```

`default:` is a parameter of `get` only; it is not a general language feature. `get(k, default: x)` follows the same ownership table as `otherwise` (§3.8.2), with the element as the left side and the default as the right:

Element	Default	Result
a view (always, from a borrowed collection)	a view or a static literal (`0`, `""`)	a **view**: `names.get(id, default: "anonymous")`
a view	an owned temporary (`Text.new()`, `make_label()`)	**owned**, so the element side needs `.copy()`: write `names.get(id, default: Text.new())` as `names.get(id).copy() otherwise Text.new()`

Without the copy, the second row is a compile-time error: a view of an owned temporary default would not outlive the call. `names.get(id).copy() otherwise Text.new()` is the intended way to get an owned element with an owned default. It works because `.copy()` on an optional view `V?` of a `Copyable` `V` yields an owned `V?` (`none` stays `none`), which then meets the owned default as in the second row of the `otherwise` table (`.copy()` is one of the optional's own methods, §8.1). Where an owned value is needed — storing it, returning it, sending it to a task — copy the view result: `names.get(id, default: "anonymous").copy()`.

Ownership of lookups. `list[i]`, `m[k]` and `get` give a *view* of the element — a second-class borrow that can be read, passed to functions, or bound to a name, but not stored in a field or collection, returned, or sent to a task. To keep an element beyond that, copy it (`m["ana"].copy()`) or take it out of a `mut` collection with `remove`/`pop`, which return owned values. `Copy` types (numbers, `Bool`, `Char`, and records made only of them) are simply copied.

Slices are views too. `nums[1..]`, `nums[2..5]` and `name[0..=2]` give a view of part of the list or text, with the same rules as `list[i]`. Write `nums[1..].copy()` or `name[0..=2].copy()` to get an owned list or text.

9.4 Sets

`Set<T>` is an unordered collection of unique values.

```
vowels = set{'a', 'e', 'i', 'o', 'u'}
seen: Set<Text> = set{}
print('e' in vowels)                  # true
both = set{"a", "b"}.union(set{"b", "c"})       # set{"a", "b", "c"}
```

Set operations: `union`, `intersection`, `difference`, `is_subset`, plus `+` (union) and `-` (difference) as conveniences.

9.5 Transforming collections

Collections share a rich family of methods (from the `Iterable` trait) that chain with dots and take lambdas. Chains read top to bottom like a recipe. A chain that is bound to a name or returned is evaluated eagerly and collected into a `List` at the end of the statement; only a chain used as a `for` target or consumed in the middle of a longer chain stays lazy, so a lazy iterator value is never stored, bound or returned. A collected chain that ends in an owning step is an owned `List<…>`; one that ends in adapters only is a local view (see *Ownership* below):

```
total = numbers.filter(n -> n % 2 == 0).map(n -> n * 2).sum()

top_scorers = players
    .filter(p -> p.active)
    .sort_by(.score, descending: true)
    .prefix(3)
    .map(.name.copy())
```

Method	Result
`map(f)`	apply `f` to each element
`filter(pred)`	keep elements where `pred` is true
`reduce(initial, f)`	combine into one value
`sum()`, `product()`, `average()`	numeric summaries
`min()`, `max()`, `min_by(key)`, `max_by(key)`	the smallest or largest element, `T?` (a view)
`count()`, `count(pred)`	number of elements (matching)
`any(pred)`, `all(pred)`	`Bool`
`find(pred)`	first match, `T?` (a view)
`first()`, `last()`	`T?` (a view)
`sort()`, `sort_by(key, descending: false)`	a sorted list
`group_by(key)`	`Map<K, List<T>>`
`prefix(n)`, `skip(n)`, `prefix_while(p)`, `skip_while(p)`	prefixes and suffixes
`enumerate()`	`(index, item)` pairs
`zip(other)`	pairs from two collections
`flat_map(f)`, `flatten()`	flatten nested collections (an owning step)
`unique()`	remove duplicates, keeping first occurrences
`reverse()`	the elements in reverse order
`chunks(n)`, `windows(n)`	groups of elements, each a view of a slice (§9.3)
`join(sep)`	`Text` from a collection of `Show` values
`to_list()`, `to_set()`, `to_map()`	collect into a concrete collection (an owning step)

Ownership: views flow through a chain; the end of the chain decides. These methods borrow the collection they are called on and pass each element to the lambda as a borrow. Nothing is ever copied silently:

- **Adapters pass views.** Within one expression chain, the methods that hand elements along (`filter`, `sort`, `sort_by`, `prefix`, `skip`, `prefix_while`, `skip_while`, `unique`, `reverse`, `zip`, `enumerate`, …) pass second-class views of the source's elements. Nothing is copied.
- **The chain ends in an owning step**, which decides what the result owns:
- `map(f)` — the result holds what the lambda returns, which must be owned: `.map(p -> p.age)` copies an `Int` (a `Copy` type), while `.map(p -> p.name)` is an error that asks for `.map(p -> p.name.copy())` (or the key path `.map(.name.copy())`);
- a reduction (`sum`, `count`, `join`, `any`, `all`, `reduce`, …), which produces a new value;
- a move out of an owned source at its last use (a temporary, or a binding that is not used afterwards), which moves the elements instead of copying them;
- an explicit `.copy()`, which duplicates the remaining elements: `people.filter(p -> p.active).copy()` is an owned `List<Person>`;
- a collecting step — `to_list()`, `to_set()`, `to_map()`, `flat_map(f)`, `flatten()` — which builds a new owned collection: it moves the elements from a source at its last use, and otherwise needs `.copy()` before it. A bound or returned chain is collected into an owned list at the end of the statement, and side effects in its lambdas run then: `tasks = urls.map(url -> spawn fetch(url.copy()))` is a `List<Task<…>>` whose tasks are spawned when that statement executes. `to_list()` is needed only where the chain would otherwise be consumed as something else — as a `for` target, or to name the element type — not for binding. (`chunks(n)` and `windows(n)` are not owning steps: each group is a view of a slice of the source, §9.3.)
- **No implicit duplication of non-`Copy` elements.** If a chain would have to duplicate an element that is not `Copy` to produce an owned result — for example `adults: List<Person> = people.filter(p -> p.age >= 18)` while `people` is still used later — that is a compile error with the usual suggestion: add `.copy()` (which needs `Copyable`, §10.4) or move the later use. `Copy` elements (numbers, `Bool`, `Char`, …) are copied implicitly as everywhere else.
- **Single-element results are views.** `find`, `first`, `last`, `min`, `max`, `min_by` and `max_by` return a view of the element, as `list[i]`, `m[k]` and `get(k, default: x)` do (§9.3). So `people.find(p -> p.id == id).email otherwise "none"` copies nothing, and keeping the element is written `people.find(p -> p.id == id).copy()` (an owned `Person?`; `.copy()` acts on the optional, §8.1). That is the rule for a *borrowed* source. On an **owned source at its last use** — a `take` parameter or a temporary — the single element **moves out** and the rest of the collection is freed; this is the owned counterpart of the view rule (see `largest` in §20.3).
- **A binding to a chain is a local view.** `adults = people.filter(…)` is allowed when `people` is used later: `adults` is a view that lives only in the current scope, exactly like `b = a` (§4.9). It can be read, iterated and passed to functions, but not stored, returned or sent to a task.

```
adults = people.filter(p -> p.age >= 18)          # a local view; nothing copied
print(adults.len())
names = adults.map(.name.copy())                  # the one visible copy
oldest = people.max_by(.age).copy()               # an owned Person?
print(people.len())                               # `people` is still usable
```

Key functions. Comparison key functions (`sort_by`, `min_by`, `max_by`) may return views, so `people.sort_by(.name)` needs no copy of each name. `group_by` stores its keys in the resulting map, so its key function must return an owned value: `people.group_by(.city.copy())`. Its groups hold the elements themselves, so `group_by` is an owning step: it moves from a source at its last use, or needs `.copy()` before it.

`min()`, `max()` and `average()` on a possibly empty collection return `T?`; on a list known non-empty by the caller, combine with `otherwise` or `.expect()`.

9.6 Equality and ordering

`==` and `!=` compare **structurally**: two values are equal when they have the same type and equal contents. Because every value has a single owner and there are no references in the language, identity is never observable, so there is no separate identity comparison.

```
assert [1, 2] == [1, 2]
assert User(name: "A", age: 1) == User(name: "A", age: 1)
assert {"a": 1} == {"a": 1}
```

Built-in types, tuples, lists, records and variants whose parts are comparable automatically implement `Eq`; types whose parts are ordered automatically implement `Order` (lexicographic for tuples and lists; declaration order for variants; field order for records). Comparing values of different types is a compile-time error, with one convenience: a `T` may be compared with a `T?` (so `counts["the"] == 3` works on a map lookup). `Float` equality follows IEEE 754 (`nan != nan`), so `Float` implements `Eq` only partially and cannot be a map key or set element; use `std.num.OrderedFloat` if needed.

---

10. Traits and generics

10.1 Traits

A trait names a set of methods that several types can share. A method without a body is **required**; a method with a body is a **default** that implementations may override. Because a required method has no body to infer from, its parameter and return types are always written in the trait; implementations may leave them out, since the trait already fixes them.

```
trait Speaker
    fn speak(self): Text

    fn speak_twice(self): Text
        "{self.speak()} {self.speak()}"
```

A type implements a trait with `impl Trait for Type`:

```
type Dog
    name: Text

type Robot
    model: Int

impl Speaker for Dog
    fn speak(self): Text
        "{self.name} says woof"

impl Speaker for Robot
    fn speak(self): Text
        "Unit {self.model} says beep"
```

A type's own methods go in a plain `impl Type` block; trait methods go in `impl Trait for Type`. An implementation must appear in the module that defines either the trait or the type (this keeps implementations easy to find and prevents conflicts between libraries).

10.2 Supertraits

A trait can require another trait:

```
trait Describe: Show
    fn describe(self): Text
        "<{self}>"
```

Every type implementing `Describe` must also implement `Show`.

10.3 Traits as types

A trait name used as a type means "any value whose type implements this trait". This allows collections of different types that share behavior:

```
speakers: List<Speaker> = [Dog(name: "Rex"), Robot(model: 7)]
for s in speakers
    print(s.speak())
```

Calls through a trait type are dispatched at run time.

10.4 Built-in traits

Trait	Provides	Automatically implemented?
`Show`	`show(self): Text`, used by interpolation and `print`	yes, structurally, for all types (override for custom text)
`Eq`	`==`, `!=`	yes, when all parts are `Eq`
`Order`	`<`, `<=`, `>`, `>=`, `sort()`	yes, when all parts are `Order`
`Hash`	use as map key / set element	yes, when all parts are `Hash`
`Iterable<T>`	`for … in`, and all the methods of §9.5	lists, maps, sets, ranges, text, channel receivers
`Default`	`Type.default()`	when all fields have defaults
`Cleanup`	`cleanup(mut self)`, run automatically at the end of the owner's scope and by `with` (§13)	no
`Copy`	implicit copying on assignment and passing (§4.9); every `Copy` type is also `Copyable`	yes, for scalars and for records, variants and tuples made only of `Copy` parts; cannot be implemented by hand
`Copyable`	the `.copy()` method, with no arguments (a duplicate) or with field arguments on records (a duplicate with changes, §4.3)	yes, structurally, for every type whose parts are all `Copyable`; **absent** for `Task<T>`, for types that implement `Cleanup`, and for anything that contains them by value (a record field, variant, tuple or collection element); cannot be implemented by hand in 0.1. The handles `Shared<T>`, `Guarded<T>`, `Sender<T>`, `Weak<Shared<T>>` and `Weak<Guarded<T>>` are always `Copyable`, whatever `T` is — including resources, as in `Guarded<File>` — because their `.copy()` copies the handle (a new reference to the same shared value) and never duplicates the value inside (§11.4); `Receiver<T>` is not `Copyable` in 0.1 (single consumer)
`Index<K, V>`	`x[k]` reads (§9.3)	lists, maps, text and slices; cannot be implemented by hand in 0.1
`Error`	`message(self)`, `cause(self)`	no
`Add`, `Sub`, `Mul`, `Div`, `Rem`	`+`, `-`, `*`, `/`, `%`	`Int`, `Float`, `std.num` types; `Add` also for `Text` and `List` (concatenation)
`Neg`	unary `-`	`Int`, `Float`, `std.num` types

`Copyable` is deliberately closed in 0.1. It cannot be implemented by hand, and a type that implements `Cleanup` is deliberately **not** `Copyable`: duplicating a file handle, a lock guard or a temporary directory would run its cleanup twice or leave two owners of one external resource. A type that does need duplication offers an explicit method with its own name and

its own meaning, such as `duplicate()` (for example, reopening the same file), so the call site says what really happens.

The arithmetic traits are how numeric types in `std.num` (such as `Decimal` and `BigInt`) and user types like vectors or money get operators. They are the only operator overloading Verse allows (§18.5); there is no way to define new operators or to overload `and`, `or`, `not`, `otherwise`, `==` (which comes from `Eq`) or indexing.

```
type Vector
    x: Float
    y: Float

impl Add for Vector
    fn add(self, other: Vector): Vector
        Vector(x: self.x + other.x, y: self.y + other.y)

v = Vector(x: 1.0, y: 2.0) + Vector(x: 0.5, y: 0.5)
```

10.5 Generic functions and types

Generic parameters go in angle brackets after the name.

```
fn first<T: Copyable>(items: List<T>): T?
    items.get(0).copy()           # `get` returns a view; the caller gets a copy

type Box<T>
    value: T

type Pair<A, B>
    left: A
    right: B

type Tree<T> = Leaf | Node(left: Tree<T>, value: T, right: Tree<T>)
```

Generic arguments are usually inferred: `first([1, 2, 3])` is `Int?`, and `Box(value: "hi")` is a `Box<Text>`. Write them explicitly when needed: `Box<Int>(value: 0)`, `names: List<Text> = []`.

Free generic functions do not take explicit type arguments in 0.1: their type arguments are inferred, or given by annotating the binding (`n: Int = parse(text)?`). `parse<Int>(text)` is a compile-time error whose help suggests that annotation. Explicit type arguments are allowed only on an associated function called through a type name, such as `Channel.new<Int>(capacity: 10)` (§11.4).

Methods on a generic type declare the parameters on the `impl`:

```
impl<T> Box<T>
    fn map<U>(self, f: fn(T): U): Box<U>
        Box(value: f(self.value))
```

10.6 Constraints

A generic parameter can require traits with `:`; several traits are joined with `+`.

```
fn largest<T: Order + Copyable>(items: List<T>): T?
    items.max().copy()            # `max` returns a view (§9.5); return a copy

fn announce<T: Speaker + Show>(who: T)
    print("{who}: {who.speak()}")

impl<T: Show> Show for Box<T>
    fn show(self): Text
        "Box({self.value})"
```

Generics are checked once, at definition: inside `largest`, only operations promised by `Order` and `Copyable` may be used on `T`. That includes the **ownership check**: a generic (or unannotated) parameter type is treated as an opaque, non-`Copy` type unless it has a `Copy` bound, so `T` values are never copied implicitly, and `.copy()` needs `Copyable` (§4.2, rule 9). Errors about a function's body appear at the definition. The only call-site errors are unmet bounds, written or inferred, and they name the bound. The compiler specializes generic code for speed; this is invisible to the programmer.

`Copy` and `Copyable` are different promises: `T: Copy` means a `T` is duplicated implicitly (numbers, all-scalar records); `T: Copyable` means a `T` can be duplicated explicitly with `.copy()` (almost everything, but not a `Task` or an open `File`).

---

11. Concurrency

11.1 Spawning tasks

`spawn` starts a function call running **concurrently** in a lightweight task and immediately returns a `Task<T>`, where `T` is the call's return type. `.wait()` blocks the current task until the result is ready.

Errors in a task surface at `.wait()`. Because anything can go wrong in another task — including a panic — `.wait()` always returns a fallible value, `T | Error`, which you handle like any other: with `?`, `otherwise`, or `match`. If the spawned function is itself fallible, its errors arrive on the same error side. For a function returning `Text | FetchError`, `.wait()` would return `Text | FetchError | Error`, which normalizes to `Text | Error` (§4.6); use `match` with `err: FetchError ->` to recover the specific type. `?` inside the spawned function returns from that function, and the error is what `.wait()` later reports.

```
import get from std.net

fn fetch(url)
    response = get("https://{url}")?
    response.body          # a field moved out of `response` at its last use (§4.9)

a = spawn fetch("a.com")
b = spawn fetch("b.com")

print(a.wait() otherwise "a failed")    # handle the failure here
page_b = b.wait()?                      # or stop early and pass it up
print(page_b)
```

There is **no `async`/`await` and no function coloring**. Any function can be spawned, and any function can call blocking operations like `get` or `read_text`: while one task waits for I/O, the runtime runs other tasks. Tasks are multiplexed over all CPU cores.

The arguments of a spawned call are evaluated *before* the task starts, in the spawning task. `spawn` applies to exactly one call expression; to spawn a small block of work, give it a name with `fn` and spawn the call.

Ownership across tasks

- **`spawn f(x)` moves `x` into the task.** The task owns its arguments for the whole call, so a spawned function's parameters are always owned values, even when the function declares them as plain borrows.
- **A borrowed value cannot cross into a task.** Inside a function, a borrowed parameter (or a view from `list[i]`, a `for` loop over a still-used list, or a key-path) cannot be passed to `spawn`; pass `.copy()` instead. Tasks cannot borrow from the spawner in 0.1.
- **No `mut` arguments.** `spawn f(mut x)` is a compile error: an exclusive borrow must end when the call returns, and a task cannot hand a `mut` borrow back to the spawner. Move the value in and get the changed value back from `wait()`, or share it through a `Guarded<T>`.
- **Using a value after spawning it is a use-after-move error.** In a loop, the error is reported on the second iteration and suggests `.copy()`:

```
verse config = load_config() for url in urls spawn crawl(url.copy(), config) # error: `config` moved into the task in # one iteration, used again in the next # help: spawn crawl(url.copy(), config.copy())
```

- **Handles are copied explicitly, and cheaply.** `Sender<T>`, `Shared<T>`, `Guarded<T>` and `Weak<Shared<T>>` / `Weak<Guarded<T>>` values are handles. Passing one to `spawn` follows the same rule as every other value — it moves — so to keep using it, pass `handle.copy()`: a new handle to the same channel or shared value, not a copy of the value inside (§11.4). There is no implicit copy on `spawn`. A `Receiver<T>` is not `Copyable`: it is moved into the one task that consumes the channel.

11.2 Working with many tasks

```
import wait_all, wait_any from std.task

tasks = urls.map(url -> spawn fetch(url.copy()))       # `url` is a borrow: copy it
pages = wait_all(tasks)                  # takes the tasks; List<Text | Error>, in order

mirrors = mirror_urls.map(url -> spawn fetch(url.copy()))
fastest = wait_any(mirrors)?             # the first to finish; the others are cancelled
```

Operation	Signature	Meaning
`task.wait()`	`wait(take self): T | Error`	block until done; consumes the task
`task.wait(timeout: d)`	`wait(take self, timeout: Duration): T | Error`	the same, failing with a `Timeout` error if it takes too long
`task.is_done()`	`is_done(self): Bool`	never blocks
`task.cancel()`	`cancel(self)`	request cancellation (§11.5)
`wait_all(tasks)`	`wait_all<T>(take tasks: List<Task<T>>): List<T | Error>`	results in task order
`wait_any(tasks)`	`wait_any<T>(take tasks: List<Task<T>>): T | Error`	the first result to complete; the rest are cancelled

Remember that `List<T | Error>` contains fallible results, so the value of `wait_all` cannot be discarded (§8.4) — loop over it, or `_ =` it on purpose.

Panics in tasks. If a task panics, the panic message and stack trace are **printed to standard error immediately, at the moment of the panic** — whether or not anyone handles it later. The waiter then receives a value of the named error type `TaskPanicked` (from `std.task`, implementing `Error`, with `message()` and `trace()`), which `match` can tell apart from ordinary errors:

```
import TaskPanicked from std.task

match task.wait()
    page: Text -> show(page)
    crash: TaskPanicked -> alert_on_call(crash.message())
    err: Error -> print("fetch failed: {err.message()}")
```

The task boundary is the one place where a panic becomes a value, so one failing request cannot bring down a whole server — but because the panic was already printed, `task.wait() otherwise "failed"` can never hide a bug.

11.3 Structured lifetimes

Tasks belong to the function (or script) that spawned them. When that function finishes, any of its tasks that were never waited on are **waited for automatically** before it returns. If such an unwaited task failed (with an error, a panic, or cancellation), its error is reported on standard error and the **process will exit with a non-zero status** (1, or 101 for a panic) when it finishes, even if everything else succeeded. This status applies only when the program ends naturally: an explicit `exit(code)` always wins, and its code is the one the process returns. Discarding a `Task` value (a bare `spawn worker(…)` statement) is therefore allowed — an accepted exception to the rule that fallible results cannot be dropped, because its outcome cannot be lost. A task therefore never outlives the code that started it, results are never silently lost, and a program ends only after all of its work is done. To deliberately run something in the background beyond the current function, return its `Task` or pass it to someone who will wait on it.

11.4 Sharing data safely

A task owns what it was given (§11.1), so two tasks can never touch the same value by accident, and data races are impossible. Sharing is **opt-in** and always goes through one of these handle types from `std.task`:

Handle	Meaning	Copying the handle
`Sender<T>`	the sending end of a channel: send owned values to the task that holds the `Receiver`	a new sender to the same channel
`Receiver<T>`	the receiving end of a channel; single consumer in 0.1	not copyable; it moves (fan-out uses several channels)
`Shared<T>`	**shared read-only ownership**: the value is freed when the last handle goes away (atomically reference-counted)	shares the value; nobody can change it
`Guarded<T>`	**shared mutable state behind a lock**: read it inside `with g.lock() as value`, change it inside `with g.lock() as mut value`	shares the value
`Weak<Shared<T>>`, `Weak<Guarded<T>>`	a **non-owning** handle, parameterised over the handle it was made from: `h.downgrade()` on a `Shared<T>` gives a `Weak<Shared<T>>`, and on a `Guarded<T>` a `Weak<Guarded<T>>`; `upgrade()` returns `Shared<T>?` or `Guarded<T>?` respectively (`none` if the value is gone)	a new weak handle to the same value; keeps nothing alive
`Task<T>`	the result of `spawn` (§11.1)	not copyable; `wait` takes it

`Sender`, `Shared`, `Guarded` and `Weak` are `Copyable` (§10.4) whatever their inner type — even `Guarded<File>` — and their `.copy()` copies the **handle**: it returns a new reference to the same shared value and never duplicates the value inside. As everywhere else, the copy is written explicitly — including when a handle is passed to `spawn`. `Receiver<T>` is not `Copyable` in 0.1. `Weak` accepts only a `Shared` or `Guarded` handle; `Sender` and `Receiver` are not weakable, so `Weak<Sender<T>>` and `Weak<Receiver<T>>` are compile errors (a channel closes when its last `Sender` is dropped, never by weak reference).

These are the only types whose copies refer to the same value; everything else in Verse has exactly one owner. They are also the only way to build a cycle (§4.10): a cycle of `Shared`/`Guarded` handles is never freed, so back-pointers should be `Weak`.

Channels send values from one task to another. `Channel.new<T>()` returns a pair `(tx, rx)`: a `Sender<T>` and a `Receiver<T>`. `send` takes its value, so ownership moves to the task that receives it. Senders are copied (one per producing task); the receiver is moved to the single consuming task. The channel closes when its last `Sender` is dropped. The capacity defaults to 16; `capacity: 0` makes an unbuffered (rendezvous) channel, where each `send` waits for a matching `recv`. The type argument is usually inferred from an annotation instead of written: `(tx, rx): (Sender<Int>, Receiver<Int>) = Channel.new(capacity: 10)`.

```
import Channel, wait_all from std.task

fn worker(id, out)
    out.send(id * id)?                  # `Closed` if the receiver is gone

fn start_workers(take tx, count)        # spawns and returns, without waiting
    (1..=count).map(id -> spawn worker(id, tx.copy()))   # a Sender per task

(tx, rx) = Channel.new<Int>(capacity: 4)
tasks = start_workers(tx, 20)           # `tx` moves; 20 values, room for 4
for square in rx                        # consume while the producers run;
    print(square)                       #   ends when every Sender is gone
for outcome in wait_all(tasks)
    outcome?                            # stop if any worker failed
```

Waiting for the producers before consuming can deadlock once the channel is full, so always consume while the producers run, and wait for them afterwards. `send` is fallible: discarding its result is the usual compile-time error for an unhandled fallible value (§8.4). `Closed` is a named error type in `std.task`, like `Timeout`.

A `Receiver` given to another task moves, like any non-copyable value: `spawn consume(rx)`.

Operation	Signature	Meaning
`Channel.new<T>(…)`	`new<T>(capacity: Int = 16): (Sender<T>, Receiver<T>)`	a new channel and its two ends; `capacity: 0` is unbuffered (rendezvous)
`tx.send(v)`	`send(self, take value: T): Nothing | Closed`	blocks while the channel is full; returns the error `Closed` if the `Receiver` was dropped or the channel was closed — including for a sender already blocked on a full channel, which wakes with `Closed`
`tx.close()`	`close(self)`	an early shortcut: closes the channel for everyone, even while other `Sender`s exist; the receiver drains what is left
`rx.recv()`	`recv(self): T?`	blocks until a value arrives; `none` once the channel is closed and drained

`for value in rx` receives until the channel is closed and drained (that is, until `recv()` returns `none`).

Shared read-only data — a large configuration, a dictionary, a parsed model — is wrapped once in `Shared<T>` and handed to as many tasks as needed without copying the data:

```
import Shared from std.task

dictionary = Shared(load_words("words.txt"))

fn check(text, words)
    text.words().filter(w -> not words.get().contains(w)).len()

a = spawn check(chapter_one, dictionary.copy())       # a new handle, same words
b = spawn check(chapter_two, dictionary)              # last use: the handle moves
```

`shared.get()` gives a read-only view of the value inside.

Shared mutable state is wrapped in `Guarded<T>` and can only be touched inside a `with` block that holds its lock:

```
import Guarded, wait_all from std.task

hits = Guarded(0)

fn record(counter)
    with counter.lock() as mut value
        value += 1

tasks = (1..=100).map(_ -> spawn record(hits.copy()))       # one handle per task
for outcome in wait_all(tasks)
    outcome?                        # stop if any task failed
with hits.lock() as value
    print("hits: {value}")
```

Inside `with guarded.lock() as value`, `value` is an exclusive borrow of the protected value; it is **mutable only when written** `as mut value`, as with every other binding (§3.9), so `with g.lock() as value` followed by `value += 1` is the usual immutable-name error. The lock is released when the block ends, even if it ends early. Like every borrow, `value` cannot escape the block.

This is the **one** place where `with … as` binds something other than the resource itself. `lock()` returns a lock guard, and the guard type implements `Cleanup` (its cleanup releases the lock), so `with` cleans it up like any resource. But `with` on a guard binds the *value inside* the guard, as an exclusive borrow (mutable with `as mut value`), rather than the guard. No other type has this behaviour (§13.1).

11.5 Cancellation and time

`task.cancel()` asks a task to stop. Cancellation is cooperative: the task is stopped at its next *blocking point* — I/O, `wait`, `sleep`, `send`, or `recv`.

Cancellation cannot be caught inside the cancelled task. At the blocking point, the task *unwinds* exactly as it would for a panic: it stops running ordinary code, its `with` blocks close their resources in reverse order, and nothing — not `otherwise`, not `match`, not `?` — can intercept it. So `get(url) otherwise "x"` cannot swallow a cancellation and keep going. Unlike a panic, a cancellation prints nothing. The *waiter* sees the outcome as a `Cancelled` error (from `std.task`) returned by `.wait()`, which it may handle normally.

Because cancellation never appears as a value inside the task, blocking operations keep simple signatures:

Operation	Signature
`std.time.sleep(d)`	`sleep(duration: Duration)` — returns `Nothing`; pauses only the current task
`rx.recv()`	`recv(self): T?`
`task.wait()`	`wait(take self): T | Error` (a cancelled *child* shows up here as `Cancelled`)

12. Modules

12.1 Files are modules

Every `.vs` file is a module named after its path relative to the project's `src/` directory (or the directory of the entry file for single-file scripts):

```
my_app/
├── verse.toml
└── src/
    ├── main.vs            # module main
    ├── shapes.vs          # module shapes
    └── geometry/
        └── vectors.vs     # module geometry.vectors
```

12.2 Importing

```
import std.fs                       # use as fs.read_text(...)
import std.json as js               # use as js.parse(...)
import geometry.vectors             # use as vectors.add(...)
import get, post from std.net       # use get(...) and post(...) directly
import Shape, area from shapes
import parse as parse_json, stringify from std.json   # rename one item
```

`import a.b.c` binds the last segment (`c`) as a name; `import a.b.c as d` binds it as `d`. `import x, y from M` binds individual items, and `import x as y from M` binds the item `x` under the name `y` (useful when two modules export the same name); `as` can follow any name in the list, as in `import parse as parse_json, stringify from std.json`. The line always starts with `import`, so every import reads the same way: what you get first, where it comes from last. Whether a line imports a module or items is decided by the trailing `from M` on the same logical line: `import a as b` imports the module `a` as `b`, while `import a as b from m` imports the item `a` of `m` as `b`; without `from`, the names are module paths. An import must therefore fit on one line, or put a long item list in parentheses, which may span lines:

```
import (
    open, read_text, write_text,
    create as create_file,
) from std.fs
```

A renamed item keeps its kind: a type must be renamed to an uppercase-first name, a value to a lowercase one (§3.5). Wildcard imports do not exist; every name in a file can be traced to a declaration or an import. Imports must appear at the top of a file, after the module doc comment. Circular imports between modules are allowed for declarations but not for module-level constant initialization.

An imported name may be hidden by a local, and an import may hide a prelude name (with a warning); the rules are in §3.9.

12.3 Visibility with `export`

Everything is **private to its module by default**. Mark items public with `export`:

```
## Geometry helpers.

export type Vector
    export x: Float
    export y: Float

export type Account
    export owner: Text
    cents: Int = 0                  # hidden: only this module can see it

impl Account
    export fn balance(self): Int
        self.cents

export fn add(a: Vector, b: Vector): Vector
    Vector(x: a.x + b.x, y: a.y + b.y)

export origin = Vector(x: 0.0, y: 0.0)

fn helper(): Int          # not exported: visible only in this module
    42
```

One word, one default, everywhere. `export` can precede `fn`, `type`, `alias`, `trait`, a module-level constant, record **fields**, and functions inside an `impl`. Anything not marked `export` — including a field of an exported record — is visible only inside its own module. Other modules can construct a record directly only if every field without a default is exported; otherwise they use an exported function such as `Account.open(owner:)`. Trait implementations are public whenever both the trait and the type are visible.

Fields and methods share **one namespace per type**: a type cannot have a field and a method with the same name, whether or not either is exported (`balance` the field and `balance()` the method would be a compile error, which is why the field above is named `cents`).

Exported signatures and `verse.api`

Exported functions may leave their types to inference, just like private ones — there is no rule that exported functions must annotate their parameters. This is safe because of three guarantees:

1. **(a) A generated interface file is committed.** `verse api update` writes `verse.api` at the package root, recording every exported item with its full inferred signature (including inferred constraints and parameter modes). The file is committed with the code.
2. **(b) The public API cannot drift silently.** Every build re-infers the exported signatures and compares them with `verse.api`. If one differs, the build fails and shows the old and new signature, until someone runs `verse api update` on purpose (and the change shows up in review as a diff of `verse.api`). Editing a function body can therefore never change the public API by accident.
3. **(c) Inference never crosses a module boundary.** Other modules — and other packages — are type-checked against `verse.api` (and the interface records of the package's own modules), never against another module's body.

```
## Geometry helpers.

export fn scale(v, factor)        # recorded in verse.api as:
    v.copy(x: v.x * factor, y: v.y * factor)
```

```
# verse.api (generated by `verse api update`; do not edit)
module geometry
fn scale<V, F>(v: V, factor: F): V
    where V has copy(x:, y:), V has field x: F, V has field y: F, F: Mul
```

The interface file uses a documented notation for inferred structural constraints (`has field`, `has method`) that cannot be written in source in 0.1 (§4.2). Annotating an exported function narrows what `verse.api` records, which is often what a library author wants.

12.4 Module contents

Only the **entry file** — the file passed to `verse run` — runs top-level statements. Imported modules may contain only declarations (`fn`, `type`, `alias`, `trait`, `impl`, `test`, `import`) and module-level constants (`name = expr`). Importing a module therefore never has side effects.

A top-level binding `name = expr` behaves in the only way that makes sense in each place:

- **In an imported module**, a top-level binding is a *constant*. It is initialized once, on first use, and its initializer **must be infallible**: `?` is not allowed, and its type may not be fallible, because a lazy initializer has no caller to return an error to. (`mut` is not allowed at module level.)

`verse export max_retries = 3 # fine export default_port = 8080 # fine config = read_text("config.toml")? # error: module constants cannot fail`

- **In the entry file**, a top-level binding is an *ordinary statement*: it runs in order, top to bottom, like every other statement there, and may use `?` (see §8.3 for what a failed top-level `?` does). Such a binding is a **script local**: it is visible to the top-level statements after it, but **not inside `fn` bodies** (nor in `impl` methods or `test` blocks). A function that needs the value takes it as an argument. This keeps every function meaningful on its own — it never depends on whether a binding below it has run yet — and matches §14.1, where tests never run top-level statements. Module constants in imported modules are the only module-wide values.

```verse config = load_config("app.toml")?

fn port(config) # takes it as an argument config.port

print(port(config)) ```

Entry-file locals are not visible in functions (M-2), so the parameter `config` is not shadowing.

## 12.5 Program entry

A Verse program is a script: the entry file's top-level statements run from top to bottom.
```

```
print("Hello, {args.first() otherwise "world"}!")
```

- `args` is a built-in `List<Text>` of command-line arguments, *not* including the program name (`verse run hello.vs Ana` gives `args == ["Ana"]`).
- **Top-level statements are the only entry mechanism.** Verse 0.1 has no `fn main`; a function named `main` is just an ordinary function that runs only if the script calls it.
- Exit status: `0` on normal completion; `1` if a top-level `?` fails (the error message is printed to standard error) or if an unwaited task failed (§11.3); `101` on a panic; or any value passed to `exit(code)`. An explicit `exit(code)` always overrides the failed-unwaited-task status, which applies only when the program ends naturally.

---

13. Resource management

13.1 `with` blocks

External resources — files, network connections, locks, temporary directories — must be released promptly and reliably. Because every value has one owner and is freed at the end of its owner's scope (§4.10), a type that implements the `Cleanup` trait is cleaned up **automatically** when its owner's scope ends, however it ends: normally, by `return`, `break`, `continue`, `?` propagation, or a panic.

```
trait Cleanup
    fn cleanup(mut self)       # must not fail
```

That automatic cleanup is a safety net: a file you open in a function is closed when the function ends, even if you forget about it. The `with` statement is the **explicit** way to say *exactly* where a resource's life ends. It acquires the resource, binds it to a name for the indented block, and cleans it up when the block ends:

```
import open from std.fs

with open("log.txt")? as file
    for line in file.lines()
        print(line)
# file is closed here, not at the end of the function
```

`with … as name` binds the resource itself, with one documented exception: a **lock guard**. `with guarded.lock() as value` (§11.4) cleans up the guard (releasing the lock) like any other resource, but binds `value` to the protected value inside the guard, as an exclusive borrow (mutable only with `as mut value`). Lock guards are the only type that behaves this way.

The same `mut` rule applies to every resource: `File`'s write methods (`write_line`, `write`, `flush`) take `mut self`, so a file you write to is bound with `as mut`: `with create("out.txt")? as mut target`.

`cleanup` must not fail: it has nowhere to report an error, because it may run during a panic or while another error is propagating. Types whose closing can fail (a buffered writer that must

flush) also offer an explicit, fallible method — `file.finish()?` — to call when the error matters; `cleanup` is the quiet fallback.

13.2 Acquisition failure

Acquiring a resource can fail: `open` returns `File | Error`. **`with` never propagates anything by itself.** Its resource expression must be a value whose type implements `Cleanup`, so a fallible acquisition needs an explicit `?`, exactly as anywhere else:

```
import open from std.fs

with open("log.txt")? as file          # `?` propagates a failed open
    ...

with open("log.txt") as file           # error: `File | Error` does not implement
Cleanup
    ...                                #   help: add `?` to propagate the error
```

So `with` means one thing — *clean this up when the block ends* — and every early exit is still marked by `?` or `return`. Infallible acquisitions such as `counter.lock()` (§11.4) need no `?`.

To handle the failure locally, acquire first and decide:

```
match open("log.txt")                  # File | Error
    err: Error -> print("no log yet: {err.message()}")
    file: File ->
        with file as log               # `file` moves into the `with`
            print(log.read_all())
```

13.3 Several resources

Several resources can be acquired in one `with`, separated by commas. They are cleaned up in reverse order of acquisition.

```
import open, create from std.fs

with open("input.txt")? as source, create("output.txt")? as mut target
    for line in source.lines()
        target.write_line(line.upper())?
```

13.4 Custom resources

```
import std.fs
import std.random

type TempDir
    path: Text

impl TempDir
    fn create()                              # inferred: TempDir | Error
        path = "/tmp/verse-{random.int(0..1_000_000)}"
        fs.make_dir(path)?
        TempDir(path:)

impl Cleanup for TempDir
    fn cleanup(mut self)
        _ = fs.remove_all(self.path)       # ignore failures during cleanup

with TempDir.create()? as dir
    fs.write_text("{dir.path}/scratch.txt", "draft")?
    ...
```

`_ = …` discards a result deliberately; `cleanup` must not fail, because it may run during a panic.

---

14. Testing and docs

14.1 Test blocks

Tests are written next to the code they test, using `test` blocks:

```
fn slugify(title: Text): Text
    title.lower().words().join("-")

test "slugify joins lowercase words with dashes"
    assert slugify("Hello Big World") == "hello-big-world"

test "slugify of empty text is empty"
    assert slugify("") == ""
```

- A `test` block has a descriptive text name and an indented body.
- `test` blocks are ignored by `verse run` and `verse build`, and compiled only by `verse test`.
- **`verse test` never runs the entry file's top-level statements.** Tests see the module's declarations (functions, types, traits, impls) and its module-level constants only. So testing a script such as §20.1 does not read files, and testing §20.2 does not touch the network or call `exit`. Code that a test needs must therefore live in a function, not in top-level statements.
- A test passes if its body finishes without a failed `assert`, a panic, or a failed `?`. Inside a test, `?` returns from the test (which then fails with the error's message): test bodies behave like functions returning `Nothing | Error`.
- Tests can access private items of their module.
- Tests run in parallel, each in its own task; they must not depend on each other's order.

14.2 `assert`

`assert condition` checks a `Bool`. On failure it reports the source text and, for comparisons, the values of both sides:

```
FAIL  slugify joins lowercase words with dashes   (text.vs:4)
    assert slugify("Hello Big World") == "hello-big-world"
    left:  "hello-big--world"
    right: "hello-big-world"
```

An optional message follows a comma: `assert total > 0, "cart cannot be empty"`. Outside tests, `assert` is also allowed in ordinary code, where a failure panics. Assertions are never removed from release builds.

Patterns combine with `is` for expressive checks:

```
test "parsing rejects letters"
    assert parse_age("abc") is _: Error
    assert parse_age("42")? == 42
```

14.3 Doc comments

`##` comments are Markdown. Where a `##` block belongs is decided by one visible rule:

- A `##` block **directly above** a declaration, with no blank line in between, documents that declaration.
- A `##` block at the top of a file **followed by a blank line** documents the module.

`verse fmt` enforces the blank line after a module doc, so the two never look alike:

```
## Geometry helpers for 2-D drawings.     ← module doc (blank line follows)

import std.math

## The distance between two points.       ← documents `distance`
export fn distance(a: Point, b: Point): Float
    ...
```

Code blocks inside doc comments are **doc tests**: `verse test` runs them too.

```
## Returns the area of a shape.
##
## ```verse
## assert area(Square(side: 2.0)) == 4.0
## ```
export fn area(shape: Shape): Float
    ...
```

`verse doc` turns doc comments into a browsable HTML site, with every type, function, trait and implementation cross-linked.

---

15. Standard library overview

The standard library is small, consistent, and readable. Every module follows the same conventions: functions that can fail return `T | Error` (or a module-specific error type); functions that may find nothing return `T?`; resources implement `Cleanup`.

15.1 The prelude

These names are available in every file without importing. A local or a file-level import may hide one (an import warns); see §3.9.

Name	Description
`print(value: Show = "", end: Text = "\n")`	write `value` to standard output followed by `end`; `print()` with no argument writes an empty line
`input(prompt: ""): Text?`	read a line from standard input; `none` at end of input
`args`	`List<Text>` of command-line arguments
`env(name): Text?`	read an environment variable
`exit(code: Int)`	end the program immediately with a status code
`error(message, cause: none): Error`	build a simple error
Types	`Int`, `Float`, `Bool`, `Char`, `Text`, `Bytes`, `Nothing`, `List`, `Map`, `Set`, `Range`, `Task`
Traits	`Show`, `Eq`, `Order`, `Hash`, `Iterable`, `Index`, `Default`, `Cleanup`, `Copy`, `Copyable`, `Error`, `Add`, `Sub`, `Mul`, `Div`, `Rem`, `Neg`

`Error` is a trait, not a type. Written where a type is expected, as in `T | Error`, a trait name means "any type that implements it" (§8.2, §10.3), which is why `Error` appears in return types throughout this spec.

File access is **not** in the prelude: there is exactly one name for each idea, and it lives in `std.fs` — `open(path): File | Error` (read), `create(path): File | Error` (write), and `read_text(path): Text | Error` (whole file at once). Import them with `import open, read_text from std.fs`.

15.2 Core modules

Module	Purpose	Highlights
`std.fs`	files and directories	`read_text`, `write_text`, `append_text`, `read_bytes`, `exists`, `list_dir`, `walk`, `make_dir`, `remove`, `remove_all`, `open`, `create`, `File`
`std.net`	networking and HTTP	`get`, `post`, `put`, `delete`, `Request`, `Response(status, headers, body)`, `serve`, `TcpStream`, `Url`
`std.text`	text processing	`words`, `lines`, `pad`, `wrap`, `similarity`, `Regex`, `Builder`, Unicode normalization
`std.time`	dates, times and durations	`now()`, `today()`, `sleep(d)`, `Duration` (`5.seconds()`, `200.millis()`, `2.hours()`), `Instant`, `Date`, `DateTime`, time zones, `format`
`std.json`	JSON	`parse(text): Json | Error`, `stringify(value, pretty: false)`, the `Json` variant type, `ToJson` / `FromJson` traits with automatic implementations for records and variants
`std.task`	concurrency	`Channel` (`Channel.new<T>()` returns a `Sender<T>` and a `Receiver<T>`), `Shared`, `Guarded`, `Weak`, `wait_all`, `wait_any`, and the named error types `Timeout`, `Cancelled`, `TaskPanicked` and `Closed` (returned by `Sender.send` once the channel is closed); `.copy()` on `Sender`, `Shared`, `Guarded` and `Weak` copies the handle, explicitly, also when passing it to `spawn`; `Receiver` is not copyable and moves (§11.4)
`std.math`	mathematics	`pi`, `e`, `sqrt`, `sin`, `cos`, `log`, `clamp`
`std.num`	numeric types	sized integers, `BigInt`, `Decimal`, `OrderedFloat`, parsing, and `ParseError`, the error returned by `Text.to_int()` (`Int | ParseError`) and `Text.to_float()` (`Float | ParseError`)
`std.random`	randomness	`int(range)`, `float()`, `choice(list)`, `shuffle(list)`, seeded generators
`std.process`	child processes	`run(cmd, args)`, `Output(status, stdout, stderr)`
`std.env`	environment	`vars()`, `current_dir()`, `home_dir()`, `platform`
`std.cli`	argument parsing	declarative flags from a record type
`std.log`	structured logging	`info`, `warn`, `error`, levels, sinks

A few examples:

```
import std.fs
import std.json
import std.time
import get from std.net

started = time.now()

response = get("https://api.example.com/items")?
items = json.parse(response.body)?

fs.write_text("items.json", json.stringify(items, pretty: true))?

names = fs.list_dir(".")?
for name in names.filter(n -> n.ends_with(".vs"))
    print(name)

print("done in {time.now() - started}")
```

`json` values can be converted to and from records directly:

```
type Item
    id: Int
    title: Text

raw = r"""
    {"id": 1, "title": "Pen"}
    """                                  # raw: `{` is not interpolation
item = Item.from_json(json.parse(raw)?)?
print(json.stringify(item))
```

---

16. Tooling

Verse ships as a single command, `verse`, that contains the compiler, runner, formatter, test runner, documentation generator, language server and package manager.

Command	Purpose
`verse run file.vs [args…]`	compile (cached) and run a script or project
`verse test [path]`	run `test` blocks and doc tests; `--filter "name"`, `--watch`
`verse fmt [path]`	format code in the one canonical style; `--check` for CI
`verse check`	type-check without running; fast feedback for editors
`verse build`	produce an optimized native executable
`verse doc`	generate HTML documentation from doc comments
`verse repl`	interactive prompt
`verse new name`	create a project skeleton
`verse add pkg` / `verse remove pkg`	manage dependencies
`verse update`	update dependencies within allowed versions
`verse publish`	publish a package to the registry
`verse lsp`	language server for editors, including inline hints for inferred types
`verse api update`	regenerate `verse.api`, the committed interface file of exported signatures (§16.4)
`verse api check`	compare inferred exported signatures with `verse.api` (run by every build)

16.1 The formatter

`verse fmt` has **no configuration options**. It normalizes indentation to four spaces, spacing around operators, blank lines between declarations and trailing commas in multi-line lists; wraps long method chains one call per line; sorts imports; enforces the blank line after a module doc (§14.3); and drops the redundant `)` that the lambda closing rule makes unnecessary (§5.4). It never removes semantically significant parentheses — those that end a lifted chain, and those in `(a?).b`, where `a?.b` would be an error (§8.1). It **never inserts or removes type annotations**: whether to spell out a type is the author's choice, and inferred types are shown by the editor as inline hints instead. A file that passes `verse fmt --check` looks the same as every other Verse file in the world.

16.2 Compiler messages

Error messages are written in plain language, point at the exact code, and suggest a fix where possible:

```
error: `email` might be missing
  --> users.vs:12:11
   |
12 |     print(user.email.lower())
   |           ^^^^^^^^^^ this is `Text?`, not `Text`
   |
help: provide a fallback:   user.email.lower() otherwise ""
help: or stop early:        print(user.email.lower()?)
help: or branch:            if user.email is email?  …
```

16.3 Packages

A project is described by `verse.toml`:

```
[package]
name = "word-tools"
version = "0.3.0"
description = "Small text utilities"
license = "MIT"
verse = "0.1"

[dependencies]
colors = "1.2"              # semantic versioning: >= 1.2.0, < 2.0.0
http_extra = { git = "https://example.com/http_extra.git", tag = "v0.4.1" }

[dev-dependencies]
snapshot = "0.9"
```

- Packages are fetched from the central registry (or git/path sources) and pinned in `verse.lock`, which is committed for applications.
- A package's modules are imported by package name: `import red, bold from colors`.
- Resolution chooses a single version of each package per major version.
- Builds are reproducible: the same `verse.lock` gives the same build.
- Packages declare no install scripts; building a dependency cannot run arbitrary code.
- A published package includes its `verse.api`; dependents are checked against it, never against the package's source.

16.4 Interface files (`verse.api`)

`verse.api` is what makes inferred exported signatures safe (§12.3):

Step	What happens
`verse api update`	infers every exported signature and writes `verse.api`; commit it
`verse build`, `verse check`, `verse test`	re-infer exported signatures; **fail** if any differs from `verse.api`, printing the recorded and the new signature
other modules and packages	are checked against `verse.api` only — inference never crosses a module boundary

So the only way to change a public signature is to change the code *and* run `verse api update`, which turns the change into a visible diff in review. `verse fmt` never touches `verse.api` and never inserts annotations into source.

---

17. EBNF grammar sketch

This grammar is a readable sketch of the core language, not a complete formal definition. The lexer turns leading whitespace into `INDENT` and `DEDENT` tokens and ends logical lines with `NEWLINE`, suppressing it inside brackets, after a trailing binary operator, and before a line that begins with `.` or `|` (§3.4). `BLANK` is an empty line, significant only after a module doc. Text in

`"quotes"` is a literal token; `{ x }` means zero or more, `[ x ]` means optional, `|` separates alternatives. Rules marked *(semantic)* are checked after parsing.

```
(* ---------- Files and declarations ---------- *)
file          = { NEWLINE } [ doc BLANK ] { import } { top_item } ;
                                  (* doc + blank line = module doc (§14.3) *)
top_item      = declaration | statement | test_block ;
declaration   = [ doc ] [ "export" ]
                ( fn_decl | type_decl | alias_decl | trait_decl | binding )
              | [ doc ] impl_block ;
doc           = DOC_COMMENT { DOC_COMMENT } ;     (* directly above its item *)

import        = "import" module_path [ "as" IDENT ] NEWLINE
              | "import" ( import_list | "(" import_list [ "," ] ")" )
                "from" module_path NEWLINE ;
                (* the line always starts with `import`; a trailing `from` on
                   the same logical line makes the names items of that module,
                   so `import a as b` is a module and `import a as b from m` an
                   item. An import is one line; only the parenthesised list may
                   span lines (§3.4). *)
import_list   = import_item { "," import_item } ;
import_item   = name [ "as" name ] ;              (* same kind on both sides *)
name          = IDENT | TYPE_IDENT ;
module_path   = IDENT { "." IDENT } ;

(* ---------- Functions ---------- *)
fn_decl       = "fn" IDENT [ generics ] "(" [ params ] ")" [ ":" type ] block ;
fn_sig        = "fn" IDENT [ generics ] "(" [ params ] ")" [ ":" type ] NEWLINE ;
params        = param { "," param } [ "," ] ;
param         = [ "mut" | "take" ] "self"
              | [ "mut" | "take" ] IDENT [ ":" type ] [ "=" expr ] ;
                (* types optional (§4.2); `mut`/`take` always explicit *)
generics      = "<" generic_param { "," generic_param } ">" ;
generic_param = TYPE_IDENT [ ":" bound { "+" bound } ] ;
bound         = TYPE_IDENT [ type_args ] ;

(* ---------- Types ---------- *)
type_decl     = "type" TYPE_IDENT [ generics ]
                ( NEWLINE INDENT field { field } DEDENT    (* record: nothing after the
name *)
                | "=" variant { "|" variant } NEWLINE ) ;  (* new variants; never a
field block *)
                (* (semantic) a variant name may not be a type already in scope:
                   `type Meters = Float` is rejected with "use alias" *)
field         = [ doc ] [ "export" ] IDENT ( ":" type [ "=" expr ] | "=" expr ) NEWLINE
;
                (* the annotation may be omitted only when a default implies the type
*)
variant       = TYPE_IDENT [ "(" field_list ")" ] ;
field_list    = IDENT ":" type { "," IDENT ":" type } [ "," ] ;
alias_decl    = "alias" TYPE_IDENT [ generics ] "=" type NEWLINE ;  (* incl. unions *)

type          = union_type ;
union_type    = optional_type { "|" optional_type } ;   (* normalized, §4.6 *)
optional_type = base_type { "?" } ;                       (* T?? is not T? *)
base_type     = { IDENT "." } TYPE_IDENT [ type_args ]      (* `time.Duration` *)
              | "(" type "," type { "," type } ")"              (* tuple   *)
              | "(" labeled { "," labeled } ")"                  (* labeled *)
              | "fn" "(" [ type { "," type } ] ")" ":" type
              | "(" type ")" ;
labeled       = IDENT ":" type ;
type_args     = "<" type { "," type } ">" ;

(* ---------- Traits and impls ---------- *)
trait_decl    = "trait" TYPE_IDENT [ generics ] [ ":" bound { "+" bound } ]
                NEWLINE INDENT { [ doc ] ( fn_sig | fn_decl ) } DEDENT ;
impl_block    = "impl" [ generics ] [ bound "for" ] type
                NEWLINE INDENT { [ doc ] [ "export" ] fn_decl } DEDENT ;

(* ---------- Statements ---------- *)
block         = NEWLINE INDENT statement { statement } DEDENT ;
```

```
statement     = binding | assign_stmt | if_stmt | loop_stmt
              | with_stmt | match_expr | exit_stmt
              | "pass" NEWLINE | assert_stmt | fn_decl | expr NEWLINE ;
exit_stmt     = exit [ "if" expr ] NEWLINE ;          (* postfix guard: one line, no
else;
                                                         an `if` right after an exit
is
                                                         always a guard: write
                                                         `return (if c then a else
b)` *)
exit          = "return" [ expr ] | "break" [ IDENT ] | "continue" [ IDENT ] ;
                (* (semantic) an `expr` statement may not have a fallible type
                   or contain fallible results (§8.4); use `?`, handle it, or `_ =` *)
binding       = [ "mut" ] pattern [ ":" type ] "=" expr NEWLINE
              | [ "mut" ] pattern [ ":" type ] "=" or_expr "otherwise" exit NEWLINE ;
                (* (semantic) a refutable pattern is allowed only in the second form.
                   `x = e` with a plain name is a *binding* when `x` is not yet
                   visible, and an *assignment* (assign_stmt) when `x` is a visible
                   `mut` binding; on a visible immutable name it is an error.
                   A name is bound once per function: no shadowing (§3.9);
                   "visible" means locals only. A pattern always binds new
                   names; the exception is a tuple made only of visible `mut`
                   names, a parallel assignment (right side evaluated first,
                   so `(a, b) = (b, a)` swaps); mixing new and existing names
                   is an error.
                   Constructor patterns start with an uppercase name (§3.5), so
                   `Circle(r) = …` is a pattern and `f(r) = …` is not valid.
                   `let` is reserved: "`let` is not used in Verse; write `x = …`" *)
assign_stmt   = place assign_op expr NEWLINE ;
assign_op     = "=" | "+=" | "-=" | "*=" | "/=" | "%=" ;
place         = ( IDENT | "self" ) { "." IDENT | "[" expr "]" } ;

if_stmt       = "if" expr block
                { "else" "if" expr block } [ "else" block ] ;
loop_stmt     = [ IDENT ":" ] ( for_stmt | while_stmt ) ;          (* loop label *)
for_stmt      = "for" pattern "in" expr block ;
while_stmt    = "while" expr block ;
with_stmt     = "with" resource { "," resource } block ;
resource      = expr "as" [ "mut" ] IDENT ;  (* (semantic) expr's type must implement
                                                Cleanup; a fallible acquisition needs `?`;
                                                the name is mutable only with `mut` *)
assert_stmt   = "assert" expr [ "," expr ] NEWLINE ;
test_block    = "test" TEXT block ;

(* ---------- Pattern matching ---------- *)
match_expr    = "match" expr NEWLINE INDENT arm { arm } DEDENT
              | "match" NEWLINE INDENT cond_arm { cond_arm } "_" "->" arm_body DEDENT ;
                                                    (* subject-less: final `_`
required *)
cond_arm      = arm_expr "->" arm_body ;                (* a Bool; `expr - lambda`, so
a
                                                         lone name is not a lambda *)
arm_body      = ( arm_expr | exit | "pass" | place assign_op expr ) NEWLINE | block ;
                                                    (* `_ -> pass`, `none -> return
0`,
                                                         `x -> total += 1` *)
arm           = top_pat [ "if" arm_expr ] "->" arm_body ;
arm_expr      = expr - lambda ;          (* a lambda body must be parenthesized *)
top_pat       = ( "_" | IDENT ) ":" type                    (* typed: top level only
*)
              | pattern ;
pattern       = present_pat { "|" present_pat } ;
present_pat   = single_pat { "?" } ;                        (* `user?`, `e?`, `none?
` *)
single_pat    = literal | "none" | "_" | IDENT
              | TYPE_IDENT [ "(" [ sub_pats ] ")" ]         (* variant / record *)
              | "(" pattern "," pattern { "," pattern } ")"  (* tuple   *)
              | "[" [ list_pats ] "]"                       (* list    *)
```

```
              | literal ( ".." | "..=" ) literal ;             (* range   *)
sub_pats      = sub_pat { "," sub_pat } ;
sub_pat       = IDENT ":" [ field_pat ]       (* always a field; `name:` alone puns *)
              | pattern ;                     (* positional *)
field_pat     = IDENT ":" type | pattern ;    (* `body: b: Text` = typed binding *)
list_pats     = pattern { "," pattern } [ "," ".." IDENT ] | ".." IDENT ;

(* ---------- Expressions ---------- *)
expr          = lambda | cond_expr | otherwise_expr ;
lambda        = lambda_params "->" ( expr | block ) ;
                (* closing rule: a block lambda opened inside "(" on the same line
                   must be the last argument; its DEDENT also closes that "(" *)
lambda_params = IDENT | "(" [ IDENT [ ":" type ] { "," IDENT [ ":" type ] } ] ")" ;
cond_expr     = "if" expr "then" expr "else" expr ;      (* one-line conditional *)
otherwise_expr= or_expr { "otherwise" fallback } [ "otherwise" exit ] ;
                                                        (* fallback, loosest; an exit
                                                           only as the whole last
operand *)
fallback      = postfix | "-" ( INT | FLOAT ) ;         (* literal, call, name, field,
                                                           index, lifted chain, ( … );
                                                           anything else: add brackets
*)
or_expr       = and_expr { "or" and_expr } ;            (* Bool only *)
and_expr      = not_expr { "and" not_expr } ;
not_expr      = "not" not_expr | comparison ;
comparison    = range_expr { comp_op range_expr }
              | range_expr "is" top_pat ;
comp_op       = "==" | "!=" | "<" | "<=" | ">" | ">=" | "in" | "not" "in" ;
range_expr    = additive [ ( ".." | "..=" ) additive ] ;  (* both ends required *)
additive      = term { ( "+" | "-" ) term } ;
term          = unary { ( "*" | "/" | "%" ) unary } ;
unary         = "-" unary
              | "spawn" postfix          (* (semantic) the postfix must end in a call
*)
              | power ;
power         = postfix [ "**" unary ] ; (* right-assoc; tighter than unary minus *)
postfix       = TYPE_IDENT "." IDENT type_args call { postfix_op }
              | primary { postfix_op } ;
postfix_op    = call | index | "." ( IDENT | INT | TYPE_IDENT ) | "?" ;
                (* (semantic) lifted chains: a "." on a `T?` looks through the
                   optional only when the chain is consumed by `otherwise`, a
                   postfix "?", an `is` / `match` subject, or a position whose
                   expected type is optional (§8.1); it short-circuits on the
                   first `none` and yields one flattened `?` level. Elsewhere a
                   "." on a `T?` is an error. There is no `?.` token. *)
                (* `. TYPE_IDENT`: qualified names such as `Shape.Circle(2.0)`
                   and `time.Duration` *)
                (* `TYPE_IDENT . IDENT type_args call`: explicit type arguments
                   only on an associated-function call whose receiver is a
                   capitalised type name, as in `Channel.new<Int>(capacity: 10)`;
                   a lowercase receiver never takes them, so `a.b < c > (d)` is
                   always a chained comparison. Free generic functions take no
                   explicit type arguments in 0.1 (§10.5). They are normally inferred
                   from an annotation instead:
                   `(tx, rx): (Sender<Int>, Receiver<Int>) = Channel.new(capacity: 10)`
*)
                (* (semantic) postfix "?" is an ordinary postfix operator applied
                   left to right, but a "?" immediately followed by "." is an
                   error ("`?.` is not Verse"): write `(f(x)?).email` to
                   propagate then read *)
                (* (semantic) no two postfix "?" in a row: write (x?)? *)
                (* x.f(y) may call a free function f(x, y) (uniform call syntax);
                   a method always wins; ambiguity is an error *)
call          = "(" [ args ] ")" ;
args          = arg { "," arg } [ "," ] ;
arg           = IDENT ":" [ expr ]        (* named; `name:` alone puns `name: name` *)
              | [ "mut" ] expr ;          (* `mut` marks an exclusive borrow *)
index         = "[" ( expr | slice ) "]" ;
```

```
slice         = additive ( ".." | "..=" ) [ additive ]    (* open end: only in slices
*)
              | ( ".." | "..=" ) additive ;

primary       = literal | IDENT | "self" | TYPE_IDENT [ type_args ]
              | "(" expr ")" | tuple | list | map | set
              | if_expr | match_expr | key_path ;
key_path      = "." ( IDENT | INT ) { "." ( IDENT | INT ) | call } ;
                (* only where an expression starts: after "(", "," or an operator;
                   plain access chains only *)
if_expr       = "if" expr block "else" ( block | if_expr ) ;   (* block form *)
tuple         = "(" arg "," [ arg { "," arg } ] [ "," ] ")" ;
list          = "[" [ elem { "," elem } [ "," ] ] "]" ;
elem          = ".." expr | expr ;                             (* prefix `..` spreads
*)
map           = "{" ":" "}"                                    (* empty map; `{}` is an
error *)
              | "{" entry { "," entry } [ "," ] "}" ;
entry         = ".." expr | expr ":" expr ;                    (* later keys win *)
set           = "set" "{" [ elem { "," elem } [ "," ] ] "}" ;  (* `set` is reserved *)
literal       = INT | FLOAT | TEXT | RAW_TEXT | CHAR | BYTES
              | "true" | "false" | "none" ;
```

Lexical tokens, informally:

```
(* The first character decides the kind of name (§3.5).                    *)
TYPE_IDENT  = upper { letter | digit | "_" } ;        (* upper = Unicode Lu or Lt  *)
IDENT       = ( other_letter | "_" ) { letter | digit | "_" } ;
                        (* other_letter = any letter that is not Lu/Lt,
                           including lowercase and uncased letters (e.g. 数) *)
INT         = digits | "0x" hex { hex | "_" }
            | "0o" oct { oct | "_" } | "0b" bin { bin | "_" } ;
FLOAT       = digits "." digits [ exponent ]
            | digits exponent ;                          (* 3.14, 6.02e23, 1e-9 *)
              (* right after ".", a digit sequence is always one INT, never a
                 FLOAT: `t.0.1` is INT 0 then INT 1. `10.seconds()` is INT 10,
                 ".", IDENT; `10.` with no digit after the dot is an error *)
exponent    = ( "e" | "E" ) [ "+" | "-" ] digits ;
digits      = digit { digit | "_" } ;
TEXT        = '"' { char | escape | interp } '"'
            | '"""' NEWLINE { line } '"""' ;
interp      = "{" expr [ ":" format ] "}" ;
                        (* (semantic) a TEXT inside `expr` may not itself
                           contain `interp`: nesting is one level deep *)
              (* the format ":" is the first colon not inside ( ), [ ], { }
                 or a nested text literal; an expression with its own
                 top-level colon, such as `v is n: Int`, needs ( ) *)
RAW_TEXT    = 'r"' { char } '"' | 'r"""' NEWLINE { line } '"""' ;
CHAR        = "'" ( char | escape ) "'" ;
BYTES       = 'b"' { byte_char | escape } '"' ;
COMMENT     = "#" { any } NEWLINE ;
DOC_COMMENT = "##" { any } NEWLINE ;
```

---

18. Design notes

Most of Verse's surface syntax — function and type declarations, `match`, error types, optionals, modules, tests, and so on — was settled in the design discussion and then refined in review passes (§21). This section records the remaining decisions the authors made themselves, and why. Each follows the same tests: *Does it read like prose? Is there exactly one reading? Is the safe behavior the default?*

18.1 Ranges

Decision: `a..b` excludes the end, `a..=b` includes it. Ranges are ordinary values, work in `for`, `in`, slicing and patterns, and support `.step(n)` and `.reverse()`. Open-ended forms (`a..`, `..b`, `..=b`) exist **only inside slice brackets** (`items[2..]`, `items[..3]`), where the missing end is obvious.

Why: The half-open range is the right default for indexing (`0..items.len()` never goes out of bounds) and splits cleanly (`0..mid` and `mid..end` do not overlap). `..=` makes the inclusive case visible instead of relying on a `- 1`. The notation is familiar from Rust, Swift and Kotlin.

18.2 Text literals

Decision: Double quotes only; `{expr}` interpolation with optional `:format`; triple-quoted multi-line text whose common indentation (set by the closing `"""`) is stripped; raw text with an `r` prefix; escapes `\n \t \r \\ \" \{ \} \0 \u{…}`. Single quotes are reserved for `Char`.

Why: One quoting style means no style debates. Indentation stripping lets multi-line text sit naturally inside indented code. Raw text keeps regular expressions and Windows paths readable. `\{` is needed because `{` always starts interpolation in normal text.

18.3 Numeric and scalar types

Decision: Core types `Int` (64-bit, overflow-checked), `Float` (64-bit), `Bool`, `Char`, `Text`, `Bytes`, and `Nothing`. Sized and arbitrary-precision types live in `std.num`. No implicit conversions; `Int / Int` truncates toward zero; division by zero and overflow panic.

Why: Most programs need one integer type and one floating type; fewer choices means simpler code. Checked overflow is the safe default; the rare code that wants wrapping says so by name. Implicit conversion is a classic source of silent precision loss, so conversions are written out.

18.4 Booleans, fallback, and the word `otherwise`

Decision: Boolean operators are the words `and`, `or`, `not`, used only with `Bool` operands (no `&&`, `||`, `!`, and no truthiness). The fallback operator is a separate word, `otherwise`, which applies only to optional and fallible values. Error and union types are written with `|`.

Why: An early draft let `or` mean both "boolean or" and "fallback", resolved by the type of the left operand. It was unambiguous to the compiler but not always to a reader: in `a or b` you had to know the type of `a` to know which operation you were looking at. Giving each idea its own word removes the guesswork: `or` is always logic, `otherwise` is always a fallback, and `find_user(id).email otherwise "no email"` reads as a sentence. Using `|` for fallible types (`Int | Error`) frees `or` from type position too and matches the existing `|` in variant declarations — both say "one of these". `otherwise` binds loosest of all binary operators because a fallback normally applies to the whole expression to its left. Its right operand is restricted to a literal, call, postfix expression, parenthesized expression or another `otherwise`, so `x otherwise 0 + 1` — which would silently mean `x otherwise 1` — is a compile-time error that asks for brackets.

18.5 Other operators

Decision: Comparison operators chain (`0 <= i < n`). `is` tests a pattern; `in` / `not in` test membership. Assignment is a statement, never an expression. No `?:` ternary operator (use the one-line `if a then b else c`, which reads as English), no `++`/`--` (use `+= 1`), and no operator overloading except through the built-in traits (`Eq`, `Order`, and the arithmetic traits `Add`, `Sub`, `Mul`, `Div`, `Rem`, `Neg` listed in §10.4).

Why: Each removal eliminates a known source of bugs or puzzles (`if (x = 3)`, `i++ + ++i`). Chained comparisons match how people write mathematics.

18.6 Equality

Decision: `==` is structural equality for all types, automatically derived. There is no identity operator. A `T` may be compared with a `T?` (the `T` is treated as present). Comparing unrelated types is a compile-time error.

Why: With single ownership and no surface references, identity is not observable (handles such as `Shared<T>` compare by the value inside), so one equality is enough. Rejecting `1 == "1"` at compile time catches real mistakes.

18.7 Tuples and destructuring

Decision: Tuples are written `(a, b)`, accessed with `.0`, `.1`, and may carry labels: `(low: Int, high: Int)`. Bindings, `for` and `match` destructure with patterns. Only irrefutable patterns are allowed in bindings and `for` (a binding may use a refutable pattern only with `otherwise return`). A lambda with *N* parameters called with an *N*-tuple spreads it across the parameters (`pairs.map((a, b) -> a + b)`).

Why: Tuples are perfect for returning two things without declaring a type; labels keep that readable as code grows. Refusing refutable binding patterns avoids a hidden panic. Auto-spread makes `zip`, `enumerate` and `entries` pleasant without a special lambda-pattern syntax; it applies only when the arity matches exactly, so it never guesses.

18.8 Loops

Decision: `while cond` with `break` and `continue`; optional loop labels written **before** the loop, `outer: for row in grid`, and used as `break outer` / `continue outer`. No `do-while`, no C-style `for`, no separate infinite-loop keyword (`while true`).

Why: `for … in` covers almost all loops; `while` covers the rest. An earlier draft wrote labels as `for row in grid as outer`, but `as` elsewhere (`with x as y`, `import m as n`) names the thing on its *left*, whereas a loop label names the loop. A leading `outer:` uses the colon in its "is named" sense (§1.2), reads like a heading, and is what readers of other languages expect.

18.9 Immutability, ownership, no shadowing

Decision: Immutability is deep and belongs to the binding. There is no `let`: `x = expr` binds a new immutable name, `mut x = expr` a mutable one, and `x = expr` on an existing `mut` name

reassigns it; on an existing immutable name it is an error. `mut` means "can change" everywhere — bindings, parameters, `self`. Every value has exactly one owner; assignment and passing either copy (copy types), borrow (function parameters, local views), or move (new owners), and moves are checked per control-flow path. `.copy()` — with no arguments, or with field changes — is the single way to duplicate a value. **Shadowing is removed:** a name is bound once per function; parameters, loop variables and pattern bindings may not reuse a visible name, though sibling blocks may reuse one. Only locals count: a local may hide a prelude, imported or module-level name. Patterns always bind new names, except that a tuple of visible `mut` names is a parallel assignment, so `(a, b) = (b, a)` swaps. An unused binding warns, and so does a new binding one edit away from a visible `mut` name. `let` stays reserved, with a friendly error.

Why: With deep immutability and single ownership, a reader can understand a function by reading only that function: nothing it borrows can change behind its back, and nothing it owns is visible elsewhere. One duplication method keeps the vocabulary small. Dropping `let` removes the most frequent keyword in ordinary code, and the one word left, `mut`, marks exactly the names a reader has to watch. The price is that without a declaration keyword, `x = …` on a visible name cannot say whether it means "a new `x`" or "change `x`" — so shadowing had to go: with it, shadowing and reassignment would look identical. Without it, the rule is simple (an existing name can only be reassigned, and only if it is `mut`), each name means one thing for the whole function, and the typo warning catches the main risk of binding by writing, a misspelt assignment that silently creates a new name. Earlier drafts allowed an inner block to shadow an outer name; that rule is superseded.

18.10 Why no garbage collector

Decision: No garbage collector and no manual memory management. Memory is managed by single ownership with **second-class borrows**: parameters borrow by default, `mut` borrows exclusively, `take` consumes; borrows cannot be stored, returned, or sent to another task; there is no reference or pointer type and no lifetime syntax. Values are freed deterministically at the end of their owner's scope; recursive types are boxed by the compiler; `Cleanup` runs at end of scope. Sharing is opt-in through `Shared<T>`, `Guarded<T>`, `Weak<Shared<T>>` / `Weak<Guarded<T>>`, the channel ends `Sender<T>` (copyable) and `Receiver<T>` (not copyable, single consumer), and `Task<T>`.

Why: Deterministic, pause-free memory and resource management, with the same safety guarantees as Rust — no use-after-free, no data races — but without lifetime annotations or elision rules, because a borrow can never outlive the call that made it. Most everyday code (transforming data, calling functions, building results) never mentions ownership at all.

The honest trade-offs:

- **Some programs need `.copy()`.** Because a borrow cannot be returned or stored, code that would hand out a view into a structure must copy instead (`p.name.copy()`), restructure to work in place with `mut`, or return an index. Copies of large values cost time and memory, and the compiler only removes copies it can prove unnecessary (a source at its last use is moved).

- **Collection chains pass views; the end of the chain decides.** Adapters such as `filter`, `sort_by` or `zip` hand along views of the elements and copy nothing; the chain ends in an owning step — `map` (the lambda must return an owned value), a reduction, a move from a source at its last use, or an explicit `.copy()`. `find`, `first`, `last`, `min` and `max` return views, and a name bound to a chain is a local view. A non-`Copy` element is never duplicated implicitly: where that would be needed, the compiler asks for `.copy()` (§9.5). The cost is that keeping a filtered list while the source is still in use needs a visible `.copy()`.
- **Ownership can flip with later code.** `b = a`, `for x in items`, pattern bindings and chain results are views when the source is used later and owners when it is the last use, so adding a line below can turn working code into an error. Every such error must name the later use that kept the source alive ("`people` is used again on line 12, so `person` is only a view; write `person.copy()` or move the later use"), §4.9.
- **Generic code is checked once.** Unannotated and generic parameter types are treated as opaque, non-`Copy` types during the ownership check, so a generic body sometimes needs a `Copy` or `Copyable` bound (or `.copy()`) where a concrete type would not (§4.2, §10.6). Unannotated getters are the exception that keeps types visually optional: returning a field read through a borrowed parameter infers a `Copy` bound, recorded in `verse.api`, and an unmet bound is a call-site error that names it.
- **Some designs need `Shared<T>` or arenas.** Graphs, caches and observer lists that would rely on many references in a GC language use `Shared`/`Weak` handles or index-based arenas. `Shared` handles are reference-counted, and a cycle of them leaks unless one link is `Weak`; the spec says so plainly (§4.10).
- **Tasks cannot borrow from their spawner** in 0.1; they receive owned values, copies, or handles.
- **Error messages carry the burden.** Every ownership error must point at the move and the later use, explain the rule in one sentence, and offer the fix (usually `.copy()`, `mut`, or `take`). This is a requirement on the compiler, not a nicety.

18.11 Missing values and errors in detail

Decisions: - **Lookups.** Map indexing `map[key]` and `list.get(i)` return a real `V?`, while list indexing `list[i]` returns `T` and panics when out of range. Two-argument `get(key, default: x)` returns a plain `V` on both maps and lists; `default:` exists only on `get`. A bare `{}` is an error pointing to `{:}` and `set{}`. *Why:* a missing map key is ordinary data, but a bad list index is almost always a bug — and `list[i]` in a loop over `0..list.len()` should not need unwrapping. `get(k, default: 0) + 1` replaces the clumsy `(m[k] otherwise 0) + 1` for the most common update. - **No silent flattening.** `T??` is a distinct type. Only lifted chains (plain dots through optionals, §8.1) flatten, because a chain only asks "was anything missing?". *Why:* on a `Map<Text, Int?>`, "key missing" and "present but `none`" must stay distinguishable. - **Layers in written order.** `?` and `otherwise` act on the outermost layer: for `Text? | Error` the error side first, then a second `otherwise` (or `?`) handles `none`. - **Union normalization.** A specific error type in a union with `Error` is absorbed by `Error`, and `?` never narrows `Error` back; `match` with a typed pattern recovers the specific type. - **Patterns.** `match` on a union uses typed patterns (`n: Int`, `err: Error`) at the top level only; inside parentheses `name: pattern` is always a field. `p?` matches a present optional, replacing the earlier `if let` form (removed, as is `let` itself, §18.9). A catch-all lowercase arm that is not last makes later arms

an *unreachable arm* error. - **No `?.` operator; the consumer decides.** A plain dot looks through an optional only when the chain is immediately consumed by `otherwise`, a postfix `?`, an `is`/`match`, or a position whose type is optional; anywhere else it is an error. *Why:* `?.` added a symbol to every hop (`order?.customer?.address?.city`) while the safety came from the place that finally handles `none` — so that place now carries the decision, and the chain reads as plain prose. The safety is the same: a missing value can still never be used as if it were present, and the compiler points at the hop that may be missing. The cost is that the meaning of a dot depends on how its result is used, which the error messages explain. A `?` immediately followed by a dot is an error, since readers from other languages would take `a?.b` for safe access; "propagate, then access" is written `(a?).b`. - `?` returns from the innermost function-like body: a lambda, a spawned function, a test, or the script. `?` on `T | E` in a function returning `U?` is an error, as is `?` on an optional in a function returning an error union (`.or_error("why")` converts). *Why:* errors should carry a message, and an error should never silently become `none`. - **Discarding a fallible result is a compile-time error**, including values that contain fallible results (such as the list from `wait_all`). `_ = …` is the one visible escape hatch. *Why:* §1.3 promises that failure is handled or explicitly propagated; a warning would break that promise.

18.12 `with` only cleans up

Decision: `with` never propagates errors by itself. Its resource must be a value whose type implements `Cleanup`, so a fallible acquisition is written with an explicit `?`: `with open("log.txt")? as file`.

Why: An earlier draft let `with` perform the `?` check implicitly. Review rejected it: §1 and §8 promise that every early exit is marked with `?` or `return`; `with` is also used for infallible things (`counter.lock()`), so the word itself does not signal failure; and an implicit check would make `with` mean "cleanup" or "cleanup plus propagation" depending on the type — the same two-meanings-by-type problem that was removed from `or`. The cost is one character.

18.13 Concurrency details

Decision: Structured task lifetimes (unwaited tasks are awaited when their spawning function ends), owned arguments moved into each task (borrows cannot cross; handles copy cheaply), communication and sharing only through channels (`Sender`/`Receiver`), `Shared`, `Guarded` and `Weak`, cooperative cancellation, and every task outcome — success, error, or panic — delivered by `.wait()` as `T | Error`. A panic is always printed when it happens and reaches the waiter as the named type `TaskPanicked`. Cancellation unwinds the cancelled task like a panic (running `Cleanup`) and cannot be caught inside it; the waiter sees `Cancelled`. An unwaited task that fails makes the process exit non-zero on natural program end (an explicit `exit(code)` always wins). `rx.recv()` returns `T?`; a channel closes when its last `Sender` is dropped (or early with `tx.close()`), and `tx.send(v)` returns `Closed` once the receiver is gone.

Why: These rules make "no async/await" safe: tasks cannot leak, race, or lose results silently, and neither `otherwise` nor `match` can hide a crash or swallow a cancellation.

18.14 Modules without side effects

Decision: Only the entry file runs top-level statements, and top-level statements are the *only* entry mechanism (no `fn main` in 0.1). Imported modules contain declarations and lazily initialized constants whose initializers must be infallible; in the entry file, a top-level binding is an ordinary statement. `verse test` never runs entry-file statements. Everything — including record fields — is hidden unless marked `export`. No wildcard imports; `import x as y from m` renames, and every import line starts with `import` (`import get, post from std.net`). Implementations (`impl Trait for Type`) live with the trait or the type.

Why: Importing a module or running its tests should never print, connect, or write files. One entry mechanism and one visibility default (`export`, for items and fields alike) leave nothing to remember. A lazy constant has no caller, so it must not be able to fail.

18.15 Smaller decisions

- `pass` for intentionally empty blocks (indentation needs *something*).
- `{:}` for an empty map: a bare `{}` would not say whether it is a map or a set; the colon is the map's signature. (`set{}` is the empty set.)
- `then` exists only for the one-line `if a then b else c`.
- `alias` for type aliases and unions of existing types; a variant name that collides with an existing type is an error, so `type Meters = Float` is rejected with a pointer to `alias`.
- `set` is a reserved word, so `set{…}` has exactly one reading.
- `.copy()` is the one duplication method: with no arguments it makes a plain copy, with field arguments a copy with changes (`.copy(age: 31)`). There is no `clone` and no `.with` (`with` is a keyword). `.copy()` comes from the built-in `Copyable` trait, which every type has except `Task`, `Cleanup` types and anything containing them; `Copy` (implicit copying) is a separate, narrower trait.
- The collection methods are `prefix(n)` / `prefix_while(p)` rather than `take(n)` / `take_while(p)`, because `take` is a keyword (§5.3); `prefix` and `skip` read as a pair.
- Field punning works in construction, calls and patterns alike: `Dog(name:)`.
- Names are classified by their first character: uppercase-first names are types and variants; all others (lowercase, `_`, or uncased scripts) are values. Constants are `snake_case`.
- `print()` with no argument prints an empty line (`value` defaults to `""`).
- Doc comments: a `##` block followed by a blank line is the module doc; one directly above an item documents that item.
- Generic `impl` blocks declare their parameters: `impl<T> Box<T>`.
- A trait type such as `List<Speaker>` uses run-time dispatch, and the trait type itself implements the trait, so it can be passed to generic functions.

18.16 Standard library and tooling

Decision: A compact, consistently named standard library (`std.fs`, `std.net`, `std.text`, `std.time`, `std.json`, `std.task`, …) and a single `verse` tool with a zero-configuration formatter, built-in test runner, doc generator and package manager with a lock file and no install scripts.

Why: Consistency across the ecosystem is part of readability: every Verse project is formatted, tested and built the same way.

18.17 Visually optional types

Decision: Static typing with annotations optional almost everywhere, including parameters and return types. Unannotated parameters become implicit generics with exactly the constraints their bodies need (trait and structural); numeric literals default to `Int`/`Float`; recursion requires annotations; fallible return types are inferred from `?`; ambiguity is a compile error at the exact spot, listing the inferred constraints. Record fields keep annotations unless a default implies the type; `mut`/`take` are always written. Exported signatures may be inferred only with a committed `verse.api` that the build checks, and inference never crosses a module boundary.

Why: Readers of prose do not want type noise, but programs still deserve a compiler that catches mistakes before they run. Implicit generics give the brevity of a dynamic language with static checking; the "never guess" rule and the exact-spot errors keep inference predictable; and `verse.api` gives libraries a stable, reviewable public surface even when the source omits types. Editors show inferred types inline, so the information is one glance away.

18.18 Borrowed sugar

Verse borrows a handful of small conveniences from other languages, each narrowed so that it has exactly one reading:

Sugar	Form in Verse	Narrowed how	Inspired by
Early exit on missing	`v = x otherwise return 0` (also `break`, `continue`)	exit only as the whole right operand; refutable binding patterns only in this form; with a fallible left side the error is discarded (use `.context(…)?` to keep it)	Swift `guard let`, Rust `let … else`
Uniform call syntax	`x.f(y)` calls `f(x, y)`	a method always wins; ambiguity is a compile error	D, Nim, Koka
Subject-less `match`	`match` + `n < 0 -> …` arms	final `_ ->` required; an expression	Kotlin `when`, Lisp `cond`
Spread in literals	`[0, ..rest]`, `{..defaults, "port": 80}`	prefix only; later map keys win	JavaScript, Python
Power operator	`a ** b`	right-assoc, tighter than unary minus; Int overflow-checked; negative Int exponent rejected	Python, Fortran
Key paths	`users.map(.age)`	plain access chains only; no operators; no leading-dot variants	Swift key paths
Lambda closing rule	`orders.map(order ->` + indented body	only when the lambda is the last argument; the dedent closes the `(`	—
Postfix guards	`continue if line.is_empty()`	only `return`/`break`/`continue`; one line; no `else`, no `unless`	Ruby, Perl

Why: each one removes a common bit of noise (a nested `if`, a temporary variable, a lonely `)`), and none introduces a second meaning for existing syntax.

18.19 Rejected sugar

Proposal	Why it was rejected
List comprehensions (`[x * 2 for x in xs if x > 0]`)	a second way to write `filter`/`map` chains, which already read left to right; nested comprehensions read inside out
Implicit lambda parameters (`$0`, `it`)	anonymous names hurt reading; key paths cover the common short case and named parameters cover the rest
Leading-dot variant names (`.north`)	would make a leading dot mean two things (key path or variant); variants are short enough to write
Multi-clause function heads (`fn fact(0) = 1`, `fn fact(n) = …`)	spreads one function over several declarations; `match` in one body is clearer and keeps one signature for inference and `verse.api`
`?` on method names (`items.empty?()`)	`?` already has three related meanings about missing or failed values (§1.2); a fourth, unrelated one would undo that
A pipe operator (`|>`)	uniform call syntax already gives left-to-right chains with the existing dot
A `guard` keyword	`x = … otherwise return` does the same job with existing words

19. Verse vs other languages

	Verse	Python	Rust	Go	Swift	Kotlin
Blocks	indentation	indentation	braces	braces	braces	braces
Typing	static; annotations optional, even on parameters	dynamic (optional hints)	static, inferred	static, light inference	static, inferred	static, inferred
Missing values	`T?`, no null	`None` anywhere	`Option<T>`	`nil` pointers	`T?`	`T?`
Errors	`T | E` + `?`	exceptions	`Result<T, E>` + `?`	`(T, error)` returns	`throws` / `try`	exceptions / `Result`
Default mutability	immutable (`x = …` / `mut x = …`)	mutable	immutable (`let` / `let mut`)	mutable	`let` / `var`	`val` / `var`
Data semantics	single owner; borrow / `mut` / `take`; explicit `.copy()`	references	ownership & borrowing	values + pointers	structs by value, classes by ref	references
Memory	ownership, no GC; second-class borrows, no lifetimes	automatic (RC + GC)	ownership, no GC	tracing GC	ARC	tracing GC (JVM)
Concurrency	`spawn` / `.wait()`, no coloring	threads, `async`/`await`	threads, `async`/`await`	goroutines, channels	`async`/`await`, actors	coroutines (`suspend`)
Generics	`<T: Trait>`	duck typing / hints	`<T: Trait>`	`[T any]`	`<T: Protocol>`	`<T : Bound>`
Interfaces	`trait` + `impl … for`	classes, protocols	`trait` + `impl … for`	implicit interfaces	protocols	interfaces
Built-in tests	`test "name"` blocks	library (`unittest`)	`#[test]` functions	`_test.go` files	XCTest library	JUnit library
Official formatter	`verse fmt`, no options	external (`black`)	`rustfmt`	`gofmt`	`swift-format`	`ktlint` (external)
Script style	yes	yes	no	no	yes (scripts)	scripts (`.kts`)

In one sentence: Verse aims for **Python's readability, Rust's safety model, Go's simple concurrency, and Swift's optionals**, while keeping punctuation to a minimum and

spelling out logic and fallbacks in words.

Compared with Rust specifically: Verse gives the same core guarantees — no garbage collector, deterministic freeing, no use-after-free, no data races — with no lifetime syntax, no reference types and no elision rules to learn. The price is that borrows are second-class: a function cannot return or store a reference, so some Rust patterns (returning a view into a structure, structs that hold references, tasks borrowing from their parent) become a `.copy()`, a `mut` operation in place, an index, or a `Shared<T>` handle in Verse.

---

20. Complete example programs

Each program below is a complete, runnable entry file. Run one with `verse run file.vs`; run its tests with `verse test file.vs`, which checks the functions and types but never runs the top-level program (§14.1) — so the tests below read no files, touch no network, and never call `exit`.

20.1 CLI word counter

```
## wordcount — show the most common words in one or more text files.
##
## Usage: verse run wordcount.vs [--top N] FILE...

import std.fs

type Options
    top: Int = 10
    files: List<Text> = []

## Reads command-line arguments into Options.
fn parse_args(raw: List<Text>)                  # returns Options | Error (inferred)
    mut options = Options()
    mut i = 0
    while i < raw.len()
        match raw[i]
            "--top" ->
                value = raw.get(i + 1).or_error("--top needs a number")?
                options.top = value.to_int().context("--top expects a number")?
                i += 2
            flag if flag.starts_with("--") ->
                return error("unknown option: {flag}")
            file ->
                options.files.push(file.copy())  # `raw` is borrowed: keep a copy
                i += 1
    if options.files.is_empty()
        return error("usage: wordcount [--top N] FILE...")
    options

## Counts words case-insensitively, ignoring surrounding punctuation.
fn count_words(text: Text): Map<Text, Int>
    mut counts: Map<Text, Int> = {:}
    for word in text.lower().words()
        clean = word.trim_matches(c -> not c.is_letter())
        continue if clean.is_empty()
        # `clean` is a view of `word`; the map must own its keys.
        counts[clean.copy()] = counts.get(clean, default: 0) + 1
    counts

# ---- program ----

options = parse_args(args)?

mut totals: Map<Text, Int> = {:}
mut word_total = 0
for path in options.files
    text = fs.read_text(path).context("reading {path}")?
    for (word, n) in count_words(text)
        totals[word] = totals.get(word, default: 0) + n
        word_total += n

file_label = if options.files.len() == 1 then "file" else "files"
print("{word_total} words, {totals.len()} distinct, in {options.files.len()}
{file_label}")
print()

top = totals
    .entries()
    .sort_by(.value, descending: true)
    .prefix(options.top)

for (rank, entry) in top.enumerate()
    print("{rank + 1:>3}. {entry.key:<20} {entry.value:>6}")

# ---- tests ----
```

```
test "counting ignores case and punctuation"
    counts = count_words("The cat. The CAT, the end!")
    assert counts["the"] == 3
    assert counts["cat"] == 2
    assert "end!" not in counts

test "parse_args reads --top and files"
    options = parse_args(["--top", "3", "a.txt", "b.txt"])?
    assert options.top == 3
    assert options.files == ["a.txt", "b.txt"]

test "parse_args requires at least one file"
    assert parse_args([]) is _: Error
```

Sample run:

```
$ verse run wordcount.vs --top 3 poem.txt
112 words, 74 distinct, in 1 file

  1. the                       9
  2. and                       6
  3. of                        4
```

20.2 Concurrent web fetcher

```
## fetch — download several pages at once and report on each.
##
## Usage: verse run fetch.vs [URL...]

import get from std.net
import TaskPanicked from std.task
import std.time

type Report
    url: Text
    status: Int
    bytes: Int
    elapsed: time.Duration

## Fetches one page. Runs as an ordinary function; `spawn` makes it concurrent.
## Takes `url` because the report keeps it.
fn fetch(take url): Report | Error
    started = time.now()
    response = get(url, timeout: 10.seconds()).context("fetching {url}")?
    Report(
        url:,
        status: response.status,
        bytes: response.body.bytes().len(),
        elapsed: time.now() - started,
    )

fn describe(report)
    mark = if report.status < 400 then "ok " else "bad"
    "{mark} {report.status}  {report.url}  {report.bytes} bytes in {report.elapsed}"

# ---- program ----

defaults = ["https://example.com", "https://example.org", "https://example.net"]
urls = if args.is_empty() then defaults else args

print("Fetching {urls.len()} pages…")
started = time.now()

# Start every download at once; each `spawn` returns a Task immediately.
# `url` is borrowed from `urls`, so each task gets its own copy.
tasks = urls.map(url -> spawn fetch(url.copy()))

mut succeeded = 0
# `zip` can hand over the tasks only because `tasks` is at its last use here:
# a `Task` cannot be copied, and `wait` below takes each one.
for (url, task) in urls.zip(tasks)
    # Errors from the task surface here, at `.wait()`.
    match task.wait()
        report: Report ->
            succeeded += 1
            print("  {describe(report)}")
        crash: TaskPanicked ->
            # The panic and its trace were already printed when it happened.
            print("  crash    {url}  (bug in fetch: {crash.message()})")
        err: Error ->
            print("  err      {url}  {err.message()}")

print("{succeeded}/{urls.len()} succeeded in {time.now() - started}")
if succeeded < urls.len()
    exit(1)

# ---- tests ----

test "describe marks client errors as bad"
```

```
    report = Report(url: "x", status: 404, bytes: 0, elapsed: 5.millis())
    assert describe(report).starts_with("bad 404")
```

Because `fetch` is an ordinary function, the same code works sequentially (`fetch(url)?`) or concurrently (`spawn fetch(url)`), with no `async` variant. In both cases `fetch` takes its `url`, so a caller that still needs the text passes `url.copy()`. The three requests overlap, so the total time is close to the slowest single request.

20.3 Shape areas with traits

```
## shapes — areas and perimeters with variants, records, traits and generics.

import std.math

type Shape = Circle(radius: Float) | Square(side: Float) | Rect(width: Float, height:
Float)

type Triangle
    a: Float
    b: Float
    c: Float

## Anything with an area and a perimeter.
trait Measurable
    fn name(self): Text
    fn area(self): Float
    fn perimeter(self): Float

    fn summary(self): Text
        "{self.name():<10} area {self.area():>7.2}   perimeter {self.perimeter():>6.2}"

impl Measurable for Shape
    fn name(self): Text
        match self
            Circle(_) -> "circle"
            Square(_) -> "square"
            Rect(_, _) -> "rectangle"

    fn area(self): Float
        match self
            Circle(r) -> math.pi * r * r
            Square(s) -> s * s
            Rect(w, h) -> w * h

    fn perimeter(self): Float
        match self
            Circle(r) -> 2.0 * math.pi * r
            Square(s) -> 4.0 * s
            Rect(w, h) -> 2.0 * (w + h)

impl Triangle
    ## Builds a triangle, rejecting side lengths that cannot close.
    fn new(a: Float, b: Float, c: Float): Triangle | Error
        if a + b <= c or a + c <= b or b + c <= a
            return error("sides {a}, {b}, {c} cannot form a triangle")
        Triangle(a:, b:, c:)

impl Measurable for Triangle
    fn name(self): Text
        "triangle"

    fn area(self): Float
        # Heron's formula
        s = self.perimeter() / 2.0
        (s * (s - self.a) * (s - self.b) * (s - self.c)).sqrt()

    fn perimeter(self): Float
        self.a + self.b + self.c

## The item with the largest area, if there is one.
fn largest<T: Measurable>(take items: List<T>): T?
    # `items` is owned and at its last use, so the single element moves out
    # and the rest of the list is freed (§9.5).
    items.sort_by(.area(), descending: true).first()
```

```
# ---- program ----

shapes: List<Measurable> = [
    Circle(radius: 1.5),
    Square(side: 2.0),
    Rect(width: 3.0, height: 1.25),
    Triangle.new(3.0, 4.0, 5.0)?,
]

print("Shapes")
for shape in shapes
    print("  {shape.summary()}")

total = shapes.map(.area()).sum()
print("Total area: {total:.2}")

if largest(shapes) is biggest?             # takes `shapes`: its last use
    print("Largest:    {biggest.name()}")

# ---- tests ----

test "a square of side 2 has area 4"
    assert Square(side: 2.0).area() == 4.0

test "a 3-4-5 triangle has area 6"
    t = Triangle.new(3.0, 4.0, 5.0)?
    assert t.area() == 6.0
    assert t.perimeter() == 12.0

test "impossible triangles are rejected"
    assert Triangle.new(1.0, 1.0, 5.0) is _: Error
```

Output:

```
Shapes
  circle     area    7.07   perimeter   9.42
  square     area    4.00   perimeter   8.00
  rectangle  area    3.75   perimeter   8.50
  triangle   area    6.00   perimeter  12.00
Total area: 20.82
Largest:    circle
```

---

21. Review log and resolved decisions

After the first draft, a review pass revised several surface details. The revisions below are **final** and are applied throughout this document; they take precedence over anything in earlier drafts.

#	Topic	Resolved decision	Replaces
R1	Boolean operators	`and`, `or`, `not` are used **only** with `Bool` operands. There is no `&&`, `||` or `!`.	Earlier draft let `or` also act as a fallback.
R2	Fallback operator	The word **`otherwise`**: `find_user(id)?.email otherwise "no email"`, `read_text(p) otherwise "default"`.	`or` as fallback.
R3	`else`	`else` belongs only to `if`: block `if … / else` and the one-line `if a then b else c`.	—
R4	Fallible / union types	Written with `|`: `Int | Error`. Optional stays `T?`.	`Int or Error`.
R5	Return types	A colon: `fn greet(name: Text): Text`. Function types: `fn(Int): Int`.	`-> Ret`.
R6	Uses of `->`	Only match arms and lambdas. A lambda that is the body of a match arm must be parenthesized: `Double -> (n -> n * 2)`.	`->` also for return types.
R7	The `?` family	Postfix `?` = stop early if error or missing; `T?` = a type that may be missing; `?.` = chain past a missing value.	— (clarified)
R8	Indentation	4 spaces, tabs rejected, a block starts on the next line after its header, with no colon.	— (clarified)
R9	One-line conditional	`if a then b else c` (both branches required).	Multi-line `if` as the only expression form.
R10	Early exit	Explicit `return`; otherwise the last expression is the result.	— (clarified)
R11	Named arguments	`name: value`, the same shape as map entries and record construction.	— (clarified)
R12	Empty literals	`{:}` is an empty map, `set{}` an empty set, `[]` an empty list.	`{}` for an empty map.
R13	Task errors	Errors (and panics) inside a spawned task surface at `.wait()`, which returns `T | Error`, to be handled with `?`, `otherwise`, or `match`.	Panics re-raised at `.wait()`.
R14	Imported files	Contain declarations only; top-level statements run only in the entry file.	— (confirmed)

Consequences worked through during the review:

- **Variant vs union declarations.** Both use `|`. In `type Shape = Circle(…) | Square(…)`, the `|` introduces *new* variants. A union of *existing* types is named with `alias`: `alias Loaded = Config | Error`. Anywhere else in type position, `A | B` is a union.
- **Pattern alternatives** (`"y" | "yes" -> …`) also use `|`; they appear only in patterns, so there is no ambiguity with types.
- **Which members are errors.** In a union, members whose types implement the `Error` trait are the *error side*; the rest are the *success side*. `?` and `otherwise` act on the error side.
- **Precedence.** `otherwise` binds more loosely than every other binary operator, including `or`, so `a or b otherwise c` means `(a or b) otherwise c` — and is rejected anyway

unless `a or b` is optional. (Review 1 added a restriction on the right operand — R16 — and the `get(k, default: x)` form — R41.)
- **`then`** became a keyword for the one-line conditional.

Review 1 (Michelangelo)

Michelangelo's first review resolved the open items below. Each row names the review item and the section where the change lives; these rows take precedence over R1–R14 where they differ (notably R13 is refined by R17, and the implicit `?` of `with` from the first draft is withdrawn by R18).

#	Item	Resolved decision	Where
R15	1, G7	A variant name may not collide with a type in scope; `type Meters = Float` is an error that says "use `alias`".	§4.4, §17
R16	2	The right operand of `otherwise` must be a literal, call, postfix expression, parenthesized expression, or another `otherwise`; otherwise a compile error suggests brackets.	§3.8.2, §17
R17	3, N3	`TaskPanicked` is a named error type distinguishable in `match`; panic message and trace are always printed to stderr at the moment of the panic.	§8.5, §11.2
R18	4	`with open("log.txt")? as file: ?` is required; `with` never propagates by itself.	§13.1–13.4, §18.12
R19	§1	Punctuation stands for "one idea each"; multi-use marks (`:`, `{ }`, `?`) are listed honestly.	§1.2
R20	L1, G8	Record copy-with-changes is `.copy(field: value)`.	§4.3
R21	L2	`set` is a reserved word.	§3.6
R22	L3, G5	Constants are `snake_case`; uppercase-first names are types/variants; names starting with any other letter (including uncased scripts) or `_` are values.	§3.5, §17
R23	L4	A next line beginning with `|` continues a variant list; a `type X =` header never opens a field block.	§3.4, §4.4, §17
R24	L5	The interpolation format colon is the first colon outside any bracket or nested text.	§3.7.2, §17
R25	L6, G4	`spawn` and `if … then … else` removed from the precedence table; `spawn` is a prefix on a call.	§3.8, §17
R26	L7, G5	Open-ended ranges `a..`, `..b`, `..=b` only inside slices; `FLOAT` accepts `1e-9`.	§3.7, §6.4, §17
R27	L8	A `##` block followed by a blank line is the module doc; directly above an item it documents the item; `verse fmt` enforces the blank line.	§3.2, §14.3, §17
R28	T1	Map lookups and `list.get(i)` return a real `V?`; `T??` is not flattened except in `?.` chains.	§8.1, §9.3
R29	T2, E4	`?` and `otherwise` act on the outermost layer in written order (`Text? | Error`: error side first, then `none`).	§8.3
R30	T3, E4	A specific error type in a union with `Error` is absorbed by `Error`; `?` cannot narrow `Error` back.	§4.6, §8.3, §11.1
R31	F1	A lambda with *N* parameters may be called with an *N*-tuple (auto-spread).	§5.4, §18.7
R32	F2	Field punning in construction and calls: `Dog(name:)`.	§4.3, §5.3, §17
R33	C1	Loop labels are written before the loop: `outer: for row in grid`; `break outer`.	§6.5, §17, §18.8
R34	C2	`if let` removed; use `if x is pattern`.	§6.2, §7.6
R35	P1, G3	Inside parentheses `name: pattern` is always a field pattern; a typed binding is `field: n: Int`; top-level `n: Int` stays.	§7.3, §17

#	Item	Resolved decision	Where
R36	P2, G9	New pattern `p?` matches a present optional and binds it (`none -> …` / `user? -> …`, `Person(email: e?)`).	§7.3, §7.5, §17
R37	P3	An unconditional lowercase catch-all arm that is not last makes later arms an "unreachable arm" error.	§7.2
R38	E1	Discarding a fallible result (including containers such as `wait_all` results) is a compile error; `let _ =` is the escape hatch.	§1.3, §8.4, §11.2, §18.11, §17
R39	E2	`?` returns from the innermost lambda, spawned function, test, or script.	§5.4, §8.3, §11.1, §14.1
R40	E3	`?` on `T | E` inside a function returning `U?` is an error.	§8.3
R41	K1	Two-argument `get(key, default: x)` on maps and lists returns plain `V`; `default:` exists only on `get`.	§9.1–9.3, §20.1
R42	K2	`m[k]` (`V?`) and `list[i]` (panics) shown side by side; bare `{}` is an error pointing to `{:}` and `set{}`.	§3.7, §9.3, §17
R43	N1	`Cancelled` unwinds the task like a panic (running cleanup), cannot be caught by `otherwise` or `match` inside it, and is reported by `.wait()`; `sleep`, `receive`, `wait` have explicit signatures.	§11.5
R44	N2	An unwaited failed task makes the process exit non-zero.	§11.3, §12.5
R45	N4	`channel.receive()` returns `T?` (`none` when closed and drained).	§11.4
R46	M1	Record fields are hidden unless marked `export`; `private` removed from the keywords.	§3.6, §4.3, §12.3, §17
R47	M2	Imported-module `let` initializers must be infallible; entry-file top-level `let` is an ordinary statement.	§12.4
R48	M3	`fn main` dropped for 0.1; top-level statements are the only entry mechanism.	§1.5, §12.5
R49	X1	`verse test` never runs entry-file top-level statements.	§14.1, §20
R50	S1	`read` removed from the prelude; `open`, `create`, `read_text` live only in `std.fs`; `print(value: Show = "")`, so `print()` prints an empty line.	§15.1
R51	S2	The §8.2 example variant is renamed `NoResponse` to avoid clashing with `std.task.Timeout`.	§8.2
R52	G1	`( "_" | IDENT ) ":" type` typed patterns in the grammar.	§17
R53	G2	`self.field = …` is a valid assignment place.	§5.6, §17
R54	G6	`from m import x as y` exists.	§12.2, §17
R55	§10, §19, ToC	`Add`/`Sub`/`Mul`/`Div`/`Rem`/`Neg` rows in §10.4; softened the "fewer symbols" claim; §21 added to the contents.	§10.4, §19, Contents
R56	addendum	Method calls are unmarked; the receiver of a `mut self` method must be a `let mut` binding (compile error otherwise).	§5.6

Round 2 — follow-up decisions

#	Topic	Resolution	Where
Q1	bare `spawn`	A bare `spawn worker(…)` statement that discards its `Task` is an accepted exception to "fallible results cannot be dropped": its outcome is still awaited and reported (§11.3).	§11.3
Q2	exit status	An explicit `exit(code)` always wins; the non-zero status for a failed unwaited task applies only on natural program end.	§11.3, §12.5, §18.13
Q3	`??`	Postfix `??` is not an expression (write `(x?)?`); `x??` exists only as a pattern meaning present-and-present.	§8.3, §7.3, §17

Round 2 — user requirements

#	Topic	Resolution	Where
U1	Visually optional types	Annotations are optional on parameters and return types (`fn greet(name)`); unannotated parameters are implicit generics with the constraints the body needs; numeric literals default to `Int`/`Float`; recursion needs an annotation; ambiguity is a compile error at the exact spot listing the inferred constraints, never a guess; fallible returns inferred from `?`; editors show inline hints and `verse fmt` never inserts annotations; record fields keep annotations unless a default implies the type; `mut`/`take` always explicit. The old "annotations required on all parameters" rule is removed.	§2, §4.2, §5, §16.1, §17, §18.17
U1a	Exported signatures	May be inferred, only with (a) a committed `verse.api`, (b) a build that fails when an inferred exported signature differs until `verse api update` is run, and (c) cross-module checking against the interface file only. There is no "export requires explicit types" rule.	§4.2, §12.3, §16, §16.4, §17
U2	No garbage collection	Single ownership, deterministic freeing at end of scope, no references/pointers or lifetime syntax; parameters borrow read-only by default, `mut` (declaration and plain-call site) borrows exclusively, `take` consumes; borrows are second-class (not stored, returned, or sent to tasks); moves per control-flow path with use-after-move errors suggesting `.copy()`; scalars and records made only of copy types copy implicitly. RC, cycle collector and copy-on-write removed.	§1.3, §4.9, §4.10, §5.3, §18.9, §18.10, §19
U2a	`.copy()`	`.copy()` with no arguments is the plain duplicate, the same method as `.copy(age: 31)`; there is no `clone`.	§4.3, §4.9, §18.15
U2b	Lambdas	Non-escaping lambdas (`map`, `filter`, …) borrow captures read-only; escaping lambdas (returned, stored, `take`n, spawned) move or copy them.	§5.4
U2c	Spawn	`spawn f(x)` moves `x`; borrowed values cannot cross into a task; spawned parameters are owned; reuse after spawn (also in a loop) is an error suggesting `.copy()`; handles copy cheaply; no borrowing from the spawner in 0.1.	§11.1
U2d	Sharing	Opt-in only: `Shared<T>` (shared read-only, atomically reference-counted), `Guarded<T>` (mutable behind `.lock()`), `Weak<T>` (`upgrade(): T?`), `Channel<T>`, `Task<T>`. Cycles are possible only through these; graphs use `Shared` or arenas. Recursive types are boxed by the compiler.	§4.10, §11.4
U2e	Cleanup	`Closeable` renamed `Cleanup`; `cleanup(mut self)` runs automatically at end of scope (also during a panic) and must not fail; `with … as` explicitly limits the scope.	§10.4, §13, §18.12
U2f	`take` keyword	`take` is reserved; the collection method `take(n)` is renamed `prefix(n)` (`prefix_while(p)`). Assignment evaluates its right side before the place, so `m[k] = m.get(k, default: 0) + 1` is valid.	§3.6, §4.9, §9.5

Sugar round

#	Feature	Resolution	Where
S-1	`otherwise return`	`let v = x otherwise return X` (or `break`/`continue`): the exit must be the whole right operand (`x otherwise (return 1) + 2` is an error); unwraps `T?` to `T`; a refutable `let` pattern is allowed only in this form; with a fallible left side the error is discarded — use `.context(…)?` to keep it. A deliberate exception to the right-operand rule. No `guard`.	§3.8.2, §5.2, §7.7, §17
S-2	Uniform call syntax	`x.f(y)` calls an in-scope `f(x, y)` when the first parameter accepts `x`; a method always wins; ambiguity is a compile error; parameter modes carry over. No `|>`.	§5.7, §17
S-3	Subject-less `match`	Arms are `Bool` conditions; a final `_ ->` arm is required ("not exhaustive" otherwise); it is an expression.	§7.8, §17
S-4	Spread in literals	`[0, ..rest]`, `set{..a, x}`, `{..defaults, "port": 80}`; prefix `..` never clashes with infix ranges; later map keys win.	§3.7, §17
S-5	`**`	Power operator: right-associative, tighter than unary minus (`-2 ** 2` is `-4`), overflow-checked for `Int`; a negative `Int` exponent is a compile error.	§3.8, §17
S-6	Key paths	A leading dot that starts an expression is a one-parameter lambda (`.name`, `.name.upper()`, `.0`); plain access chains only (`filter(.age >= 18)` is an error suggesting a lambda); after a complete expression a leading dot continues a chain; no leading-dot variants.	§3.4, §5.4, §17
S-7	Lambda closing rule	When a line inside an open `(` ends with `->`, the dedent ending the lambda body also closes that `(`; the lambda must be the last argument. No parenless trailing lambdas.	§3.4, §5.4, §17
S-8	Postfix guards	Only `return`, `break`, `continue` take a postfix `if cond`; one line, no `else`; no `unless`, no general postfix `if`.	§6.7, §17
S-9	Rejected	List comprehensions, `$0`/`it`, leading-dot variants, multi-clause function heads, `?` on method names, `|>`, `guard` — with reasons.	§18.19

Review 2 (Michelangelo)

Every item of the second review was accepted as proposed. These rows take precedence over earlier rows where they differ (notably X1 replaces the "filter copies its elements" rule of §9.5).

#	Item	Resolved decision	Where
U2f-1	`prefix/skip`	`take(n)/take_while` renamed `prefix(n)/prefix_while` — agreed; the keyword aside moved from the §9.5 table to §18.15.	§9.5, §18.15
U2f-2	Assignment order	Right side first; receiver and index expressions on the left after it, left to right among themselves; everything else left to right.	§4.9
X1	Chains pass views	Adapters (`filter`, `sort`, `sort_by`, `prefix`, `skip`, `unique`, `zip`, `enumerate`, `reverse`, ...) pass views; the chain ends in an owning step (`map`, a reduction, a move from a source at its last use, or `.copy()`); `find/first/last/min/max` return views; a `let` bound to a chain is a local view; implicitly duplicating a non-`Copy` element is a compile error suggesting `.copy()`. Tour line: `people.filter(p -> p.age >= 18).map(.name.copy())`.	§2, §4.9, §9.5, §18.10
X2	Flip errors	Every error caused by a view/owner flip names the later use that kept the source alive.	§4.9, §18.10
X3	Generic ownership; `Copyable`	The ownership check runs once, at the definition, treating unannotated/generic parameter types as opaque non-`Copy` types unless bounded. New built-in trait `Copyable` provides `.copy()`, derived structurally, absent for `Task`, `Cleanup` types and anything containing them; `Copy` stays the implicit-copy marker. `first<T: Copyable>` uses `items.get(0)?.copy()`.	§4.2, §4.3, §5.5, §10.4, §10.5, §10.6, §15.1, §18.10, §18.15
X4	Argument order	Arguments are evaluated first; then the exclusive borrow for `mut` parameters and `mut self` begins. The §4.10 `Tree.add` example is valid under this rule.	§4.10, §5.3, §5.6
L-1	Spread	Spread moves from a source at its last use; otherwise a compile error suggesting `..xs.copy()`.	§3.7
L-2	Refutable `let`	With a refutable pattern, the left side of `otherwise` may be any type.	§3.8.2
L-3	`t.0.1`	After `.`, a digit sequence is always one `INT`.	§3.7, §17
L-4	Format colon	An interpolated expression with its own top-level colon needs parentheses: `"{(v is n: Int)}"`.	§3.7.2, §17
L-5	`return if`	`if` right after `return/break/continue` is always a postfix guard; parenthesize a conditional value.	§3.8, §17
T-1	"small records"	Dropped: `Copy` is structural with no size limit.	§4.9, §9.3
T-2	Partial moves	Moving a field out of an owned value at its last use is allowed; the rest is freed.	§4.9, §11.1
F-1	Durations	`10.seconds()`, `5.millis()` everywhere.	§15.2, §20.2
F-2	Closing rule details	One dedent closes every block-lambda parenthesis on the lines it ends; nothing may follow on that line; an explicit `)` is accepted and removed by `verse fmt`.	§5.4
E-1	`ParseError`	Defined in `std.num`; `Text.to_int()` returns `Int | ParseError`.	§8.2, §15.2
K-1	`take` leftover	§9.5 ownership text uses `prefix`; keyword aside removed from the table.	§9.5

#	Item	Resolved decision	Where
K-2	Key functions	`sort_by`/`min_by`/`max_by` keys may be views; `group_by` keys must be owned.	§9.5
K-3	Laziness	No lazy iterator value can be stored or returned; a `let` bound to a chain is a local view.	§9.5
K-4	Slices	Slices are views; `.copy()` makes an owned list or text.	§9.3
P-1	Variant-like catch-all	Warning for a catch-all binding named like a lowercase variant, even in last position.	§7.2
C-1	Lock guards	The guard implements `Cleanup`; `with` on a guard binds the value inside as an exclusive borrow — the only such case.	§11.4, §13.1
C-2	`spawn f(mut x)`	A compile error.	§11.1
M-1	Field/method names	Fields and methods share one namespace per type; example field renamed `cents`.	§12.3
M-2	Script locals	Entry-file top-level `let`s are not visible inside `fn` bodies; pass them as arguments.	§12.4
G-1	One-line arms	`arm_body` allows `arm_expr`, an exit, `pass`, or an assignment.	§17
G-2	Lone-name guards	Guards and subject-less conditions use `expr - lambda`.	§17
G-3	Qualified names	`base_type` takes `{ IDENT "." } TYPE_IDENT`; `postfix` takes `"." TYPE_IDENT`.	§17
G-4	`let … otherwise`	The second `let_stmt` form takes `pattern`, not `top_pat`.	§17
G-5	Inferred constraints	Indexing gives `Index`; calling a parameter makes it callable.	§4.2, §10.4
D-1	"by vote"	Reworded to "settled in the design discussion".	§18
D-2	`?` meanings	"Four related meanings".	§18.19
D-3	Trade-offs	§18.10 states the chain rule, flip errors and the generic check.	§18.10
D-4	Log	This table; the Review 1 intro no longer calls it "the second review".	§21
EX-1	`largest`	`fn largest<T: Measurable>(take items: List<T>): T?`; `shapes` is at its last use.	§20.3
EX-2	Word keys	`counts[clean.copy()] = …`.	§20.1
EX-3	Boundary types	`raw: List<Text>`, `text: Text` annotated.	§20.1
EX-4	`zip(tasks)`	Comment: works because `tasks` is at its last use.	§20.2
S	Prelude `Error`	`Error` listed only as a trait; a trait name in type position means "any implementing type".	§15.1
N-1	`otherwise` and ownership	Literals and other static data count as views when needed; `a otherwise b` is a view when both sides are views (`args.first() otherwise "world"`); with an owned non-static temporary on the	§3.8.2, §8.3

#	Item	Resolved decision	Where
		right the result is owned and a view on the left needs `.copy()` (compile error otherwise, suggesting it). Tour and §12.5 examples already conform.	
N-2	`get(k, default: x)`	Returns a view of the element when present, of the default otherwise (static literals fine); owned positions need `.copy()`.	§9.3, §9.5
N-3	Guard cross-reference	§6.7 states that an exit followed by `if` is always a guard; write `return (if c then a else b)`.	§6.7
N-4	`find_user`	The M-2 comment (`users` is a module constant) is kept as is.	§8.1
N-5	`Copyable` closed	`Copyable` cannot be implemented by hand in 0.1; `Cleanup` types are deliberately non-copyable; a type needing duplication offers an explicit method such as `duplicate()`.	§10.4

Review 3 (Michelangelo)

Re-review of the round-3 changes: all of Review 2 landed; N-3, N-4 and N-5 were agreed as stated, and N-1 and N-2 with the amendments below. All items were accepted.

#	Item	Resolved decision	Where
R3-1	`otherwise` table	New row: owned left + non-static view right is an error asking for `.copy()` on the right. A literal in an owned position is materialised — the one allowed implicit creation, since it duplicates no user value. `?.` on an owned temporary yields an owned field. `if c then a else b`, block `if` values and `match` arms follow the same rule: a view only if every branch is a view; otherwise owned, and view branches need `.copy()`.	§3.8.2, §6.1, §8.3
R3-2	`get(k, default: x)`	Same table as `otherwise`: a view with a view or static default; an owned temporary default (`Text.new()`) makes the result owned and the element side needs `.copy()`.	§9.3
R3-3	Single element from an owned source	On an owned source at its last use (`take items`, a temporary) the single element moves out and the rest is freed — the owned counterpart of the view rule.	§9.5, §20.3
R3-4	Collecting steps	`to_list`, `to_set`, `to_map`, `flat_map`, `flatten` are owning steps (move at last use, else `.copy()`); `chunks` and `windows` are views of slices.	§9.5
R3-5	Handles	`Shared`, `Guarded`, `Channel` are `Copyable`; `.copy()` copies the handle, not the value. Passing a handle to `spawn` needs an explicit `.copy()`; examples fixed.	§10.4, §11.1, §11.4, §15.2
R3-6	Handles always `Copyable`	`Shared<T>`, `Guarded<T>`, `Channel<T>`, `Weak<T>` are `Copyable` whatever `T` is, including resources (`Guarded<File>`), because copying a handle never duplicates the value; the "anything containing a `Cleanup` type" exclusion applies only to containment by value.	§10.4, §11.4
R3-7	`Weak<T>`	`Weak<T>` is `Copyable` (copies the weak handle) and needs an explicit `.copy()`, listed with the other handles.	§10.4, §11.1, §11.4, §15.2
R3-8	Explicit handle copies	§4.10 point 5: copying a handle must be written as `.copy()`.	§4.10
R3-9	Owned element, owned default	`names.get(id).copy() otherwise Text.new()` is the intended form; `.copy()` on an optional view `V?` of a `Copyable V` yields an owned `V?`.	§9.3
R3-10	`Weak` type parameter	`Weak` is parameterised over the handle type: `Weak<Shared<T>>` or `Weak<Guarded<T>>`, created with `h.downgrade()`. Supersedes the `Weak<T>` spelling in rows U2d, R3-6 and R3-7 above.	§4.10, §10.4, §11.1, §11.4, §18.10
R3-11	`upgrade()` type	`upgrade()` returns `Shared<T>?` or `Guarded<T>?` respectively (`none` if the value is gone).	§11.4
R3-12	`Weak<Channel<T>>`	`Weak` accepts only a `Shared` or `Guarded` handle; `Weak<Channel<T>>` is a compile error (channels are closed by dropping the last sender, not by weak reference).	§11.4

Review 4 / Rafael's changes

Three language changes proposed by Rafael and agreed by Michelangelo, applied throughout the spec (every section, grammar, tour, examples, stdlib table). Earlier rows above are kept as history; where they use `let`, `?.` or the `from m import x` form, the rows below supersede them (in particular the `?.` wording of R3-1).

#	Item	Resolved decision	Where
RF-1	Remove `let`; keep `mut`	`x = expr` binds a new immutable name, `mut x = expr` a mutable one; `x = expr` on a visible `mut` name reassigns, on a visible immutable name it is an error ("`x` is immutable; write `mut x = …` where it is declared, or pick a new name"). Typed: `n: Int = 5`, `mut n: Int = 0`.	§1.3, §3.9, §4.9, grammar
RF-1a	Destructuring	`(a, b) = pair`; refutable patterns only with `otherwise return/break/continue` (`Circle(r) = shape otherwise return 0.0`); constructor patterns are told apart from bindings by capitalisation.	§3.8.2, §3.9, §7.7, §18.7, grammar
RF-1b	`mut` means "can change"	Same meaning on bindings, parameters and `mut self`.	§3.9, §4.9, §5.3, §5.6, §18.9
RF-1c	No shadowing	A name is bound once per function; parameters, loop variables and pattern bindings may not reuse a visible name; sibling blocks may reuse a name; entry-file top-level bindings keep the M-2 rules. §18.9 rewritten (shadowing would be indistinguishable from reassignment). Supersedes the earlier "inner block may shadow" rule.	§1.3, §3.9, §5.3, §12.4, §18.9, grammar
RF-1d	Diagnostics	Unused binding warns; a new binding within edit distance 1 of a visible `mut` name warns ("did you mean to reassign `total`?").	§3.9, §18.9
RF-1e	`let` reserved	Stays in the keyword list with the error "`let` is not used in Verse; write `x = …`".	§3.6, grammar
RF-1f	Rewrites	All `let`/`let mut` uses rewritten; one renamed binding in §3.8.2 (`n` → `count`).	all sections
RF-2	Remove `?.`	Plain dots look through optionals only when the chain is immediately consumed by `otherwise`, postfix `?`, an `is` pattern / `match` subject, or an optional-typed position (annotated binding, `return` in an optional-returning function, optional parameter). Elsewhere a dot on `T?` is an error suggesting `otherwise`/`?`/`is` (and annotating `T?` for an unannotated binding).	§3.4, §3.8, §8.1, §16.2, grammar
RF-2a	Lifted-chain rules	Short-circuits entirely on the first `none`; one flattened optional level (lookups like `m.get(k)` not flattened); ownership follows the view rule; the optional's own methods (`.expect()`, `.copy()` on `V?`, …) win over lifting; a name that is both an optional method and a field of the inner type is an error.	§8.1, §3.8.2, §9.3, §9.5
RF-2b	`?` has three jobs	The `T?` type, postfix propagation, the `x?` pattern.	§1.2, §8.1, §18
RF-2c	Examples and rationale	All `?.` examples rewritten (including `x?.copy()` in §9.3); §18 explains the drop (less noise, same safety, the consumer decides).	§2, §3, §5.5, §8, §9, §10, §16, §18
RF-3	Imports	`import get, post from std.net`; `import parse as parse_json from std.json`; per-name `as` in lists; `import std.fs` and `import std.json as js` unchanged; every import line starts with `import`.	§12.2, §18.14, grammar

#	Item	Resolved decision	Where
RF-3a	Examples	All item imports rewritten (sections, tour, §11/§20 examples, stdlib table, packages).	§2, §8, §11, §12, §13, §15, §16, §18, §20
RF-1g	Confirmed	A leading `_` suppresses the unused-binding warning; parameters count as bindings.	§3.9, §5.3
RF-4	Import parsing	Module vs item import (`import a as b` / `import a as b from m`) is decided by the trailing `from` on the same logical line; an import fits on one line, or uses a parenthesised item list that may span lines.	§12.2, grammar
RF-5	Reassignment visibility	One form `x = …` is kept; editors and `verse fmt --check` tooling must render reassignments distinctly from new bindings (inlay hint "reassign"). No syntax change.	§3.9
RF-6	Scope of the no-shadowing rule	You cannot bind a name that is currently visible: a name bound inside a block may be bound again after the block ends; a name bound before a block cannot be rebound inside it. Example added.	§3.9
RF-7	Lambda parameters	Count as bindings; they cannot reuse a visible name.	§3.9
RF-8	Assigning a `mut` parameter	`list = […]` replaces the caller's value in place (the parameter is the caller's place under an exclusive borrow).	§3.9, §5.3
RF-9	Unused-binding warning	Warning only; illustrative snippets are exempt; the spec's examples are not compiled with warnings-as-errors.	§3.9
RF-10	`.or_error` on a lifted chain	Applies to the final result: a `none` anywhere becomes the error.	§8.1
RF-11	Significant parentheses	`verse fmt` never removes semantically significant parentheses; `(find_user(id)?).email` stays; parentheses end a chain, so `?` inside them propagates there.	§8.1, §16.1
RF-12	`MAX_RETRIES = 3`	Error reworded: names are `snake_case`; capitalised names are types and patterns.	§3.5
RF-13	`fn port(config)`	Kept; note that entry-file locals are not visible in functions (M-2), so this is not shadowing.	§12.4
RF-14	Channel closing	Ruled as "`close()` or last handle dropped"; superseded before it was applied by RF-15. The spec already read "last handle" in §11.4; no "last sender" wording existed outside the R3-12 history row, which is kept.	—
RF-15	Channel split	`Channel.new<T>()` returns `(tx, rx)`: `Sender<T>` and `Receiver<T>`. The channel closes when the last `Sender` is dropped; `tx.close()` is an early shortcut that closes for everyone. Supersedes the `Channel<T>` handle in R3-5…R3-7 and R3-12.	§11.4
RF-15a	Copyability	`Sender<T>` is `Copyable` (handle copy, explicit `.copy()`, also for `spawn`); `Receiver<T>` is not `Copyable` in 0.1 (single consumer; fan-out uses several channels) and moves into a task with `spawn`.	§4.10, §10.4, §11.1, §11.4,

#	Item	Resolved decision	Where
			§15.2, §18.10
RF-15b	Operations	`for value in rx` ends when the channel is closed and drained; `rx.recv(): T?`; `tx.send(v)` returns the error `Closed` if the `Receiver` was dropped or the channel closed. `Closed` added to `std.task`.	§11.4, §11.5, §15.2, §18.13
RF-15c	`Weak`	`Weak` accepts only `Shared`/`Guarded`; `Sender` and `Receiver` are not weakable.	§11.4
RF-15d	Examples and lists	§11.4 channel example rewritten (Senders copied per task, `tx` moved into `start_workers`, `for square in rx`); "channels" → "channel receivers" in the `Iterable` and `for` lists; §1.3 handle list.	§1.3, §6.3, §10.4, §11.4
RF-16	§3.9 error row	"a visible immutable binding or immutable parameter" — `mut` parameters are assignable (RF-8).	§3.9
RF-17	`with … as mut` on a lock	The name bound by `with g.lock() as value` is mutable only when written `as mut value`; §11.4 example fixed (`with counter.lock() as mut value`); rule stated with C-1 and in §3.9; grammar `resource` takes optional `mut`. Amends C-1.	§3.9, §11.4, §13.1, grammar
RF-18	Explicit type arguments	Allowed on associated-function calls through a type name (`Channel.new<Int>(capacity: 10)`); normally inferred from an annotation (`(tx, rx): (Sender<Int>, Receiver<Int>) = Channel.new(capacity: 10)`).	§11.4, grammar
RF-19	Channel capacity	`Channel.new<T>(capacity: Int = 16)`; `capacity: 0` is unbuffered (rendezvous).	§11.4
RF-20	Safe worker example	`start_workers(take tx, count)` spawns workers (each with `tx.copy()`) and returns the tasks without waiting; the caller consumes `for square in rx`, then waits. 20 items with capacity 4. New sentence: waiting for producers before consuming can deadlock once the channel is full; always consume while producers run.	§11.4
RF-21	Blocked senders	A sender blocked on a full channel wakes with `Closed` when the `Receiver` is dropped or `close()` is called.	§11.4
RF-22	`send` is fallible	Discarding its result is the usual compile error (§8.4).	§11.4
RF-23	§4.10 point 5	`Receiver<T>` removed from the "two owners share one value" list; `Sender<T>` kept.	§4.10
RF-24	`Closed`	A named error type in `std.task`, like `Timeout`.	§11.4, §15.2
RF-25	Postfix `?` then dot	Postfix `?` is an ordinary postfix operator applied left to right: `find_user(id)?.email` means `(find_user(id)?).email` (propagate, then read) and is not optional lifting. Parentheses are allowed and equivalent there; they matter only when they end a lifted chain, and `verse fmt` keeps those. Amends RF-11.	§3.8, §8.1, §16.1, grammar
RF-26	Type arguments only after a type name	`TYPE_IDENT "." IDENT type_args call`; a lowercase receiver never takes type arguments, so there is no ambiguity with chained comparison. Amends RF-18.	§3.8.3, grammar

#	Item	Resolved decision	Where
RF-27	Spawning in a chain	The §11.4 worker and `Guarded` examples end in `.to_list()`; `to_list` on a mapped range yields an owned `List<…>`, and `spawn` side effects run when the lazy chain is collected.	§9.5, §11.4
RF-28	`Guarded` example import	`import Guarded, wait_all from std.task`.	§11.4
RF-29	File writes need `mut`	`write_line`, `write`, `flush` take `mut self`; written files are bound `as mut target`.	§13.1, §13.3
RF-30	§4.10 point 5	`Task<T>` removed from the sharing list (tasks are moved, never shared).	§4.10
RF-31	No explicit type arguments on free functions	Inferred, or given by annotating the binding (`n: Int = parse(text)?`); `parse<Int>(…)` is a compile error with that hint.	§10.5, grammar
RF-32	Eager chains	A chain that is bound or returned is evaluated eagerly and collected into a list at the end of the statement (an owned `List<…>` when it ends in an owning step), and lambda side effects such as `spawn` run then; only a `for` target or a mid-chain use stays lazy. `.to_list()` is needed only where the chain would otherwise be consumed as something else, not for binding. RF-27's `.to_list()` additions reverted. Amends RF-27.	§9.5, §11.4

Review 4 (Michelangelo full read)

Rulings on `reviews/review-4.md` (plus R4-14, added by the reviewer). Where a row amends an RF row above, it supersedes it.

#	Item	Resolved decision	Where
R4-1	Scope of "visible"	Locals only (parameters, earlier bindings, loop/pattern/lambda names in scope). A local may hide a prelude name silently and may hide a module constant or imported name; hiding an imported name that the same function also uses warns. A file-level import hides a prelude name of the same spelling (`import error from std.log`); there is no prelude qualifier, so import `std.log` as a module to keep both. Amends RF-6.	§3.9, grammar
R4-2	`a?.b`	A `?` immediately followed by `.` is a compile error ("`?.` is not Verse; write `(a?).b` to propagate then read, or `a.b otherwise …` / `a.b?` to look through the optional"). `(a?).b` stays valid. Supersedes RF-25's "bare `x?.y` means `(x?).y`".	§3.8, §8.1, §16.1, §18.11, grammar
R4-3	Getters	An unannotated function that returns a field, element or lookup read through a borrowed parameter gets an inferred `Copy` bound on the returned type, recorded in `verse.api`; an unmet bound is a call-site error naming it and suggesting `.copy()` inside, an annotation, or `take`.	§4.2 rule 9, §5.3, §18.10
R4-4	Raw text in the `std.json` example	Example uses a multi-line `r"""…"""` literal; §3.7.3 expands the raw-text rule (no escapes, no interpolation; layout as `"""`; single-line `r"…"` cannot contain `"`). Raw text was already in the grammar (`RAW_TEXT`).	§3.7.3, §15.2
R4-5	Destructuring	A pattern always creates new bindings (an existing name is an error pointing to `mut`); a tuple made only of visible `mut` names is a parallel assignment, right side first, so `(a, b) = (b, a)` swaps; mixing is an error. Swap added to the tour.	§2, §3.9, §7.7, grammar
R4-6	Nested text in interpolation	One level deep; interpolation inside the nested literal is an error; style note: bind long ones to a name first (`verse fmt --check` suggests it).	§3.7.2, grammar
R4-7	§4.9 copy/move sentence	Rewritten: `Copy` depends on the kinds of fields, not size; a copy is never inserted silently except for `Copy` types and literals; moves happen at last use; adding a `Text` field makes a record non-`Copy` and the compiler reports the places that relied on the copy.	§4.9
R4-8	`10.seconds()`	A number followed by `.` and an identifier is an `INT` and a method call; a float needs a digit after the dot; `10.` alone is an error.	§3.7, grammar
R4-9	§8.1 ways out	"four ways out" (fallback, `?`, `is`, annotation); `.or_error` added to the tool table.	§8.1
R4-10	§3.8 row 1	Notes moved to a list below the table; meaning unchanged.	§3.8
R4-11	`Map.with_default`	Declined. The counting idiom `counts[word] = counts.get(word, default: 0) + 1` is already in §9.2; no edit.	§9.2
R4-12	Nested optionals	Style note: avoid `T??` in public APIs; prefer a variant; `none?` / `score??` patterns are for generic code.	§8.1

#	Item	Resolved decision	Where
R4-13	`.to_list()` in §11.4	Confirmed absent from the worker example (removed by RF-32, which made it unnecessary); no edit.	§11.4
R4-14	Where errors appear	One sentence, stated identically in §4.2 rule 9, §5.3 (by the getter example) and §10.6: "Errors about a function's body appear at the definition. The only call-site errors are unmet bounds, written or inferred, and they name the bound."	§4.2, §5.3, §10.6
R5-1	§4.2 rule 9 help text	Help now says "return `person.name.copy()`, or take the parameter with `take`" (annotating would move the error to the definition). Supersedes "an annotation" in R4-3.	§4.2
R5-2	§16.1 formatter sentence	Rewritten as one list.	§16.1
R5-3	Import hiding a prelude name	Warns on the import line: "`error` hides the prelude `error`" (e.g. `import info, error from std.log`). §16 has no diagnostics list, so it is stated in §3.9 only.	§3.9
R5-4	Pointers to the hiding rule	One line each in §12.2 and §15.1 pointing to §3.9.	§12.2, §15.1
R5-5	§18.9 summary	Clauses added: only locals count; tuple of `mut` names is a parallel assignment (swap).	§18.9